



FanZone Marketplace

מגיש: יואב נעמן

ת.ז: 209673755

שם המנחה: עידן פלדברג

שם בית הספר: תיכון אלון רמת השרון

שם החלופה: "הגנת סייבר"

תאריך הגשה: 3.6.2025



תוכן עניינים:

2	מבוא
2	ייזום:
2	אפיון:
4	תיאור תחום הידע
4	פירוט מעמיק של היכולות:
10	מבנה הפרויקט
10	תיאור הארכיטקטורה של המערכת:
11	טכנולוגיה רלוונטית:
12	זרימת המידע במערכת:
17	תיאור האלגוריתמים המרכזיים בפרויקט:
26	תיאור סביבת הפיתוח
26	תיאור פרוטוקול התקשורת
33	תיאור מסכי המערכת
42	תיאור מבני הנתונים
45	סקירת חולשות ואיומים
48	מימוש הפרויקט
48	סקירת כל המודולים/מחלקות המרכיבים את המערכת
60	קטעי קוד האלגוריתמים המרכזיים
76	מסמך בדיקות מלא
81	מדריך למשתמש
81	עץ קבצים
81	התקנת מערכת
90	רפלקציה
91	ביבליוגרפיה
92	קוד הפרויקט
92	main.py
97	ctk_ui.py
98	client_communication.py
102	encryption_client.py
104	user_introduction.py
121	main_windows.py
179	server.py
197	server_communication.py
204	database.py
216	encryption_server



מבוא

ייזום:

תיאור ראשוני של המערכת:

הפרוייקט FanZone Marketplace היא אפליקציה הממשת מעין חנות אונליין של מוצרי כדורגל וכדורסל של אוהדים ישראלים (כמו חולצות, צעיפים, כובעים וכדומה) רק שהאוהדים הם המוכרים וגם הקונים (דוגמא לממשק מסוג כזה היא האפליקציה e-bay). בחרתי לעשות את הפרוייקט הזה מכיוון שבתור אוהד ספורט בארץ אני עד לכך שכיום הדרך שבה נמכרים מוצרים בין אוהדים היא דרך קבוצות Whatsapp, אינסטגרם או פייסבוק, בקיצור בצורה מבולגנת שמביאה איתה הרבה בעיות הן בתקשורת בין הלקוח למוכר והן בתשלום. מטרת האפליקציה FanZone Marketplace היא ליצור מקום נוח ומרכז בו אוהדים יוכלו למכור מוצרים אחד לשני ולתת לאפליקציה לעשות את רוב העבודה שהם צריכים לעשות כשמוכרים בקבוצות.

אלו שירותים המערכת תיתן:

המערכת תספק שירותי פרסום מוצרים אשר יופיעו בפיד ראשי של האפליקציה ואופציה של קניית המוצרים דרך האפליקציה (מאחר ומדובר בפרוייקט בית ספרי ושירותי תשלום אמיתיים עולים כסף, השתמשתי בתשלומים פיקטיביים אך ניתן ליישם מנגנון תשלומים בתוך הקוד הקיים).

תיחום הפרוייקט:

הפרוייקט עוסק בקיום ממשק גרפי שאותו ניתן להריץ בכל מחשב (כלומר האפליקציה לא יכולה לרוץ על טלפונים ניידים), וקיום תקשורת בין הלקוח לשרת.

אפיון:

תיאור מפורט של המערכת:

המערכת מתחילה עם מסך כניסה למשתמש בממשק UI שנכתב בספריית tkinter כשברקע רץ השרת. המשתמש יכול להכנס לחשבון קיים או ליצור חשבון חדש ולפי אימות דו שלבי שיתבצע על ידי שליחת קוד למייל של המשתמש, המשתמש יועבר לעמוד הראשי בו הוא יוכל לגלול בעמוד הראשי בו יופיעו כל המוצרים הקיימים שלא נקנו, ויוכל גם לראות את המוצרים שהוא פרסם על ידי לחיצה על הפרופיל שלו ובנוסף הוא יכול גם לפרסם מוצר חדש. כמובן שכל המוצרים נשמרים ב-databases הממוקם בשרת. כאשר משתמש מחליט לקנות מוצר, נשלח מייל לקונה וגם למוכר בו מפורטים פרטי העסקה ודרכי תקשורת בין המשתמשים בכדי שיתאמו את העברת המוצר. המערכת תשתמש בהצפנה סימטרית (AES) במהלך כל שליחת ההודעות בין השרת ללקוח ולהיפך.

פירוט היכולות שהיא תעניק לכל סוג משתמש במערכת:

המערכת מאפשרת לכלל משתמשיה להכנס לחשבונם וממנו להעלות למכירה או לקנות מוצרים בכך שהיא שומרת את כל המידע לגבי המוצרים והמשתמשים (שניתן לה על ידיהם כמובן) ב-databases שממוקם בשרת.

פירוט הבדיקות:

תהיה בדיקה לסיסמא ושם משתמש כאשר משתמש יעשה log in, הבדיקה תתבצע בשרת תוך שליפת הסיסמא המתאימה לשם המשתמש שהוכנס והשוואתה לסיסמא שהוכנסה. בדיקה לכפילויות כאשר יש נסיון ליצור משתמש חדש, תוך הגדרת המייל ב-databases כייחודי לכל משתמש, בדיקה לקוד שנשלח, האם הוא תואם את הקוד שהוכנס על ידי המשתמש, בדיקה איזה מוצרים קיימים לא נקנו עדיין בכך שבכל מוצר ב-databases יהיה תכונה בה רשום אם הוא זמין או שהוא נמכר ולפי זה לשלוח את כל הזמינים בחזרה למשתמש שיוצגו במסך הראשי, בדיקת מוצרים לפי משתמש ספציפי



בכך שלכל מוצר תהיה תכונה שבה רשום המייל של המשתמש שפרסם את המוצר, בדיקת פרטי רכישה בכך שייבדקו כל המשתנים והתכונות הרלוונטיות של המוצר הנרכש בכדי שהוא יסומן כנרכש ושישלח מייל לקונה ולמוכר.

לוח זמנים:

תחילת פברואר 2025 - אישור רעיון הפרוייקט

עד סוף מרץ - כתיבה ועיצוב ממשק גרפי (בפועל עד אמצע-סוף אפריל)

עד סוף אפריל - כתיבת תקשורת בין לקוח לשרת (בפועל עד אמצע-סוף מאי)

עד סוף מאי - טיפול בהצפנה ובשאר הנושאים שנותרו (כך קרה גם בפועל)



תיאור תחום הידע

פירוט מעמיק של היכולות:

פירוט היכולות בצד הלקוח:

שם היכולת	מהות היכולת	אוסף היכולות הנדרשות למימוש היכולת	אובייקטים נחוצים
התחברות	התחברות לחשבון אשר קיים במערכת על ידי קליטת כתובת מייל וסיסמא מהלקוח	ממשק גרפי: מסך התחברות קליטת קלט מהמשתמש הצפנת הסיסמא מטעמי בטיחות תקשורת עם השרת: שליחת כתובת המייל והסיסמא וקבלת תשובה מהשרת אם הם נכונים או לא ניתוח המידע שהתקבל והצגתו במידת הצורך למשתמש	ממשק גרפי: custom tkinter הצפנת ההודעות באמצעות הצפנת AES הצפנת הסיסמא באמצעות hash תקשורת עם השרת: socket
הרשמה	יצירת חשבון חדש על ידי המשתמש על ידי קליטת כתובת מייל, מספר טלפון, שם משתמש וסיסמא	ממשק גרפי: מסך הרשמה קליטת קלט מהמשתמש הצפנת הסיסמא מטעמי בטיחות תקשורת עם השרת: שליחת שם המתמש והסיסמא וקבלת אישור שהחשבון נרשם במערכת (או אם קרתה תקלה) ניתוח המידע שהתקבל והצגתו במידת הצורך למשתמש	ממשק גרפי: custom tkinter הצפנת ההודעות באמצעות הצפנת AES הצפנת הסיסמא באמצעות hash תקשורת עם השרת: socket
"שכחתי סיסמא"	לאפשר ללקוח להכנס לחשבון אם הוא שכח את הסיסמא שלו על ידי שליחת קוד למייל של החשבון	ממשק גרפי: מסך בו נקלט המייל קליטת קלט מהמשתמש תקשורת עם השרת: שליחת כתובת המייל לשרת וקבלת אישור שהמייל נשלח (או אם קרתה תקלה) ניתוח המידע שהתקבל והצגתו במידת הצורך למשתמש	ממשק גרפי: custom tkinter הצפנת ההודעות באמצעות הצפנת AES תקשורת עם השרת: socket



אימות דו שלבי	לאחר שהלקוח התחבר או נרשם לחשבון נשלח לכתובת המייל של חשבון קוד בעל 5 ספרות שהלקוח יצטרך להזין בכדי שיוכל להכנס סופית לחשבון	ממשק גרפי: מסך בו נקלט הקוד קליטת קלט מהמשתמש תקשורת עם השרת: שליחת הקוד שנקלט לשרת וקבלת תשובה אם הקוד תואם לקוד שנשלח או לא ניתוח המידע שהתקבל והצגתו במידת הצורך למשתמש	ממשק גרפי: מסך בו נקלט הקוד תקשורת עם השרת: שליחת הקוד שנקלט לשרת וקבלת תשובה ניתוח המידע שהתקבל והצגתו למשתמש	ממשק גרפי: מסך בו נקלט הקוד תקשורת עם השרת: שליחת הקוד שנקלט לשרת וקבלת תשובה ניתוח המידע שהתקבל והצגתו למשתמש
בקשת כל הפריטים	הפעולה הזאת מתבצעת אוטומטית כאשר הלקוח עובר ממסך ההתחברות למסך הראשי, ובה הלקוח שולח בקשה לשרת שיתן לו את כל הפריטים הזמינים שלא פורסמו על ידו של הלקוח	ממשק גרפי: מסך בו נקלט הקוד תקשורת עם השרת: שליחת הקוד שנקלט לשרת וקבלת תשובה ניתוח המידע שהתקבל והצגתו למשתמש	ממשק גרפי: מסך בו נקלט הקוד תקשורת עם השרת: שליחת הקוד שנקלט לשרת וקבלת תשובה ניתוח המידע שהתקבל והצגתו למשתמש	ממשק גרפי: מסך בו נקלט הקוד תקשורת עם השרת: שליחת הקוד שנקלט לשרת וקבלת תשובה ניתוח המידע שהתקבל והצגתו למשתמש
קניית פריט	הלקוח יכול לקנות כל פריט אשר מוצג לו בעמוד הראשי	ממשק גרפי: מסך בו נקלט הקוד תקשורת עם השרת: שליחת הקוד שנקלט לשרת וקבלת תשובה ניתוח המידע שהתקבל והצגתו למשתמש	ממשק גרפי: מסך בו נקלט הקוד תקשורת עם השרת: שליחת הקוד שנקלט לשרת וקבלת תשובה ניתוח המידע שהתקבל והצגתו למשתמש	ממשק גרפי: מסך בו נקלט הקוד תקשורת עם השרת: שליחת הקוד שנקלט לשרת וקבלת תשובה ניתוח המידע שהתקבל והצגתו למשתמש
פרסום פריט	הלקוח יכול לפרסם פריט שברצונו למכור על ידי הזנת פרטי הפריט ותמונה של הפריט	ממשק גרפי: מסך בו נקלט הקוד תקשורת עם השרת: שליחת הקוד שנקלט לשרת וקבלת תשובה ניתוח המידע שהתקבל והצגתו למשתמש	ממשק גרפי: מסך בו נקלט הקוד תקשורת עם השרת: שליחת הקוד שנקלט לשרת וקבלת תשובה ניתוח המידע שהתקבל והצגתו למשתמש	ממשק גרפי: מסך בו נקלט הקוד תקשורת עם השרת: שליחת הקוד שנקלט לשרת וקבלת תשובה ניתוח המידע שהתקבל והצגתו למשתמש
הצגת פריטים אישיים	הלקוח יכול להציג בפניו בלחיצת כפתור את כל הפריטים אשר הוא פרסם ולראות מה	ממשק גרפי: מסך בו נקלט הקוד תקשורת עם השרת: שליחת הקוד שנקלט לשרת וקבלת תשובה ניתוח המידע שהתקבל והצגתו למשתמש	ממשק גרפי: מסך בו נקלט הקוד תקשורת עם השרת: שליחת הקוד שנקלט לשרת וקבלת תשובה ניתוח המידע שהתקבל והצגתו למשתמש	ממשק גרפי: מסך בו נקלט הקוד תקשורת עם השרת: שליחת הקוד שנקלט לשרת וקבלת תשובה ניתוח המידע שהתקבל והצגתו למשתמש



התמונה בעזרת לינק מAPI בו התמונה שמורה הצפנת ההודעות באמצעות הצפנת AES	מהשרת וקבלת תשובה עם כל הפריטים (או תשובה שאין פריטים על שמו של הלקוח) -ניתוח המידע שהתקבל והצגתו למשתמש	מצבם (האם נקנו או לא) ולמחוק אותם לפי רצונו	
---	--	---	--



פירוט היכולות בצד השרת:

שם היכולת	מהות היכולת	אוסף היכולות הנדרשות למימוש היכולת	אובייקטים נחוצים
השוואת פרטי התחברות	השרת משווה בין פרטי ההתחברות שהמשתמש שלח לו ולטבלת פרטי המשתמשים השמורים ב-database	תקשורת בין לקוח לשרת: קבלת המידע והחזרת תשובה גישה ל-database ושלילת נתונים ממנו לפי כתובת המייל והשוואת הסיסמאות המוצפנות.	תקשורת עם הלקוח: socket גישה לקובץ database מסוג sqlite3 הצפנת ההודעות באמצעות הצפנת AES
מניעת כפילויות בביצוע הרשמה	השרת מקבל את פרטי ההרשמה שהמשתמש שלח לו ובודק שאין חשבון קיים לכתובת המייל שניתנה	תקשורת בין לקוח לשרת: קבלת המידע והחזרת תשובה גישה ל-database ווידוא שאין חשבון קיים למייל על ידי מימוש primary key ושמירת הנתונים במקרה של אי כפילויות	תקשורת עם הלקוח: socket גישה לקובץ database מסוג sqlite3 הצפנת ההודעות באמצעות הצפנת AES
שליחת קוד (אימות דו שלבי #1)	השרת מקבל בקשה מהלקוח לשלוח קוד והשרת שולח למייל שניתן לשרת על ידי המתמש קוד שאותו הלקוח צריך להזין בצד שלו	תקשורת בין לקוח לשרת: קבלת המידע והחזרת תשובה שהקוד נשלח -יצירת קוד רנדומלי ושמירתו לצורך השוואה -שליחת מייל מכתובת מייל שנוצרה במיוחד לאפליקציה, אל המייל שניתן על ידי המשתמש	תקשורת עם הלקוח: socket שימוש בספריית random ליצירת קוד רנדומלי בעל 5 ספרות -שמירת הקוד מוצמד לכתובת המייל של הלקוח ברשימת קודים -מימוש פרוטוקול smtp לצורך שליחת מייל מהשרת -הצפנת ההודעות באמצעות הצפנת AES
אימות הקוד (אימות דו שלבי #2)	השרת מקבל מהלקוח את הקוד שהזין והוא בודק האם זה באמת הקוד שנשלח אל הלקוח או לא. במידה וכן השרת שולח למשתמש קוד סודי הנוצר בעת	תקשורת בין לקוח לשרת: קבלת המידע והחזרת תשובה האם הקוד תואם או לא -בדיקה האם הקוד תואם את הקוד שנוצר -יצירת קוד רנדומלי (לצורך הקוד הסודי)	תקשורת עם הלקוח: socket -שלילת הקוד שמוצמד לכתובת המייל שהתקבלה מרשימת הקודים והשוואתו לקוד שהתקבל



ההתחברות שאיתו המשתמש יבצע את כל הפעולות מכן ואילך	ושמירתו בהצמדה לכתובת המייל של הלקוח	הצפנת ההודעות באמצעות הצפנת AES	
שליחת הפריטים למסך בית	השרת מקבל מהלקוח את הקוד הסודי של המשתמש ומחזיר למשתמש את כל הפריטים שעדיין זמינים ושלא רשומים על שמו של הלקוח שאליו הפריטים נשלחים	תקשורת בין לקוח לשרת: קבלת המידע והחזרת הנתונים -שליפת כתובת המייל מהרשימה לפי הקוד הסודי שהתקבל מהלקוח ושימוש בו -גישה לקובץ database מסוג sqlite3 -הצפנת ההודעות באמצעות הצפנת AES	
פרסום פריט	השרת מקבל מהלקוח את פרטי הפריט אותו הוא רוצה לפרסם ואת הקוד הסודי שלו. בפרטים של הפריט, השרת מקבל תמונה של הפריט אותו הוא מעלה ל-API חיצוני ברשת ומקבל חזרה לינק לצורך הצגת התמונה והוא מוסיף את כל הפרטים (כאשר הdata של התמונה מוחלפת בלינק שנוצר) לטבלה databse של המוצרים כשבטבלה יש את נתוני הפריט, מייל המשתמש שפרסם את הפריט ומספר זהות ייחודי של הפריט שנוצר אוטומטית על ידי databsen	תקשורת עם הלקוח: -שליפת המייל שמוצמד לקוד הסודי שהתקבל מהלקוח ושימוש בו -גישה לקובץ database מסוג sqlite3 -הצפנת ההודעות באמצעות הצפנת AES	
בקשה לקניית פריט	השרת מקבל מהלקוח בקשה לקניית פריט מסוים על ידי קבלת מספר הזהות של הפריט ואת הקוד הסודי של הלקוח ולפי שני הפרמטרים השרת מעדכן	תקשורת עם הלקוח: -שליפת המייל שמוצמד לקוד הסודי שהתקבל מהלקוח ושימוש בו -גישה לקובץ database מסוג sqlite3	



sqlite3 -מימוש פרוטוקול smtp לצורך שליחת מיילים מהשרת -הצפנת ההודעות באמצעות הצפנת AES	-שינוי פרטי הפריט וגישה לנתונים, והוספת פרטי העסקה לטבלת הסטורית העסקאות database -שליחת 2 מיילים מכתובת המייל שנוצרה במיוחד לאפליקציה, אל המיילים של הקונה והמוכר	בdatabase את הפריט לקנוי ומוסיף את פרטי העסקה לטבלה שונה של כל הסטוריית העסקאות, ולפי הפרטים שהתקבלו ושקיימים בdatabase, הוא שולח ללקוח ולמוכר מייל בו פרטי המוכר/הקונה (הקונה מקבל את פרטי המוכר ולהיפך) ואת פרטי העסקה	
--	--	---	--



מבנה הפרויקט

תיאור הארכיטקטורה של המערכת:

הלקוח(ות):

המערכת מסוגלת להכיל מספר של לקוחות שהם מחשבים המחוברים לשרת דרך כרטיס רשת המחובר לרשת אלחוטית או דרך כבל אינטרנט המחובר למחשב פיזית. המחשבים מריצים את התוכנה של הclient שמריצה את הממשק הגרפי שבאמצעותו הלקוחות יכולים לבצע כל מיני פעולות אשר עוברות בין מרכיבי המערכת דרך תקשורת רשתית.

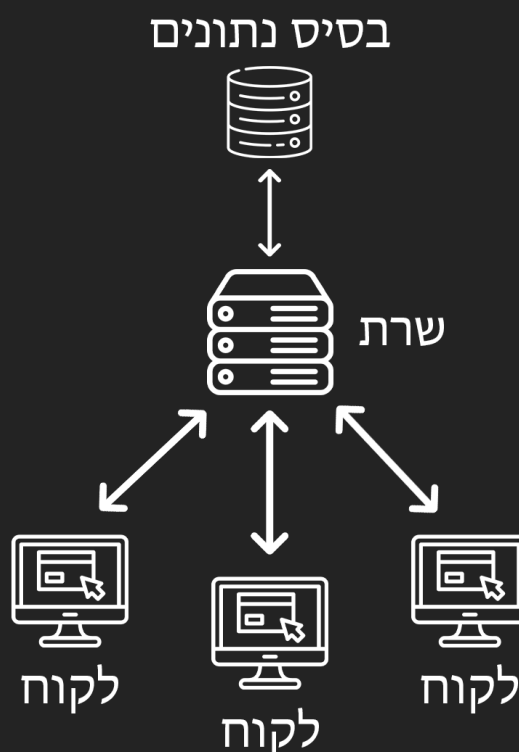
השרת:

השרת הוא מחשב עליו רץ תוכנת server ומשתמש בתקשורת TCP ובאמצעותה הוא מקשיב דרך port מסוים לכל הבקשות שהלקוחות שולחים את השרת, ולפיהם מבצע פעולות באמצעות בפעולות הכתובות לו בתוכנה או מרכיב בסיס הנתונים בכך ששומר בו ושולף ממנו מידע שנשלח על ידי הלקוחות או מידע אודות הלקוחות ומחזיר בתקשורת תשובה או נתונים אל הלקוחות.

בסיס הנתונים:

בסיס הנתונים או database הוא מרכיב אשר יושב על אותו מחשב עליו יושב השרת ובו מאוכסן כל המידע אודות הלקוחות, המוצרים שפורסמו והיסטורית העסקאות שבוצעו. באמצעות בסיס הנתונים השרת יכול להוסיף, לשלוף, לעדכן ולמחוק מידע בצורה בטוחה ומקומית (אין צורך בתקשורת רשתית מכל סוג שהיא בין השרת לבסיס הנתונים)

סרטוט מבנה רכיבים אלו:





טכנולוגיה רלוונטית:

שפת תכנות:

שפת התכנות היחידה שהשתמשתי בה במהלך בניית הפרוייקט הייתה Python, אך במהלך הפרוייקט השתמשתי בנוסף בHtml באלגוריתמיקה של שליחת המייל.

מערכת ההפעלה:

הקוד נכתב במחשב שעליו מערכת ההפעלה windows אך התוכנה עצמה יכולה לרוץ על כל סוג מערכת הפעלה.

תקשורת:

התקשורת מתבצעת באמצעות פרוטוקול TCP (Transmission Control Protocol), פרוטוקול תקשורת בטוח ואמין שמוודא שכל המידע מועבר אל המקום הנכון בצורה נכונה ומסודרת מבלי לפגוע בשלמות וסידור המידע שנשלח. כאשר משתמש חדש מפעיל את התוכנה הוא מבצע עם השרת לחיצת יד משולשת (פרוטוקול בסיסי ב-TCP) ובכך מבססים חיבור מוצלח וחזק שיאפשר לשלוח הודעות בין השניים.

תחומי עניין:

- **פיתוח התוכנה:** פיתוח התוכנה הוא התחום עיקרי בפרוייקט ובאמצעותו מתממשים תחומי העניין האחרים. פיתוח התוכנה כלל פיתוח פונקציות ומחלקות וניהולן, פיתוח שרת אשר מנהל משתמשים ומקבל מידע מהם ופיתוח הממשק הגרפי שאיתו המשתמש יכול לבצע פעולות.
- **רשתות:** הרשתות גם הוא תחום עיקרי וחשוב בפרוייקט ובאמצעותו מתממשת התקשורת בין הלקוח לשרת.
- **שימוש ב-API:** העלאת תמונות של הפריטים ל-API ציבורי של אתר imgur המאפשר להעלות תמונה ולקבל לינק לתמונה בחזרה בכדי להציג את התמונה בקוד.
- **הגנת סייבר:** הצפנה מסוג AES וhash והגנה מפתני תקיפות סייבר כמו sql injection ועוד.



זרימת המידע במערכת:

הרשמה והתחברות:



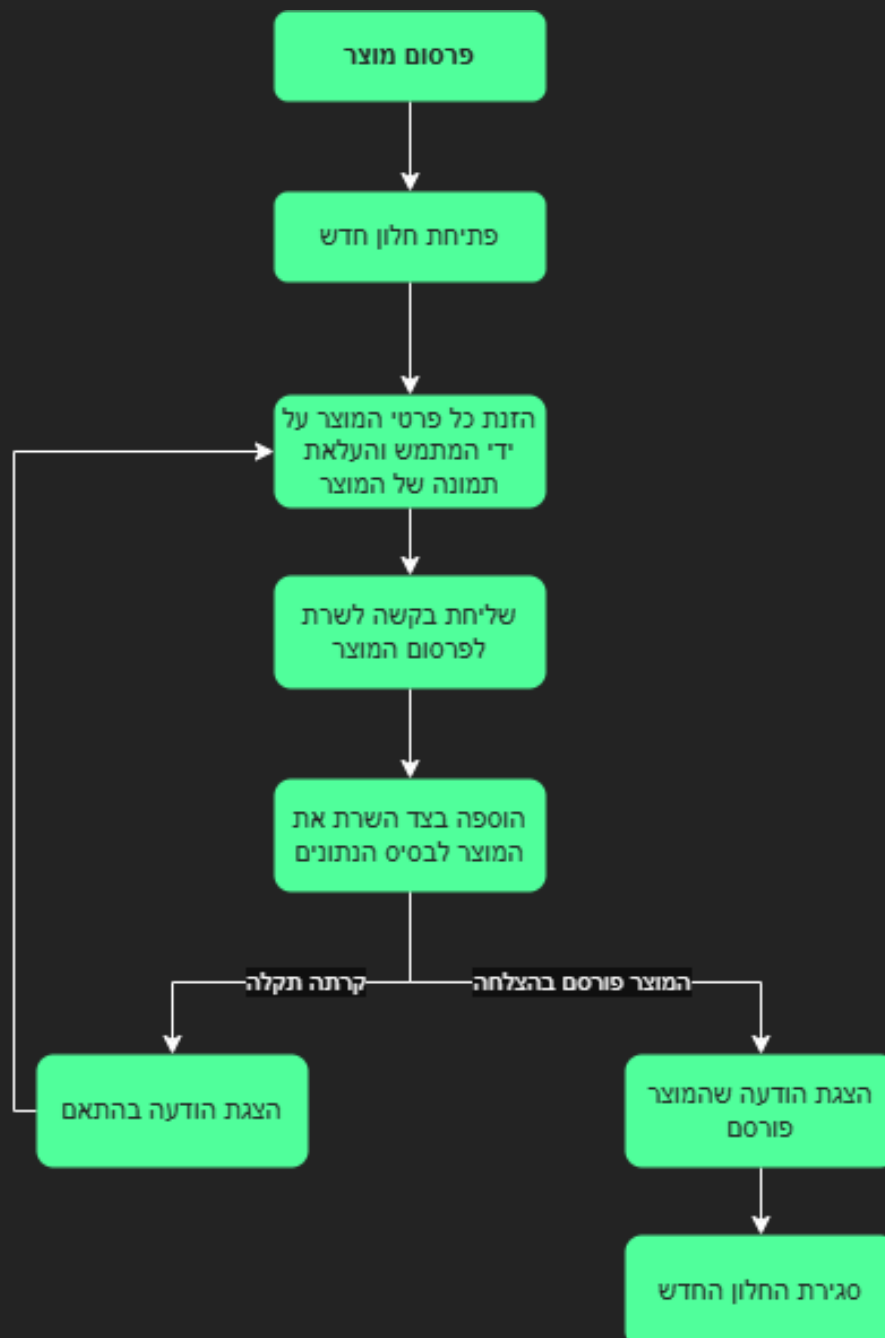


מסך הבית:



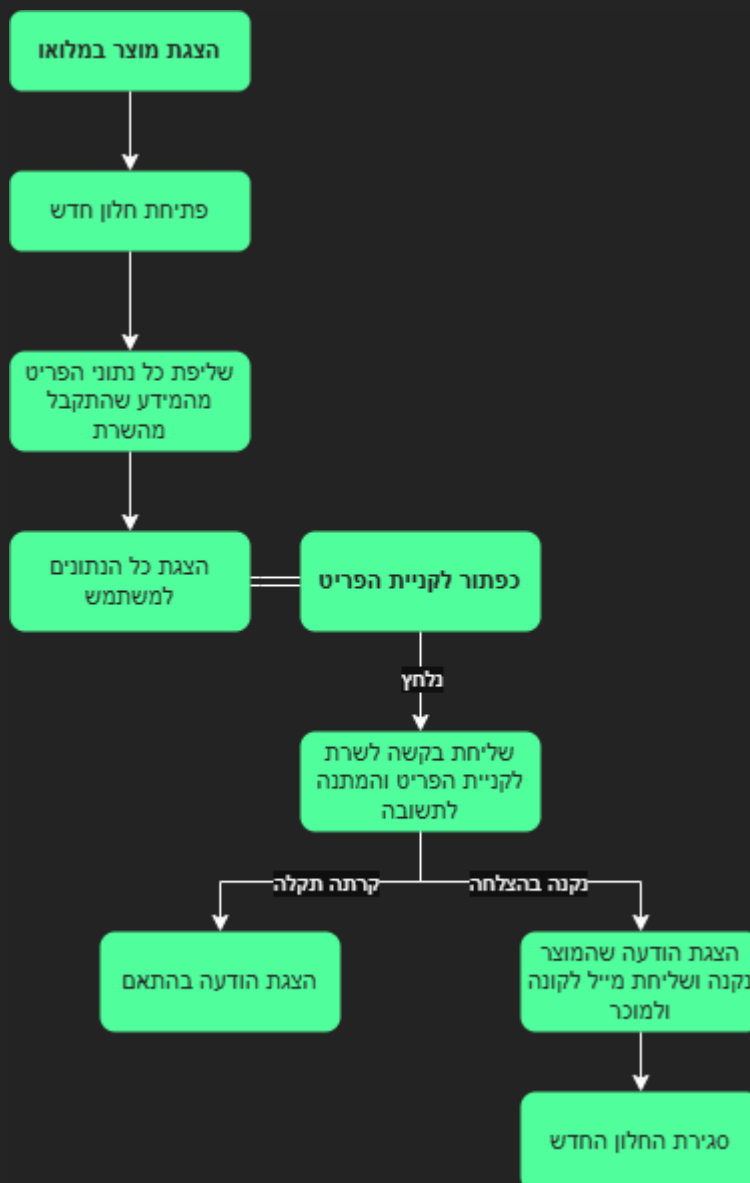


פרסום מוצר:



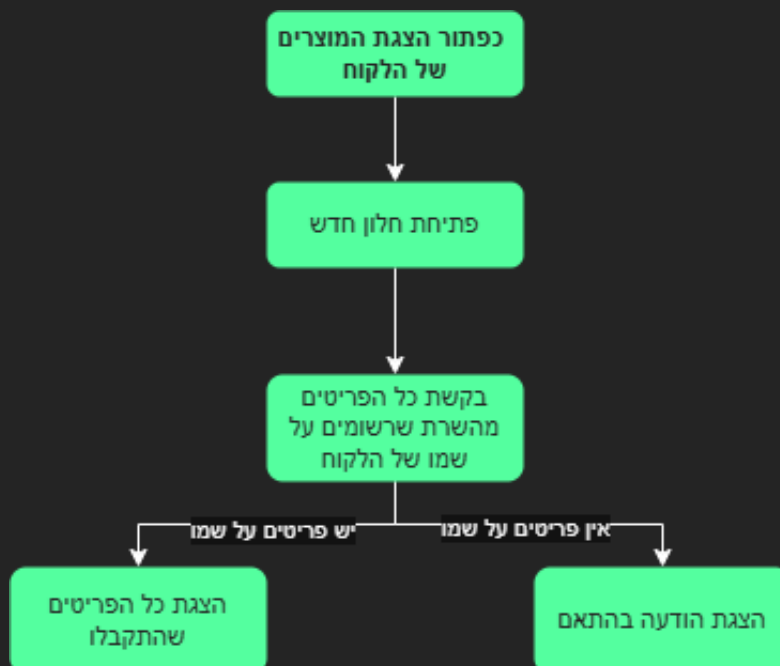


הצגת מוצר במלואו:





הצגת המוצרים של הלקוח:





תיאור האלגוריתמים המרכזיים בפרויקט:

הצפנת תקשורת סימטרית באמצעות AES ו-Diffie-Hellman:

ניסוח וניתוח הבעיה האלגוריתמית:

ביצירת הפרוייקט, נדרשתי להתמודד עם מספר אתגרים מרכזיים בתחום אבטחת המידע:

1. הצפנת תקשורת בטוחה: כל המידע המועבר בין הלקוח לשרת חייב להיות מוצפן
2. חלוקת מפתחות בטוחה: יצירת מפתח הצפנה משותף מבלי לחשוף אותו ברשת
3. יעילות בביצועים: טיפול בכמויות גדולות של נתונים כמו תמונות
4. פשטות ביישום: אמינות ללא טעויות קריטיות

אלגוריתמים קיימים לפתרון הבעיה:

- AES/DES/ChaCha20 להצפנה סימטרית
- Diffie-Hellman/RSA/ECDH לחלוקת מפתחות

הפנייה למקורות רלוונטיים:

- <https://cryptography.io/en/latest/hazmat/primitives/ciphers/>
- https://en.wikipedia.org/wiki/Diffie%E2%80%93Hellman_key_exchange
- <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>

סקירת הפתרון הנבחר:

בחרתי בשילוב Diffie-Hellman ו-AES מהסיבות הבאות: אבטחה גבוהה, יעילות מעולה לכמויות גדולות של נתונים, ופשטות יחסית ביישום.

תהליך חלוקת המפתחות:

• יצירת פרמטרים ציבוריים ($p=23$, $g=5$) המשותפים לכל הלקוחות • יצירת מפתח פרטי אקראי לכל צד (לקוח ושרת)

• חישוב מפתח ציבורי באמצעות החזקה מודולרית

• החלפת מפתחות ציבוריים בין הלקוח לשרת

• יצירת מפתח סודי משותף באמצעות המפתח הפרטי והמפתח הציבורי של הצד השני

• המרת המפתח הסודי למפתח AES באמצעות SHA-256

תהליך ההצפנה והפענוח:

• יצירת IV (Initialization Vector) אקראי בגודל 16 בתים לכל הודעה

• הוספת padding להודעה באמצעות PKCS7 להתאמה לבלוקים של 16 בתים



- הצפנת ההודעה באמצעות AES במצב CBC
- שליחת ה-IV ולאחר מכן הנתונים המוצפנים
- פענוח באמצעות אותו IV והסרת ה-padding

אלגוריתם ניהול התקשורת הרב-לקוחי (Multi-Client Communication):

ניסוח וניתוח הבעיה האלגוריתמית:

השרת צריך לטפל במספר לקוחות בו-זמנית ולנהל את המידע והחיבורים שלהם:

1. ניהול חיבורים מרובים: השרת צריך לקלוט חיבורים חדשים תוך המשך טיפול בקיימים
2. זיהוי לקוחות: כל לקוח צריך קוד סודי ייחודי לביצוע פעולות
3. ניהול הודעות: קבלה ושליחה של הודעות למספר לקוחות במקביל
4. טיפול בניתוקים: ניקוי נתונים כשלקוח מתנתק

אלגוריתמים קיימים לפתרון הבעיה:

- Threading: יצירת thread נפרד לכל לקוח
- Select/Poll: מנגנון I/O multiplexing
- Asyncio: תכנות אסינכרוני
- Fork: יצירת תהליך נפרד לכל לקוח

הפנייה למקורות רלוונטיים:

- <https://docs.python.org/3/library/select.html>
- <https://docs.python.org/3/library/threading.html>
- <https://realpython.com/python-sockets/>

סקירת הפתרון הנבחר:

בחרתי ב-Select-based I/O Multiplexing עם threading לסיבות של יעילות זיכרון, מנגנון אחיד לכל הלקוחות, וטיפול נוח בשגיאות.

מנגנון ניהול החיבורים:

- שימוש בפונקציית select עם timeout של 0.1 שניות למניעת blocking
- בדיקת socket השרת לחיבורים חדשים ובדיקת כל socket של לקוח להודעות נכנסות
- קבלה של חיבור חדש ויצירת מפתח הצפנה משותף
- הוספת הלקוח לרשימת הלקוחות המחוברים



מנגנון ניהול ההודעות:

- קבלת IV בגודל 16 בתים ולאחר מכן קבלת ההודעה המוצפנת
- פענוח ההודעה והוספתה לתור ההודעות הנכנסות
- עיבוד הודעות יוצאות מהתור ושליחתן ללקוחות המתאימים
- שליחת IV חדש ולאחר מכן ההודעה המוצפנת

מנגנון טיפול בניתוקים:

- זיהוי ניתוק לקוח על ידי קבלת נתונים ריקים או ConnectionResetError
- הסרת הלקוח מרשימת הלקוחות המחוברים
- ניקוי הקוד הסודי של הלקוח מהמערכת • סגירת ה-socket של הלקוח

אלגוריתם אימות דו-שלבי (Two-Factor Authentication)

ניסוח וניתוח הבעיה האלגוריתמית:

צורך ליצור מערכת אימות בטוחה שלא מסתמכת רק על סיסמה:

1. יצירת קוד אקראי: קוד שלא ניתן לנחש
2. שליחת מייל אמין: הקוד חייב להגיע ליעד
3. ניהול תוקף זמני: קודים לא יכולים להישאר תקפים לנצח
4. מניעת שימוש חוזר: קוד ששומש לא יכול לשמש שוב

אלגוריתמים קיימים לפתרון הבעיה:

- TOTP (Time-based OTP): קודים מבוססי זמן
- SMS-based OTP: שליחה בהודעת טקסט
- Email-based OTP: שליחה במייל
- Hardware tokens: מכשירים פיזיים

הפנייה למקורות רלוונטיים:

- <https://tools.ietf.org/html/rfc6238> (TOTP)
- <https://docs.python.org/3/library/smtplib.html>
- <https://docs.python.org/3/library/random.html>

סקירת הפתרון הנבחר:



בחרתי ב-Email-based OTP עם קוד אקראי פשוט מהסיבות הבאות: נגישות (כל משתמש יש לו מייל), פשטות (לא דורש אפליקציות נוספות), אמינות ועלות (חינמי לשימוש). לשליחת המיילים עצמם התשמתי בפרוטוקול smtp או בשמו המלא Simple Mail Transfer Protocol, פרוטוקול רשתי המיועד לשליחה ולקבלת מיילים.

כל מסר SMTP מורכב משני חלקים עיקריים: **Headers** - כותרות המסר ו**Body** - תוכן המסר. הכותרות מכילות מידע על המסר ומופיעות בתחילת המסר:

From: sender@example.com

To: recipient@example.com

Subject: נושא המסר

Date: Su, 01 Jun 2025 12:00:00 +0200

Message-ID: <12345@gmail.com>

MIME-Version: 1.0

Content-Type: text/plain; charset=UTF-8

תהליך יצירת וניהול הקוד:

- יצירת קוד אקראי בן 5 ספרות באמצעות random.randint
- שמירת הקוד ברשימת קודים פעילים עם הצמדה לכתובת המייל
- הגדרת תוקף זמני של הקוד (מחיקה אוטומטית לאחר זמן מסוים)

תהליך שליחת המייל:

- התחברות לשרת Gmail באמצעות SMTP SSL על פורט 465
- יצירת הודעת HTML מעוצבת עם הקוד
- שליחת המייל ושמירת סטטוס השליחה
- החזרת תשובה ללקוח על הצלחה או כישלון בשליחה

תהליך אימות הקוד:

- קבלת הקוד מהלקוח יחד עם כתובת המייל
- חיפוש הקוד ברשימת הקודים הפעילים
- השוואת הקוד שהתקבל לקוד השמור
- במקרה של התאמה - יצירת קוד סודי ייחודי למשתמש ומחיקת קוד האימות



אלגוריתם ניהול בסיס נתונים ומניעת SQL Injection

ניסוח וניתוח הבעיה האלגוריתמית:

ניהול מידע רגיש בבסיס נתונים תוך מניעת פגיעות אבטחה:

1. מניעת SQL Injection: קלט משתמש לא יכול לשנות שאילתות
2. ניהול חיבורים: טיפול ב-database locks ובעיות חיבור
3. שמירת שלמות נתונים: מניעת נתונים פגומים או חסרים
4. ביצועים: שאילתות יעילות למספר רב של פריטים

אלגוריתמים קיימים לפתרון הבעיה:

- Prepared Statements: שאילתות מוכנות מראש
- ORM (Object-Relational Mapping): SQLAlchemy, Django ORM
- Input Validation: בדיקת קלט לפני השאילתה
- Connection Pooling: מאגר חיבורים לבסיס נתונים

הפנייה למקורות רלוונטיים:

- <https://docs.python.org/3/library/sqlite3.html>
- https://owasp.org/www-community/attacks/SQL_Injection
- <https://bobby-tables.com/>

סקירת הפתרון הנבחר:

בחרתי ב-Parameterized Queries עם SQLite3 מהסיבות הבאות: מניעת SQL Injection, קריאות קוד, ביצועים טובים ופשטות ביישום.

מנגנון חיבור בטוח לבסיס הנתונים:

- ניסיון התחברות במספר נסיונות (עד 5) עם timeout של 30 שניות
- טיפול בשגיאות חיבור והמתנה בין נסיונות
- יצירת טבלאות בסיסיות אם הן לא קיימות
- שימוש ב-Primary Key למניעת כפילויות

מנגנון שאילתות בטוחות:

- שימוש בפרמטרים מופרדים בכל שאילתת (? placeholders) של SQL
- אימות קלט משתמש לפני הכנסה לבסיס הנתונים



• שימוש ב-try-catch לטיפול בשגיאות SQL

• ביצוע commit אוטומטי לאחר כל שינוי

מנגנון ניהול נתונים:

• הוספת משתמשים חדשים עם בדיקת כפילויות

• עדכון סטטוס מוצרים (זמין/נמכר)

• שליפת נתונים לפי משתמש ולפי סטטוס

• שמירת היסטוריית עסקאות עם פרטים מלאים

אלגוריתם עיבוד ועירור תמונות לאחסון ולהצגה

ניסוח וניתוח הבעיה האלגוריתמית:

טיפול בתמונות מהמשתמש לצורך הצגה במערכת:

1. גדלי קבצים: תמונות יכולות להיות גדולות מדי
2. פורמטים שונים: תמיכה במגוון פורמטי תמונה
3. אחסון יעיל: לא להעמיס על השרת או על הרשת
4. הצגה אחידה: כל התמונות צריכות להיראות טוב בממשק הגרפי

אלגוריתמים קיימים לפתרון הבעיה:

- Local Storage: שמירה בשרת המקומי
- Cloud Storage: Amazon S3, Google Cloud Storage
- Image CDN: שירותי הפצת תמונות
- Base64 Encoding: קידוד התמונה כטקסט

הפנייה למקורות רלוונטיים:

- <https://pillow.readthedocs.io/en/stable/>
- <https://apidocs.imgur.com/>
- <https://docs.python.org/3/library/base64.html>



סקירת הפתרון הנבחר:

בחרתי בשילוב של PIL לעיבוד + Imgur API לאחסון מהסיבות הבאות: חסכון במשאבי השרת, מהירות גבוהה, תמיכה בכל הפורמטים, אפשרות של הצגת התמונה באמצעות לינק מה-API והשירות עצמו חינמי.

תהליך עיבוד התמונה בצד הלקוח:

- פתיחת התמונה באמצעות PIL
- בדיקת גודל התמונה והקטנתה במידת הצורך (מקסימום 200x200 פיקסלים)
- שמירת יחס הגובה-רוחב המקורי בעת שינוי הגודל
- שימוש באלגוריתם LANCZOS לאיכות גבוהה בשינוי גודל
- המרת התמונה לפורמט בינארי והצגתה בממשק הגרפי

תהליך העלאה ל-API חיצוני:

- קידוד התמונה ל-Base64 והעברתה לשרת
- שליחת בקשת POST ל-Imgur API עם האימות הנדרש
- קבלת לינק ישיר לתמונה מה-API
- שמירת הלינק במקום התמונה עצמה בבסיס הנתונים
- טיפול בשגיאות העלאה ומנגנון retry במקרה של כישלון

תהליך הצגת התמונה בממשק הגרפי:

- שליפת לינק התמונה מבסיס הנתונים
- שימוש ב-User-Agent לחיקוי דפדפן ומניעת חסימות כאשר הקוד מנסה לגשת ללינק
- הורדת התמונה מה-URL באמצעות requests
- המרת התמונה ל-PIL Image ולאחר מכן ל-CTkImage לצורך הצגתה
- מנגנון fallback להצגת תמונת placeholder במקרה של שגיאה

אלגוריתם ניהול מצב לקוחות ואימות פעולות

ניסוח וניתוח הבעיה האלגוריתמית:



מעקב אחר מצב התחברות של לקוחות ואימות זכויות לפעולות:

1. ניהול מצב: זכירה איזה לקוח מחובר ועם איזה קוד סודי
2. אימות פעולות: וידוא שרק משתמשים מחוברים יכולים לבצע פעולות
3. ניקוי זיכרון: הסרת נתונים של לקוחות שהתנתקו
4. מניעת זיוף: לקוח לא יכול להתחזות לאחר

אלגוריתמים קיימים לפתרון הבעיה:

- Session Tokens: טוקנים זמניים
- JWT (JSON Web Tokens): טוקנים מוצפנים
- Cookie-based Sessions: מבוסס על עוגיות דפדפן
- Server-side Sessions: מידע נשמר בשרת

הפנייה למקורות רלוונטיים:

- <https://jwt.io/introduction/>
- https://owasp.org/www-community/Session_Management

סקירת הפתרון הנבחר:

בחרתי ב-Server-side Session Management עם קודים סודיים מהסיבות הבאות: אבטחה גבוהה, שליטה מלאה בהרשאות, פשטות ביישום וגמישות.

מנגנון יצירת ואימות זהות:

- יצירת קוד סודי ייחודי לכל לקוח לאחר אימות מוצלח (מכפלת שני מספרים אקראיים)
- שמירת הקוד עם פרטי ה-socket וכתובת המייל של המשתמש
- שליחת הקוד הסודי ללקוח לשימוש בפעולות עתידיות
- אימות הקוד בכל פעולה על ידי חיפוש ברשימת הקודים הפעילים

מנגנון בקרת גישה לפעולות:

- בדיקה שהקוד הסודי קיים ברשימה לפני ביצוע כל פעולה
- שליפת כתובת המייל המשויכת לקוד לצורך ביצוע פעולות
- מניעת גישה למידע של משתמשים אחרים

מנגנון ניקוי וניהול זיכרון:

- הסרת קוד סודי כאשר לקוח מתנתק מהשרת
- ניקוי אוטומטי של קודים שאינם בשימוש



- עדכון רשימת הלקוחות הפעילים בזמן אמת
- מניעת דליפת זיכרון עקב הצטברות נתונים ישנים

אלגוריתם עיבוד JSON ונתונים מורכבים

ניסוח וניתוח הבעיה האלגוריתמית:

טיפול בנתונים מורכבים המועברים בין לקוח לשרת:

1. נתונים מעורבים: מוצרים מכילים טקסט, מספרים, תמונות
2. גדלים משתנים: כמות הפריטים יכולה להשתנות
3. עמידות בשגיאות: טיפול בנתונים פגומים או חסרים
4. תאימות: פורמט שמובן גם ללקוח וגם לשרת

אלגוריתמים קיימים לפתרון הבעיה:

- JSON: פורמט קל וקריא
- XML: פורמט מובנה אך כבד
- Pickle: פורמט Python ספציפי
- Protocol Buffers: פורמט בינארי יעיל

הפנייה למקורות רלוונטיים:

- <https://docs.python.org/3/library/json.html>
- <https://www.json.org/json-en.html>

סקירת הפתרון הנבחר:

בחרתי בJSON עם מנגנון fallback לטיפול בשגיאות מהסיבות הבאות: תקן תעשייתי, קריאות גבוהה, גמישות ועמידות בפני שגיאות.

מנגנון עיבוד נתוני JSON:

- ניסיון פירוש JSON רגיל באמצעות `json.loads`
- הסרת `escape characters` ותווים מיותרים
- תיקון פורמט JSON פגום על ידי הוספת סוגריים חסרים
- חלוקת נתונים מורכבים לפי `separator` מיוחד (©)



מנגנון Fallback לנתונים פגומים:

- שימוש ב-Regular Expressions לחילוף שדות חשובים
- יצירת אובייקט JSON חלקי עם הנתונים שנמצאו
- הגדרת ערכי default לשדות חסרים
- המשך עיבוד גם עם נתונים לא שלמים

מנגנון הצגה והתצוגה:

- המרת נתוני JSON לאובייקטים גרפיים בממשק
- טיפול בתמונות על ידי הורדה מ-URL וקידוד מחדש
- יצירת רשימות מוצרים עם pagination לביצועים טובים
- הצגת הודעות שגיאה ידידותיות למשתמש במקרה של כישלון בעיבוד

תיאור סביבת הפיתוח

כלי הפיתוח הדרושים לפיתוח:

- PyCharm Community Edition 2023 = סביבת העבודה בה כתבתי את הקוד
- Python 3.13 = שפת התכנות וגרסתה איתה כתבתי את הקוד
- DB Browser (SQLite) = הממשק בו יצרתי ויכולתי לצפות בבסיס הנתונים
- <https://apidocs.imgur.com/> = ממשק תכנות היישומים אליו בעלתי את כל התמונות של המוצרים

כלים הנדרשים לבדיקות:

- Consolens ב-PyCharm שבו הוספתי logger אשר מתעד כל פעולה המתבצעת אצל השרת ואצל הלקוח.
- CMD - כדי לבדוק את כתובת ה-IP של המחשב בו השרת מורץ לפני הרצת צד הלקוח.
- DB Browser - כדי שאוכל לבדוק את מצב הבסיס נתונים בכל עת ולראות אם הנתונים נשמרים כמו שצריך.
- WireShark - כדי לבדוק כיצד עוברות הפקטות מידע בין השרת ללקוח ולבדוק האם הן עוברות בצורה נכונה ומוצפנת כמו שצריך.

תיאור פרוטוקול התקשורת

תיאור מילולי של פרוטוקול התקשורת:



פרוטוקול התקשורת במערכת FanZone Marketplace הינו פרוטוקול TCP. פרוטוקול זה נמצא בשכבת התעבורה (השכבה הרביעית), והוא מבטיח העברה אמינה ובטוחה של מידע ו/או נתונים בין שתי תחנות ברשת מחשבים באמצעות יצירת חיבור מקושר (Connection Oriented).

פרוטוקול TCP הוא למעשה צורת התקשורת שכאשר היא מופעלת על ידי שתי תחנות היא מאפשרת, באמצעות הוספת כמות קטנה של מידע (header בגודל של 20 עד 60 בתים) לכל חבילה, העברת מידע אמינה וללא שגיאות בין שני הצדדים, גם כאשר התווך ביניהם רעוע.

פרוטוקול TCP מאפשר מעבר נתונים דרך כתובות IP ושיון לאפליקציות דרך פורט 5678 שמשויך לאפליקציה (זאת במסגרת חבילת הפרוטוקולים TCP/IP).

מבנה התקשורת במערכת:

1. יצירת חיבור TCP בין הלקוח לשרת על פורט 5678
2. חלוקת מפתחות הצפנה באמצעות Diffie-Hellman Key Exchange
3. שליחת נתונים מוצפנים באמצעות AES עם CBC mode
4. כל הודעה מורכבת מ:
 - בגודל 16 בתים (Initialization Vector) IV
 - נתונים מוצפנים בגודל משתנה (עד 10,485,760 בתים {הגודל הגדול הוא בכדי להעביר את התמונות בצורה תקינה וללא בעיות})

פירוט כלל ההודעות הזורמות במערכת:

הודעות שנשלחות מהלקוח לשרת:

1. בקשת התחברות למערכת

- שם ההודעה: LOGIN_REQUEST
- נשלחת מ/אל: לקוח ← שרת
- מבנה השדות:
`USER_INTRO#LOGIN_REQUEST#{email}{password}`
- דוגמה:
`USER_INTRO#LOGIN_REQUEST#yoav@gmail.com 12345`

2. בקשת הרשמה למערכת

- שם ההודעה: SIGNUP_REQUEST
- נשלחת מ/אל: לקוח ← שרת
- מבנה השדות:
`USER_INTRO#SIGNUP_REQUEST#{email}@#{hashed_password}@#{username}@#{phone}`
- דוגמה:
`USER_INTRO#SIGNUP_REQUEST#user@gmail.com@$2b$12$...@0501234567@יואב`

3. בקשת קוד אימות

- שם ההודעה: CODE_REQUEST
- נשלחת מ/אל: לקוח ← שרת



- מבנה השדות:
USER_INTRO#CODE_REQUEST#{email}
- דוגמה:
USER_INTRO#CODE_REQUEST#user@gmail.com

4. אימות קוד דו-שלבי

- שם ההודעה: CODE_VALIDATION
- נשלחת מ/אל: לקוח ← שרת

- מבנה השדות:
USER_INTRO#CODE_VALIDATION#{email}:{code}
- דוגמה:
USER_INTRO#CODE_VALIDATION#user@gmail.com:12345

5. בקשת שם משתמש

- שם ההודעה: USERNAME_REQUEST
- נשלחת מ/אל: לקוח ← שרת

- מבנה השדות:
MAIN#USERNAME_REQUEST#{secret_code}
- דוגמה:
MAIN#USERNAME_REQUEST#123456

6. בקשת כל הפריטים הזמינים

- שם ההודעה: DATA_REQUEST
- נשלחת מ/אל: לקוח ← שרת

- מבנה השדות:
MAIN#DATA_REQUEST#{secret_code}#1
- דוגמה:
MAIN#DATA_REQUEST#123456#1

7. בקשת פריטים של משתמש

- שם ההודעה: DATA_FOR_USER_REQUEST
- נשלחת מ/אל: לקוח ← שרת

- מבנה השדות:
MAIN#DATA_FOR_USER_REQUEST#{secret_code}
- דוגמה:
MAIN#DATA_FOR_USER_REQUEST#123456

8. פרסום פריט חדש

- שם ההודעה: PUBLISH_ITEM
- נשלחת מ/אל: לקוח ← שרת

- מבנה השדות:
MAIN#PUBLISH_ITEM#{json_data}#{secret_code}



- דוגמה (אלו לא כל המשתנים שיש בעצם של הפריט):
MAIN#PUBLISH_ITEM#{team:"Hapoel Tel Aviv","size":"M","price":"100"}#123456

9. קניית פריט

- שם ההודעה: BUY_ITEM
- נשלחת מ/אל: לקוח ← שרת
- מבנה השדות:
MAIN#BUY_ITEM#{secret_code}#{item_id}©{price}©{item_title}
- דוגמה:
MAIN#BUY_ITEM#123456#42©100©Hapoel Tel Aviv 2025 Shirt



הודעות שנשלחות מהשרת ללקוח:

1. תשובת התחברות מוצלחת

- שם ההודעה: LOGIN_SUCCESS
- נשלחת מ/אל: שרת → לקוח

- מבנה השדות:

USER_INTRO#LOGIN_RESPONSE#LOGIN_SUCCESS

2. תשובת התחברות כושלת

- שם ההודעה: LOGIN_FAILED
- נשלחת מ/אל: שרת → לקוח

- מבנה השדות:

USER_INTRO#LOGIN_RESPONSE#USER_OR_PASS_INVALID

3. תשובת הרשמה מוצלחת

- שם ההודעה: SIGNUP_SUCCESS
- נשלחת מ/אל: שרת → לקוח

- מבנה השדות:

USER_INTRO#SIGNUP_RESPONSE#OK

4. תשובת הרשמה כושלת (משתמש קיים)

- שם ההודעה: USER_EXISTS
- נשלחת מ/אל: שרת → לקוח

- מבנה השדות:

USER_INTRO#SIGNUP_RESPONSE#USER_EXISTS

5. אישור שליחת קוד

- שם ההודעה: CODE_SENT
- נשלחת מ/אל: שרת → לקוח

- מבנה השדות:

USER_INTRO#CODE_RESPONSE#OK

6. כישלון שליחת קוד

- שם ההודעה: CODE_FAILED
- נשלחת מ/אל: שרת → לקוח

- מבנה השדות:

USER_INTRO#CODE_RESPONSE#FAIL

7. אימות קוד מוצלח

- שם ההודעה: CODE_VALID
- נשלחת מ/אל: שרת → לקוח



- מבנה השדות:
USER_INTRO#CODE_VALIDATION#OK:{secret_code}
- דוגמה:
USER_INTRO#CODE_VALIDATION#OK:123456

8. אימות קוד כושל

- שם ההודעה: CODE_INVALID
- נשלחת מ/אל: שרת → לקוח

- מבנה השדות:
USER_INTRO#CODE_VALIDATION#NO

9. החזרת שם משתמש

- שם ההודעה: USERNAME_RESPONSE
- נשלחת מ/אל: שרת → לקוח

- מבנה השדות:
MAIN#USERNAME#{username}
- דוגמה:
MAIN#USERNAME#Yoav

10. החזרת רשימת פריטים

- שם ההודעה: DATA_RESPONSE
- נשלחת מ/אל: שרת → לקוח

- מבנה השדות:
MAIN#DATA#{json_array_of_items}
- דוגמה (אלו לא כל המשתנים שיש בעצם של הפריט):
MAIN#DATA#[{"1":{"team":"Maccabi Tel Aviv"}}, {"2":{"team":"Hapoel Tel Aviv"}}]

11. החזרת פריטים של משתמש

- שם ההודעה: USER_DATA_RESPONSE
- נשלחת מ/אל: שרת → לקוח

- מבנה השדות:
MAIN#DATA_FOR_USER#{json_array_with_status}
- דוגמה:
MAIN#DATA_FOR_USER#[{"1":{"team":"Hapoel Tel Aviv"},"@Available"}, {"2":{"team":"Maccabi Tel Aviv"},"@Sold"}]

12. אישור פרסום פריט

- שם ההודעה: PUBLISH_SUCCESS
- נשלחת מ/אל: שרת → לקוח

- מבנה השדות:
MAIN#PUBLISH_ITEM#OK



13. כישלון פרסום פריט

- שם ההודעה: PUBLISH_FAILED
- נשלחת מ/אל: שרת → לקוח

- מבנה השדות:
MAIN#PUBLISH_ITEM#{error_message}
- דוגמה:
MAIN#PUBLISH_ITEM#ERROR_INVALID_JSON

14. אישור קניית פריט

- שם ההודעה: BUY_SUCCESS
- נשלחת מ/אל: שרת → לקוח

- מבנה השדות:
MAIN#BUY_ITEM#OK

15. כישלון קניית פריט

- שם ההודעה: BUY_FAILED
- נשלחת מ/אל: שרת → לקוח

- מבנה השדות:
MAIN#BUY_ITEM#FAIL

16. אין נתונים

- שם ההודעה: NO_DATA
- נשלחת מ/אל: שרת → לקוח

- מבנה השדות:
MAIN#DATA#NO_DATA או MAIN#DATA_FOR_USER#NO_DATA

17. שגיאת אימות

- שם ההודעה: NOT_AUTHENTICATED
- נשלחת מ/אל: שרת → לקוח

- מבנה השדות:
MAIN#ERROR#NOT_AUTHENTICATED



תיאור מסכי המערכת

מסך כניסה:

1. **מסך ה"Log In"**: המסך בו הלקוח יכול להכנס לחשבון הקיים שלו

FanZone Marketplace

Log In

Enter your email

Enter your password

Log In

Forgot password

Dont have an account? SingUp

2. **מסך ה"Forgot Password"**: המסך בו הלקוח יכול לרשום את המייל ולקבל אליו קוד ישירות ללא הסיסמא במקרה שהקלוח שכח את הסיסמא שלו

FanZone Marketplace

Forgot Password

Choose an email to which a verification code will be sent:

Enter your email

Send Code

Back To Login



3. מסך ה"Sign Up": המסך בו הלקוח יכול ליצור לעצמו חשבון חדש

FanZone Marketplace

Sign Up

Enter your email

Enter your username

Enter your phone number

Enter your password

Re-Enter your password

Sign Up

Back To Login

4. מסך האימות דו שלבי: המסך בו הלקוח יצטרך להכניס את הקוד הנשלח אל כתובת המייל שלו בכדי להשלים את הכניסה למשתמש

FanZone Marketplace

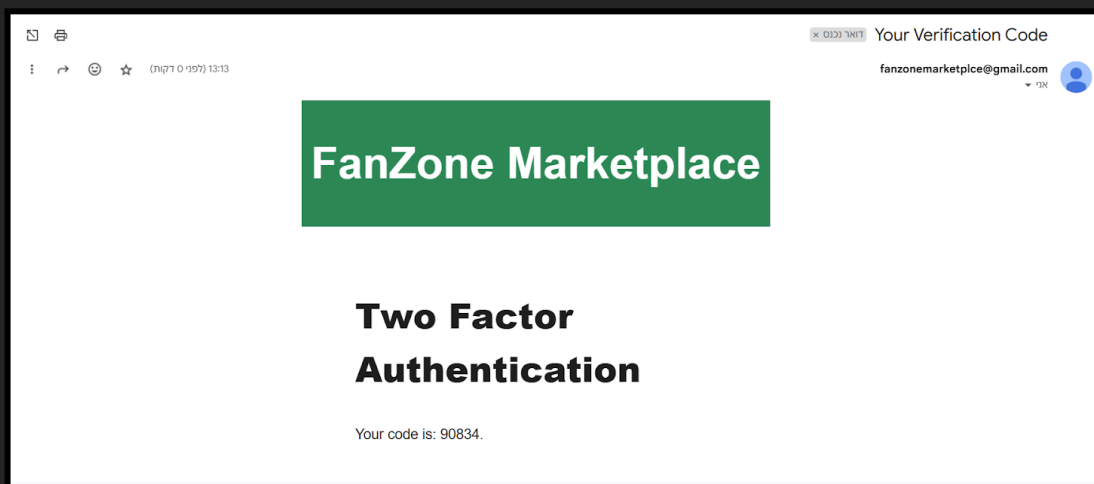
A code was sent to yoavi.naaman@gmail.com

Confirm

Resend Code

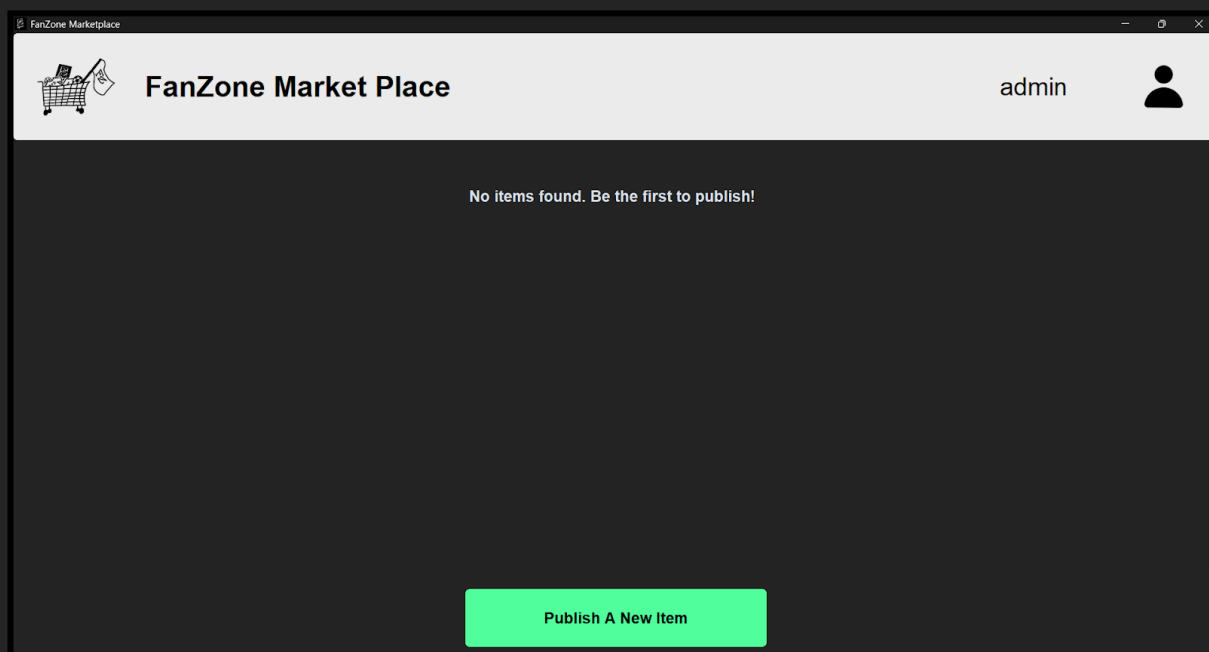


5. המייל של הקוד אשר נשלח לכתובת המייל של הלקוח:



מסך ראשי:

6. מסך הבית: המסך בו יוצגו כל המוצרים שהלקוח יכול לקנות





7. מסך פרסום מוצר (לפני שהוזנו פרטי המוצר): המסך בו הלקוח יוכל לפרסם מוצר

No image selected

Select Product Type

Select Size

Select Team

Enter The Description/Name Of The

The Year The Product Was Released

Price (In shekels)

City For Pickup

Load Image

Publish Item

8. מסך פרסום מוצר (לאחר שהוזנו פרטי המוצר):



Match-Worn Shirt

XL

Hapoel Tel Aviv

Double Season 2000 Matchworn

2000

250

Tel Aviv

Load Image

Publish Item



9. מסך פרסום מוצר (לאחר שפורסם המוצר בהצלחה):



Match-Worn Shirt

XL

Hapoel Tel Aviv

Double Season 2000 Matchworn

2000

250

Tel Aviv

Load Image

Publish Item

Item published successfully!

10. מסך המוצרים של הלקוח: המסך בו הלקוח יוכל לראות את כל המוצרים אותם פירסם

admin

Hapoel Tel Aviv



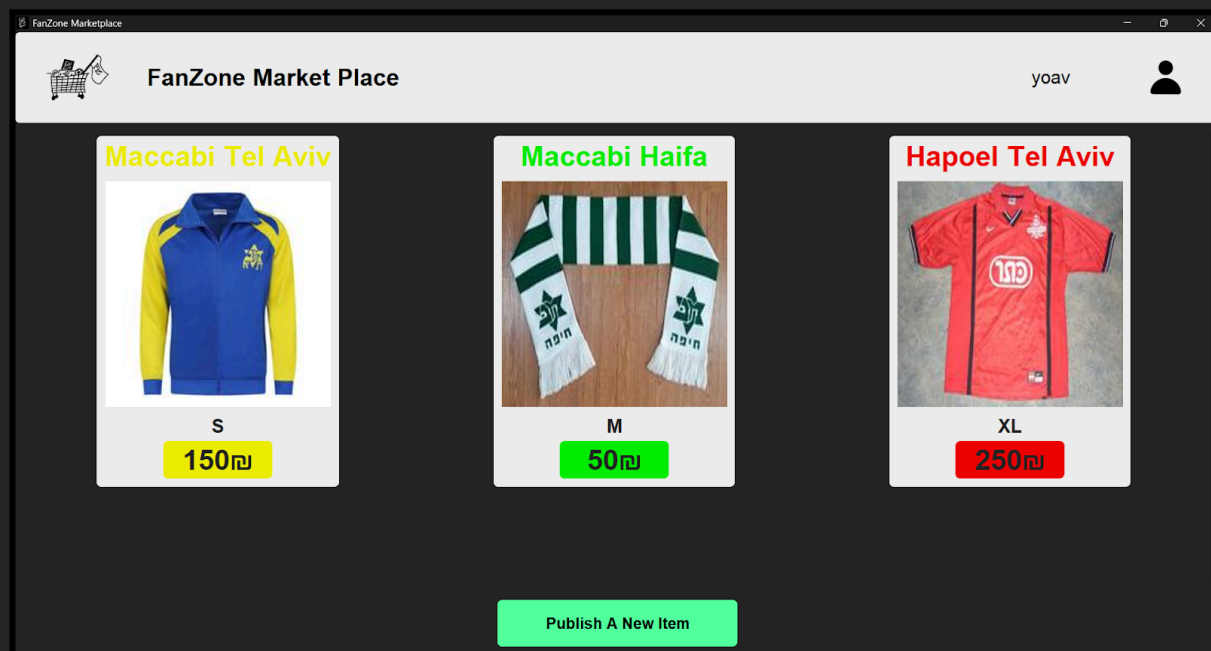
XL
250

ON SALE

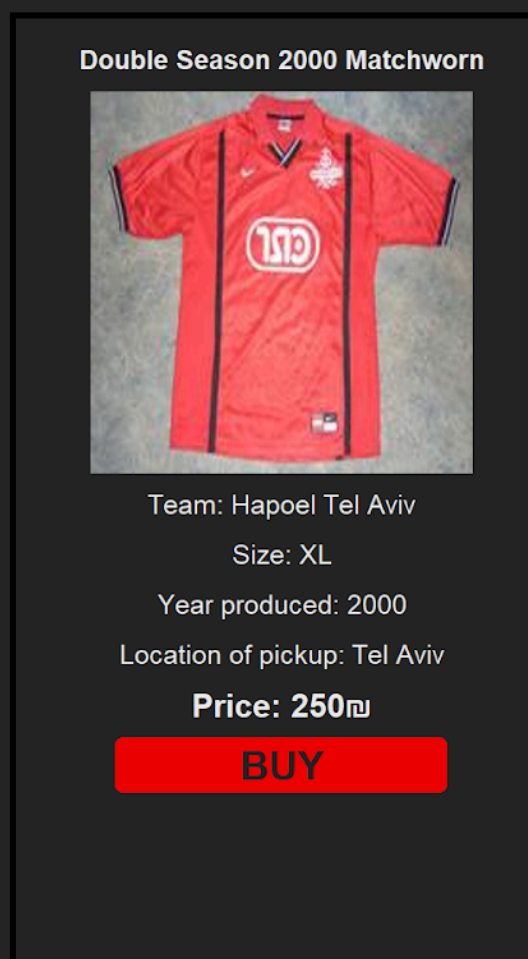
Maccabi Haifa



11. מסך הבית של חשבון אחר לאחר שפורסמו מוצרים:



12. מסך הקנייה של מוצר ספציפי: המסך הנפתח לאחר שהלקוח לחץ על הכפתור בו מופיע המחיר של המוצר, בו הלקוח רואה את כל פרטי המוצר ויוכל לקנות אותו במידה וירצה





13. מסך קנייה של מוצר ספציפי (לאחר שהמוצר נקנה בהצלחה):

Double Season 2000 Matchworn



Team: Hapoel Tel Aviv
Size: XL
Year produced: 2000
Location of pickup: Tel Aviv
Price: 250₪
BUY
Item bought successfully!

14. המייל שהקונה קיבל לאחר הקנייה:

FanZone Marketplace

Item Bought Successfully!

Your purchase of the item: Double Season 2000 Matchworn was successful!

Seller's contact information:

Email address: yoav.naaman@gmail.com

Phone number: 0543409321

We suggest you contact the seller as soon as possible for arranging the pickup.

Transaction information:

Item: Double Season 2000 Matchworn

Price: 250

Buyer: yoav

Seller: admin

Time And Date Of Transaction: 02-06-2025 13:33

Transaction ID: 541792

Item Bought Successfully!

fanzonemarketplace@gmail.com

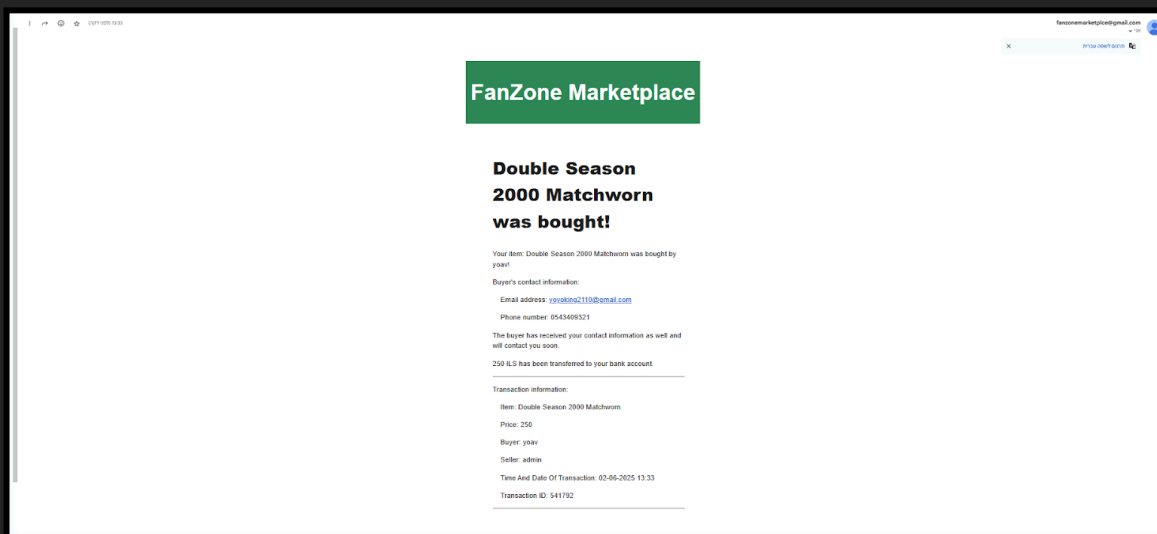
הודעה

אישור

ללא תחילה



15. המייל שהמוכר קיבל לאחר הקנייה:



16. מסך המוצרים של המשתמש Admin לאחר שהמוצר שלו נקנה:





תרשים זרימה של החלונות:





תיאור מבני הנתונים

קבצים:

קבצי הקוד:

- "main.py"
- "ctk_ui.py"
- "client_communication.py"
- "user_introduction.py"
- "main_windows.py"
- "encryption_client.py"
- "server.py"
- "server_communication.py"
- "database.py"
- "encryption_server.py"

קובץ בסיס הנתונים:

- "database.db"

קבצים נוספים:

- "FZ.ico"
- "FanZone.ico"
- "FanZone no bg.png"
- "FanZone no bg white.png"
- "default profile black.png"
- "default profile white.png"

בסיס הנתונים:

טבלה #1: UserPass - הטבלה בה מאוחסנים כל פרטי המשתמשים הקיימים.

שם	טיפוס	תיאור הערך	מאפיינים	דוגמא
User	TEXT	כתובת המייל של החשבון	NOT NULL, PRIMATY KEY	"yoav@gmail.com"
Password	TEXT	הסיסמא של החשבון - מוצפנת בהצפנת hash	NOT NULL	"b'\$2b\$12\$7TUjnCVIbx8wUzc8VfT1m.QkD0/F2J0uhclR5lm9owny56ReM1gH."
Username	TEXT	שם המשתמש של החשבון	NOT NULL	"yoav"



"0541234567"	NOT NULL	מספר הטלפון של החשבון	TEXT	PhoneNumber
--------------	----------	-----------------------	------	-------------

טבלה #2: ItemData - הטבלה בה מאוחסנים כל פרטי המוצרים הקיימים.

שם	טיפוס	תיאור הערך	מאפיינים	דוגמא
Id	INTEGER	מספר זהות ייחודי של המוצר (המספר נוצר באופן אוטומטי בעת הכנסת המוצר לטבלה)	NOT NULL, PRIMATY KEY, AUTOINCREMENT	"10"
Item	TEXT	הצגת string של עצם מסוג json בו מוכלים כל הפרטים של המוצר	NOT NULL	{ "item_type": "Match-Worn Shirt", "image": "https://i.imgur.com/eYgYNzv.jpeg", "item_title": "Double Season 2000 Matchworn", "size": "XL", "price": "250", "team": "Hapoel Tel Aviv", "year": "2000", "city_of_pickup": "Tel Aviv", "team_color": "red2", "secret_code": "816424", "submission_id": "41a018c6"}
Email	TEXT	כתובת המייל של מי שפרסם את המוצר	NOT NULL	"yoav@gmail.com"
Status	TEXT	משתנה המראה האם המוצר נקנה או שהוא עדיין זמין	NOT NULL	"Available" או "Sold"



טבלה #3: TransactionHistory - הטבלה בה מאוחסנים כל פרטי העסקאות שבוצעו.

שם	טיפוס	תיאור הערך	מאפיינים	דוגמא
Id	INTEGER	מספר זהות ייחודי של העסקה (המספר נוצר באופן אוטומטי בעת הכנסת פרטי העסקה לטבלה)	NOT NULL, PRIMATY KEY, AUTOINCREMENT	"351393"
Item_title	TEXT	הכותרת של המוצר שנקנה בעסקה	NOT NULL	"Double Season 2000 Matchworn"
buyer	TEXT	כתובת המייל של מי שקנה את המוצר	NOT NULL	"yoav@gmail.com"
Seller	TEXT	כתובת המייל של מי שמכר את המוצר	NOT NULL	"itay@gmail.com"
Date	TEXT	התאריך והשעה בו בוצעה העסקה	NOT NULL	"24-05-2025 12:37"
Price	INTEGER	המחיר של המוצר שנקנה	NOT NULL	"150"



סקירת חולשות ואיומים

שכבת האפליקציה:

- **SQL Injection (הזרקת SQL):** מכיוון שבסיס הנתונים בפרויקט (SQLite) הוא מבוסס SQL - הדבר חושף את המערכת לסיכון של הזרקת SQL. הזרקת SQL מתרחשת כאשר משתמש מזין בשדה הזנה (שתוכנו מיועד להישלח לשרת ולשמש כפרמטר בשאילתת SQL בצד השרת) הצהרת SQL שהשרת יפעיל על בסיס הנתונים. כך קיים סיכון שהמשתמש ישבש את בסיס הנתונים או שיוחזר לו מידע רגיש מבסיס הנתונים.

כדי למגר את הסיכויים ל-SQL injection, השתמשתי ב-Parameterized Queries בכל פעם שמתבצעת קליטת נתונים מהמשתמש. כל קלט שעתיד להיכנס לטבלה עובר דרך פרמטרים מופרדים (placeholders ?) המונעים מהקלט להיפרש כקוד SQL. לדוגמה:

```
self.cursor.execute("SELECT User, Password FROM UserPass WHERE User = ?", (username,))
```

- **עבודה עם אתרי Web:** המערכת משתמשת ב-Imgur API להעלאת תמונות. סיכון זה טופל על ידי:

- שימוש באימות API מאובטח עם Client-ID
- בדיקת תקינות התשובות מה-API לפני עיבוד
- מנגנון fallback למקרה של כישלון ב-API
- הגבלת גודל התמונות שניתן להעלות

- **תהליך כניסה והרשמה:** קיימת סכנה שאדם שמסניף פקטות שעוברות בין לקוח לשרת או שמשיג גישה לבסיס הנתונים יוכל לצפות בפרטי משתמש קריטיים ולפרוץ לחשבון. מסיבה זאת הטמתי במערכת 3 שכבות הגנה:

1. **הצפנת AES על כל ההודעות הנשלחות במערכת** - מפורט באלגוריתמים המרכזיים
2. **הצפנת סיסמאות** - הסיסמאות בבסיס הנתונים מוצפנות באמצעות bcrypt hashing.
3. **אימות דו שלבי** - ניתן להתחבר למערכת רק לאחר ביצוע אימות דו שלבי. לאחר הזנת מייל וסיסמה תקינים, נשלח לכתובת המייל של המשתמש קוד אימות אקראי בן 5 ספרות שנועד לאמת את זהותו

- **התקפת Man In The Middle:** מתקפת האזנה (הסנפה) למידע הקשור בין שני מחשבים על ידי התחזות של גורם שלישי לישות אמצעית שמעבירה את המידע. בסיכון זה טיפתי באמצעות:

- **הצפנה סימטרית AES** עם מפתח משותף שנוצר באמצעות Diffie-Hellman
- **IV אקראי** לכל הודעה למניעת התקפות replay
- כל התקשורת מוצפנת מקצה לקצה

- **התקפת DOS/DDOS:** במידה והמון לקוחות יתחברו בו זמנית וינסו להזין מידע שהשרת אמור לטפל בו, השרת לא ידע איך להגיב ולהתחבר לכולם, והדבר עלול להוביל לקריסתו. התקפת



DDOS/DOS יכולה להפיל את המערכת אך מכיוון שמדובר בפרויקט בית ספרי עם משאבים מוגבלים, סיכון זה לא טופל במלואו. עם זאת, יושומו מנגנונים בסיסיים:

- timeout על חיבורים
- ניהול חיבורים באמצעות select למניעת blocking
- הגבלת גודל הודעות (10MB מקסימום)

• **העלאת קבצים:** העלאת תמונות מטופלת באמצעות:

- בדיקת גודל קובץ לפני העלאה
- קידוד Base64 בטוח
- העלאה לשירות חיצוני (imgur) ולא לשרת המקומי
- בדיקת תקינות פורמט התמונה

שכבת התעבורה:

- **פרוטוקול TCP:** המערכת משתמשת בפרוטוקול TCP הכולל לחיצת יד משולשת אוטומטית. הפרוטוקול מבטיח:

- העברה אמינה של נתונים
- זיהוי וטיפול בחיבורים שבורים
- סדר נכון של ההודעות
- בקרת זרימה אוטומטית

• **הצפנה:** כל התקשורת ברשת מוצפנת באמצעות:

- חלוקת מפתחות: Diffie-Hellman Key Exchange
- הצפנה סימטרית: AES-128 במצב CBC
- IV אקראי: 16 בתים לכל הודעה
- Padding: PKCS7 להתאמה לבלוקים

הסבר על גיבוב סיסמאות:

כדי למנוע גניבת סיסמאות ממסד הנתונים השתמשתי באלגוריתם **bcrypt** במקום SHA-256. bcrypt הוא אלגוריתם איטי בכוונה שנועד למניעת brute force attacks ויוצר salt ייחודי לכל סיסמה.

האלגוריתם פועל באופן הבא:

1. יצירת salt אקראי לכל סיסמה
2. שילוב הסיסמה עם ה-salt
3. ביצוע מספר סיבובי הצפנה (cost factor)
4. שמירת התוצאה הכוללת את ה-salt וה-hash

יתרונות bcrypt על פני SHA-256:

- איטיות בכוונה המקשה על תקיפות



- salt ייחודי לכל סיסמה
- rainbow table attacks עמידות בפני
- תמיכה ב-cost factor מתכוון

סיכום החולשות שטופלו:

1. **SQL Injection** - טופל באמצעות Parameterized Queries
2. **חשיפת סיסמאות** - טופל באמצעות bcrypt hashing
3. **חשיפת תקשורת** - טופל באמצעות הצפנת AES
4. **אימות חלש** - טופל באמצעות 2FA
5. **תקיפות MITM** - טופל באמצעות הצפנה מקצה לקצה
6. **העלאת קבצים מסוכנת** - טופל באמצעות שירות חיצוני ובדיקות תקינות



מימוש הפרויקט

סקירת כל המודולים/מחלקות המרכיבים את המערכת

מודולים/מחלקות מיובאים:

שם המודול המיובא	למה מיועד המודול
socket	תקשורת בין שרת לבין לקוחות במחשבים שונים
threading	יצירת תהליכונים כדי לאפשר ריצה/הפעלה במקביל של יכולות מסוימות
select	חיבור ריבוי לקוחות ומשתמשים באמצעות I/O multiplexing
time	מדידה וקבלת מידע על זמן, השהיות בתוכנה
logging	רישום פעולות המערכת לצורך debug ומעקב
os	פעולות מערכת הפעלה כמו יצירת קבצים ונתיבים
customtkinter	יצירת ממשק משתמש גרפי (GUI) מתקדם ומעוצב
tkinter	בסיס לממשק משתמש גרפי
PIL (Image)	עיבוד ועריכת תמונות
json	טעינה וקריאה של נתונים בפורמט JSON



sqlite3	הרצת פקודות SQL לבסיס הנתונים מסוג SQLite
random	יצירת מספרים אקראיים לקודים סודיים ואימות
hashlib	גיבוב מידע עם פונקציות hash כמו SHA-256
bcrypt	הצפנת סיסמאות באמצעות hashing מתקדם
smtplib	שליחת דואר אלקטרוני
EmailMessage	יצירת הודעות דואר אלקטרוני מעוצבות
requests	ביצוע בקשות HTTP ל-APIs חיצוניים
base64	קידוד וענקוד של נתונים בינאריים
io	אובייקטי קובץ דמויים לעבודה עם נתונים בזיכרון
ast	פירוש וניתוח מבני נתונים של Python
re	בדיקת התאמה של מחרוזות לפורמט מסוים בעזרת regex
cryptography	ספריית הצפנה מתקדמת ל-AES ו-Diffie-Hellman

**מודולים/מחלקות שאני פיתחתי:****המחלקה:** Communication

תפקיד המחלקה: אחראית על התקשורת בין הלקוח לשרת, כולל הצפנה, שליחה וקבלה של הודעות.

תכונות המחלקה:

שם תכונה	תפקיד ושימוש
MAX_MSG_LENGTH	הגודל המקסימלי של הודעה שניתן לשלוח (10,485,760 בתים)
my_socket	אובייקט ה-socket לתקשורת עם השרת
iv	Initialization Vector לצורך הצפנת AES
en	אובייקט הצפנה מסוג Client_Encryption
connection_active	סטטוס החיבור לשרת (True/False)
in_msg	הודעה נכנסת מהשרת
out_msg	הודעה יוצאת לשרת
l	Lock object למניעת race conditions



פעולות במחלקה:

שם פעולה	טענת כניסה	טענת יציאה
init	אין פרמטרים	יוצרת חיבור לשרת ומאתחלת הצפנה
run	אין פרמטרים	שולחת הודעות יוצאות ומקבלת הודעות נכנסות
close_com	אין פרמטרים	סוגרת את החיבור לשרת

המחלקה: Client_Encryption

תפקיד המחלקה: מטפלת בהצפנה ופענוח של הודעות באמצעות Diffie-Hellman ו-AES.

תכונות המחלקה:

שם תכונה	תפקיד ושימוש
p	המספר הראשוני לאלגוריתם Diffie-Hellman
g	הגנרטור לאלגוריתם Diffie-Hellman
private_parameter	המפתח הפרטי של הלקוח
public_parameter	המפתח הציבורי של הלקוח



secret	המפתח הסודי המשותף
key	מפתח ה-AES שנגזר מהמפתח הסודי

פעולות במחלקה:

שם פעולה	טענת כניסה	טענת יציאה
init	אין פרמטרים	מאתחלת פרמטרים להחלפת מפתחות
generate_secret_key	client_parameter (str) - המפתח הציבורי של השרת	יוצרת מפתח AES משותף
encrypt	(message (str/bytes), iv (bytes	מחזירה הודעה מוצפנת (bytes)
decrypt	(message (bytes), iv (bytes	מחזירה הודעה מפוענחת (str)

המחלקה: Database

תפקיד המחלקה: מנהלת את בסיס הנתונים, כולל משתמשים, מוצרים והסטוריית עסקאות.

תכונות המחלקה:

שם תכונה	תפקיד ושימוש
logged_users	רשימת משתמשים מחוברים



code	קוד אימות זמני
processed_submissions	סט של submission IDs שכבר עובדו למניעת כפילויות
conn	חיבור למסד הנתונים SQLite
cursor	cursor לביצוע שאילתות SQL
api_client_id	מזהה לקוח ל-API ל-Imgur

פעולות במחלקה:

שם פעולה	טענת כניסה	טענת יציאה
init	אין פרמטרים	יוצרת חיבור למסד נתונים ומאתחלת טבלאות
upload_image	(str) img - תמונה מקודדת ב-Base64	מחזירה לינק לתמונה או None
AddToUserPass	username, password, user, (str) number	מחזירה "OK" או קוד שגיאה
AddData	(str) email, (str) json_item	מחזירה "OK" או קוד שגיאה
GetData	email (str), page=1, items_per_page=12	מחזירה JSON עם רשימת מוצרים



מחזירה JSON עם מוצרים של המשתמש	(email (str	GetDataForUser
מחזירה dictionary עם פרטי העסקה	item_id, buyer, price, (item_title (str	BuyItem
מחזירה "LOGIN_SUCCESS" או שגיאה	(user_password (str	login_compare
מחזירה שם משתמש או "Unknown User"	(email (str	get_user
סוגרת חיבור למסד נתונים	אין פרמטרים	Disconnect

המחלקה: Server_com

תפקיד המחלקה: מנהלת את השרת הרב-לקוחי, כולל קבלת חיבורים וניהול הודעות.

תכונות המחלקה:

שם תכונה	תפקיד ושימוש
MAX_MSG_LENGTH	הגודל המקסימלי של הודעה
SERVER_PORT	פורט השרת (5678)
SERVER_IP	כתובת IP של השרת
server_socket	socket ראשי של השרת



client_sockets	רשימת sockets של לקוחות מחוברים
client_secret_codes	רשימת קודים סודיים של לקוחות
out_msgs	תור הודעות יוצאות
in_msgs	תור הודעות נכנסות
en	אובייקט הצפנה של השרת

פעולות במחלקה:

שם פעולה	טענת כניסה	טענת יציאה
init	אין פרמטרים	מקים שרת ומאתחל משתנים
add_to_out	(tuple msg) - הודעה ו-socket יעד	מוסיפה הודעה לתור הנתונים היוצא
run	אין פרמטרים	מנהל חיבורים ומעבד הודעות
handle_client_disconnect	socket	מנקה נתונים של לקוח שהתנתק
shutdown	אין פרמטרים	סוגר את השרת וכל החיבורים



המחלקה: App

תפקיד המחלקה: מנהלת את הממשק הראשי של האפליקציה ומתאמת בין המרכיבים.

תכונות המחלקה:

שם תכונה	תפקיד ושימוש
u	אובייקט UserIntro לניהול ההתחברות

פעולות במחלקה:

שם פעולה	טענת כניסה	טענת יציאה
init	root (CTk) - חלון ראשי	מאתחלת ממשק ויוצרת אובייקט UserIntro
handle_in_msg	msg (str) - הודעה נכנסת	מעבירה הודעות למרכיב המתאים

המחלקה: UserIntro

תפקיד המחלקה: מנהלת תהליכי התחברות, הרשמה ואימות דו-שלבי.

תכונות המחלקה:

שם תכונה	תפקיד ושימוש
root	חלון ראשי של האפליקציה



צבעי ערכת הנושא	defaultbg, opposite_color, white_or_black
פרטי המשתמש	user, password
קוד סודי לאחר התחברות מוצלחת	secret_code
הודעות נכנסות ויוצאות	in_message, out_message
אובייקט MainWin לאחר התחברות מוצלחת	m

פעולות במחלקה:

שם פעולה	טענת כניסה	טענת יציאה
init	root, defaultbg, opposite_color, white_or_black	מאתחלת ממשק התחברות
Send_Message	(msg (str), msg_type (str)	שולחת הודעה לשרת
login	אין פרמטרים	יוצרת מסך התחברות
log_pressed	אין פרמטרים	מטפלת בלחיצה על כפתור התחברות
sign_up	אין פרמטרים	יוצרת מסך הרשמה



שולחת בקשה לקוד אימות	אין פרמטרים	code_send
בודקת קוד אימות שהוזן	אין פרמטרים	check_code

המחלקה: MainWin

תפקיד המחלקה: מנהלת את הממשק הראשי לאחר התחברות, כולל צפייה ופרסום מוצרים.

תכונות המחלקה:

שם תכונה	תפקיד ושימוש
user	שם המשתמש המחובר
secret_code	קוד סודי לאימות פעולות
scrollable_frame_items	רשימת אלמנטים בגלילה
in_message, out_message	הודעות נכנסות ויוצאות
screen_width, screen_height	רזולוציית המסך



פעולות במחלקה:

שם פעולה	טענת כניסה	טענת יציאה
init	root, defaultbg, opposite_color, white_or_black, secret_code	מאתחלת ממשק ראשי
Send_Message	(data (str	שולחת הודעה לשרת
landing_page	אין פרמטרים	יוצרת דף הבית עם רשימת מוצרים
load_items	אין פרמטרים	טוענת מוצרים מהשרת
publish	אין פרמטרים	פותחת חלון פרסום מוצר
load_image	אין פרמטרים	טוענת תמונה למוצר
buy_item	item_id, price, item_title, window	מבצעת רכישת מוצר
profile_pressed	אין פרמטרים	מציגה מוצרים של המשתמש



קטעי קוד האלגוריתמים המרכזיים

הצפנת תקשורת באמצעות AES ו-Diffie-Hellman

הסבר על היכולת: הקוד מטפל ביצירת מפתח הצפנה משותף ובהצפנת ההודעות לפני שליחה.

```
class Client_Encryption:

    def __init__(self):

        # Diffie-Hellman פרמטרים ציבוריים לאלגוריתם

        self.p = 23 # מספר ראשוני

        self.g = 5 # גנרטור

        # יצירת מפתח פרטי אקראי

        self.private_parameter = random.randint(1, self.p-1)

        # חישוב מפתח ציבורי באמצעות החזקה מודולרית

        self.public_parameter = pow(self.g, self.private_parameter, self.p)

        self.secret = None # המפתח הסודי המשותף

        self.key = None # שנגזר מהמפתח הסודי AES מפתח

    def generate_secret_key(self, server_public):

        # יצירת מפתח סודי משותף מהמפתח הציבורי של השרת

        self.secret = pow(int(server_public), self.private_parameter, self.p)

        # SHA-256 באמצעות המרת המפתח הסודי למפתח

        self.key = sha256(str(self.secret).encode()).digest()[16]

    def encrypt(self, message, iv):

        # המרת מחרוזת לבתים במידת הצורך

        if isinstance(message, str):

            message = message.encode('utf-8')
```



```
# PKCS7 להודעה באמצעות padding הוספת
```

```
from cryptography.hazmat.primitives import padding
```

```
padder = padding.PKCS7(128).padder()
```

```
padded_data = padder.update(message)
```

```
padded_data += padder.finalize()
```

```
# CBC במצב AES הצפנת ההודעה באמצעות
```

```
cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv))
```

```
encryptor = cipher.encryptor()
```

```
return encryptor.update(padded_data) + encryptor.finalize()
```

```
def decrypt(self, message, iv):
```

```
# AES פענוח ההודעה באמצעות
```

```
cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv))
```

```
decryptor = cipher.decryptor()
```

```
padded_data = decryptor.update(message) + decryptor.finalize()
```

```
# padding-הסרת ה
```

```
from cryptography.hazmat.primitives import padding
```

```
unpadder = padding.PKCS7(128).unpadder()
```

```
data = unpadder.update(padded_data)
```

```
data += unpadder.finalize()
```

```
return data.decode('utf-8')
```

**ניהול תקשורת רב-לקוחי באמצעות Select:**

הסבר על היכולת: השרת מנהל מספר לקוחות בו-זמנית ללא blocking באמצעות select.

```
def run(self):

    """Main server loop that handles connections and messages"""

    try:

        # blocking למניעת timeout עם select-שימוש ב
        ready_to_read, ready_to_write, in_error = select.select(

            [self.server_socket] + self.client_sockets, # לקריאה sockets
            self.client_sockets, # sockets לכתובה
            [], # sockets עם שגיאות
            0.1 # של 0.1 שניות timeout

        )

        # טיפול בהודעות נכנסות וחיבורים חדשים

        self.inL.acquire()

        try:

            for current_socket in ready_to_read:

                if current_socket is self.server_socket:

                    # חיבור לקוח חדש

                    try:

                        (client_socket, client_address) = current_socket.accept()

                        client_socket.settimeout(5) # לפעולות timeout

                        logging.info(f"New client connected: {client_address}")

                    except:

                        pass

                    # Diffie-Hellman ביצוע חלוקת מפתחות

                    client_socket.send(str(self.en.public_parameter).encode())

                    self.en.generate_secret_key(client_socket.recv(1024).decode())

                    self.client_sockets.append(client_socket)
```



```
except Exception as e:

    logging.error(f"Error accepting client connection: {e}")

else:

    # קבלת נתונים מלקוח קיים

    data = ""

    try:

        # ולאחר מכן הנתונים המוצפנים IV קבלת

        self.iv = current_socket.recv(16)

        if self.iv != "":

            recv_data = current_socket.recv(self.MAX_MSG_LENGTH)

            if recv_data:

                # פענוח ההודעה

                data = self.en.decrypt(recv_data, self.iv)

            else:

                # לקוח התנתק

                self.handle_client_disconnect(current_socket)

                continue

    except ConnectionResetError:

        # טיפול בניתוק פתאומי

        self.handle_client_disconnect(current_socket)

        continue

    if data:

        # הוספת ההודעה לתור העיבוד

        self.in_msgs.append((current_socket, data))

finally:

    self.inL.release()
```



#שליחת הודעות יוצאות

```
self.outL.acquire()
```

```
try:
```

```
    for message in self.out_msgs[:]: # עותק של הרשימה
```

```
        current_socket, data = message
```

```
        if current_socket in ready_to_write:
```

```
            try:
```

```
                # חדש והצפנת ההודעה IV יצירת
```

```
                self.iv = os.urandom(16)
```

```
                current_socket.send(self.iv)
```

```
                current_socket.send(self.en.encrypt(data, self.iv))
```

```
                self.out_msgs.remove(message)
```

```
            except Exception as e:
```

```
                logging.error(f"Error sending message: {e}")
```

```
                self.handle_client_disconnect(current_socket)
```

```
                self.out_msgs.remove(message)
```

```
        finally:
```

```
            self.outL.release()
```

```
except Exception as e:
```

```
    logging.error(f"Error in server run loop: {e}")
```

**אימות דו-שלבי עם שליחת מייל:**

הסבר על היכולת: יצירת קוד אימות אקראי ושליחתו במייל HTML מעוצב.

```
def send_mail(mail, data):

    """שליחת קוד אימות למייל המשתמש"""

    global codes, server

    sender_email = 'fanzonemarketplce@gmail.com'

    sender_password = 'mdxgiaebdaxehkug' # App Password של Gmail

    # מעוצבת HTML יצירת הודעת
    msg = EmailMessage()

    msg['Subject'] = 'Your Verification Code'

    msg['From'] = sender_email

    msg['To'] = mail

    msg.set_content(f"""

    <!DOCTYPE html>

    <html>

    <head>

        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">

        <style type="text/css">

            h1{{font-size:56px}}

            h2{{font-size:45px;font-weight:900}}

            p{{font-weight:100}}

            #email{{margin:auto;width:600px;background-color:#fff}}

        </style>

    </head>

    <body bgcolor="#F5F8FA" style="font-family:Lato, sans-serif, font-size:18px;">

    <div id="email">

        <table role="presentation" width="100%">

    """)
```



```
<tr>

    <td bgcolor="#2E8B57" align="center" style="color: white;">

        <h1> FanZone Marketplace</h1>

    </td>

</tr>

</table>

<table role="presentation" style="padding: 30px 30px 30px 60px;">

    <tr>

        <td>

            <h2>Two Factor Authentication</h2>

            <p>Your code is: {data}.</p>

        </td>

    </tr>

</table>

</div>

</body>

</html>

"', subtype='html')

try:
```

```
# והשליחה Gmail התחברות לשרת
```

```
with smtplib.SMTP_SSL('smtp.gmail.com', 465) as smtp:
```

```
    smtp.login(sender_email, sender_password)
```

```
    smtp.send_message(msg)
```

```
logging.info(f'Verification code {data} sent to {mail}')
```

```
# שליחת אישור לכל הלקוחות
```

```
for sock in server.client_sockets:
```



```
server.outL.acquire()

server.out_msgs.append((sock, 'USER_INTRO#CODE_RESPONSE#OK'))

server.outL.release()

# שמירת הקוד ברשימת הקודים הפעילים
codes.append(f'{mail}#{data}')

logging.info(f'Active codes: {codes}')

except Exception as e:

    logging.error(f'Failed to send email to {mail}. Error: {e}')

# שליחת הודעת שגיאה

for sock in server.client_sockets:

    server.outL.acquire()

    server.out_msgs.append((sock, 'USER_INTRO#CODE_RESPONSE#FAIL'))

    server.outL.release()
```

עיבוד וניתוח JSON עם מנגנון :Fallback

הסבר על היכולת: פירוש נתוני JSON מורכבים עם טיפול בנתונים פגומים.

```
def process_items_list(self, json_list):

    """עיבוד רשימת מוצרים עם טיפול בשגיאות"""

    # ניקוי רשימת האלמנטים הקיימת

    for widget in self.scrollable_frame_items:

        if widget.winfo_exists():

            widget.destroy()

    self.scrollable_frame_items = []

    self.row = 0

    self.col = 0
```



```
max_cols = 3
```

```
logging.info(f"Processing {len(json_list)} items")
```

```
for i, json_item in enumerate(json_list):
```

```
    try:
```

```
        if not json_item:
```

```
            continue
```

```
        # טיפול בפורמטים שונים של נתונים
```

```
        if isinstance(json_item, str):
```

```
            # חלוקת item_id@json_data
```

```
            if '@' in json_item:
```

```
                parts = json_item.split('@', 1)
```

```
                if len(parts) == 2:
```

```
                    item_id = parts[0].strip()
```

```
                    json_data = parts[1].strip()
```

```
                else:
```

```
                    json_data = json_item
```

```
            else:
```

```
                json_data = json_item
```

```
        # ניקוי נתונים מתווים מיותרים
```

```
        json_data = json_data.strip()
```

```
        if (json_data.startswith('"') and json_data.endswith('"')):
```

```
            json_data = json_data[1:-1]
```

```
        # ב-escape characters טיפול ב
```



```
        json_data = json_data.replace("\\'", "'").replace("'\\'", "'")

    else:

        # נתונים שכבר פורסרו

        json_data = json_item

    # יצירת והצגת האלמנט

    self.pack_items_to_frame(json_data, item_id, self.row, self.col)

    # מעבר לעמודה הבאה

    self.col += 1

    if self.col >= max_cols:

        self.col = 0

        self.row += 1

except Exception as e:

    logging.error(f"Error processing item {i}: {e}")

    # יצירת אינדיקטור שגיאה

    error_frame = ctk.CTkFrame(self.scrollable_frame, fg_color="red")

    error_frame.configure(width=int(self.screen_width * 0.2),

                           height=int(self.screen_width * 0.2 * 0.5))

    error_frame.grid(row=self.row, column=self.col, padx=10, pady=10)

    error_label = ctk.CTkLabel(error_frame, text="Error loading item",

                               font=("Arial", 14, "bold"), text_color="white")

    error_label.place(relx=0.5, rely=0.5, anchor="center")

    self.scrollable_frame_items.append(error_frame)
```



```
מעבר לפוזיציה הבאה  
  
self.col += 1  
  
if self.col >= max_cols:  
  
    self.col = 0  
  
    self.row += 1
```

העלאת תמונות ל-API חיצוני עם עיבוד:

הסבר על היכולת: קטינת גודל תמונה, קידוד ב-Base64 והעלאה ל-Imgur API.

```
def load_image(self):  
  
    """טעינת ועיבוד תמונה למוצר"""  
  
    try:  
  
        import tkinter as tk  
  
        from tkinter import filedialog  
  
  
        # בחירת קובץ תמונה  
  
        file_path = filedialog.askopenfilename(  
  
            filetypes=[("Image files", "*.jpg *.jpeg *.png *.gif *.bmp *.tiff")]  
  
        )  
  
  
        if not file_path:  
  
            return  
  
  
        self.image_file_path = file_path  
  
        logging.info(f"Selected image: {file_path}")  
  
  
        # לעיבוד התמונה PIL-שימוש ב
```



```
from PIL import Image, ImageTk

pil_image = Image.open(file_path)

logging.info(f"Image size: {pil_image.width}x{pil_image.height}")

# קטינת גודל התמונה במידת הצורך
max_size = 200

if pil_image.width > max_size or pil_image.height > max_size:

    # שמירה על יחס גובה-רוחב
    ratio = min(max_size / pil_image.width, max_size / pil_image.height)

    new_width = int(pil_image.width * ratio)

    new_height = int(pil_image.height * ratio)

    pil_image = pil_image.resize((new_width, new_height), Image.LANCZOS)

    logging.info(f"Resized to: {new_width}x{new_height}")

# שמירת התמונה לשימוש עתידי
self.image_pil = pil_image

# להצגה בממשק PhotoImage-המרה ל
photo = ImageTk.PhotoImage(pil_image)

self.photo_reference = photo # מניעת garbage collection

# הצגת התמונה בממשק
self.image_field.config(image=photo)

self.image_field.update()

# שמירת נתונים בינאריים להעלאה
import io

buffer = io.BytesIO()
```



```
img_format = pil_image.format if pil_image.format else 'JPEG'

pil_image.save(buffer, format=img_format)

self.binary_data = buffer.getvalue()

logging.info("Image loaded and displayed successfully")

except Exception as e:

    logging.error(f"Error loading image: {e}")

def upload_image(self, img):

    """API-Imgur העלאת תמונה ל"""

    logging.info("Uploading image")

    try:

        # API key עם headers הגדרת

        headers = {"Authorization": f"Client-ID {self.api_client_id}"}

        data = {"image": img} # Base64-תמונה מקודדת ב

        # API-שליחת בקשה ל

        response = requests.post("https://api.imgur.com/3/image",

                                headers=headers, data=data)

        response.raise_for_status()

        # חילוץ לינק התמונה מהתשובה

        image_url = response.json()["data"]["link"]

        logging.info(f"Uploaded image successfully: {image_url}")

        return image_url

    except requests.exceptions.RequestException as e:
```



```
logging.error(f"Request error: {e}")

return None

except KeyError:

    logging.error("Error: Could not extract image URL from response.")

    return None

except Exception as e:

    logging.error(f"An unexpected error occurred: {e}")

    return None
```

שאלות SQL בטוחות עם Parameterized Queries:

הסבר על היכולת: מניעת SQL injection באמצעות פרמטרים מופרדים.

```
def BuyItem(self, item_id, buyer, price, item_title):

    """רכישת מוצר עם עדכון מסד נתונים ושליחת מיילים"""

    try:

        # עדכון סטטוס המוצר לנמכר - שימוש בפרמטרים בטוחים

        self.cursor.execute("""

            UPDATE ItemData

            SET Status = 'Sold'

            WHERE Id = ?

            """, (item_id,))

        self.conn.commit()

        # שליפת פרטי המוכר

        self.cursor.execute("""

            SELECT Email FROM ItemData

            WHERE Id = ?

            """, (item_id,))
```



```
seller_result = self.cursor.fetchone()

if not seller_result:

    logging.error(f"No item found with ID: {item_id}")

    return {"response": "ERROR", "message": "Item not found"}

seller_email = seller_result[0]

# שליפת פרטי הקונה והמוכר לצורך המיילים

self.cursor.execute("SELECT Username, PhoneNumber FROM UserPass WHERE User = ?",

                    (buyer,))

buyer_contact_result = self.cursor.fetchall()

self.cursor.execute("SELECT Username, PhoneNumber FROM UserPass WHERE User = ?",

                    (seller_email,))

seller_contact_result = self.cursor.fetchall()

# בדיקת קיום הנתונים

if not buyer_contact_result or not seller_contact_result:

    return {"response": "ERROR", "message": "Contact info not found"}

buyer_contact = buyer_contact_result[0]

seller_contact = seller_contact_result[0]

# יצירת פרטי עסקה

from datetime import datetime

time_of_purchase = datetime.now().strftime("%d-%m-%Y %H:%M")

transaction_id = random.randint(100000, 999999)
```



```
# הכנת נתוני התשובה
```

```
data = {  
  
    "seller_mail": seller_email,  
  
    "seller_user": seller_contact[0],  
  
    "seller_phone": seller_contact[1],  
  
    "buyer_user": buyer_contact[0],  
  
    "buyer_phone": buyer_contact[1],  
  
    "time": time_of_purchase,  
  
    "transaction_id": transaction_id,  
  
    "response": "OK",  
  
}
```

```
# שמירת פרטי העסקה בהיסטוריה
```

```
self.cursor.execute("""  
  
    INSERT INTO TransactionHistory (Id, Item_title, Buyer, Seller, Date, Price)  
  
    VALUES (?, ?, ?, ?, ?, ?)  
  
    """, (transaction_id, item_title, buyer, seller_email,  
  
        time_of_purchase, price))  
  
self.conn.commit()
```

```
logging.info(f"Transaction {transaction_id} completed successfully")  
  
return data
```

```
except Exception as e:
```

```
    logging.error(f"Error in BuyItem: {e}")  
  
    return {"response": "ERROR", "message": str(e)}
```



מסמך בדיקות מלא

הבדיקות שתוכננו בשלב האפיון:

בדיקה 1: תקינות מנגנון ההצפנה והפענוח

מטרת הבדיקה: ווידוא שהצפנת AES עם Diffie-Hellman עובדת כראוי והמידע מוצפן ומפוענח נכון.

מה בוצע בפועל: הדפסת המידע לפני ההצפנה, לאחר ההצפנה ולאחר הפענוח. בדיקת יצירת מפתחות משותפים זהים בצד הלקוח והשרת.

תוצאות הבדיקה:

- המידע הוצפן ופוענח בהצלחה
- המפתחות הסודיים שנוצרו בשני הצדדים היו זהים
- IV אקראי נוצר נכון לכל הודעה

בעיות שהתגלו וכיצד נפתרו:

- **בעיה:** בהתחלה השתמשתי בפרמטרים קטנים מדי ל- $p=23$ (Diffie-Hellman) מה שלא מספיק בטוח
- **פתרון:** למרות שבפרויקט בית ספרי השארתי $p=23$ לפשטות, בפרויקט אמיתי היה צריך להשתמש במספרים ראשוניים גדולים בהרבה

בדיקה 2: אימות כניסה וסיסמאות

מטרת הבדיקה: לוודא שמערכת ההתחברות בטוחה ומונעת כניסה לא מורשית.

מה בוצע בפועל:

- ניסיון התחברות עם פרטים נכונים ולא נכונים
- בדיקת הצפנת סיסמאות במסד הנתונים
- ניסיון התחברות עם סיסמה מוצפנת ישירות

תוצאות הבדיקה:

- התחברות עם פרטים נכונים הצליחה
- התחברות עם פרטים שגויים נדחתה
- הסיסמאות במסד הנתונים מוצפנות עם bcrypt ולא ניתן לפענח אותן

בעיות שהתגלו וכיצד נפתרו:

- **בעיה:** בהתחלה הסיסמאות נשמרו בטקסט פשוט
- **פתרון:** מעבר להצפנת bcrypt עם salt אוטומטי

**בדיקה 3: אימות דו-שלבי ושליחת מיילים**

מטרת הבדיקה: לוודא שקודי האימות נשלחים נכון למייל ומאומתים כראוי.

מה בוצע בפועל: השתמשתי בכתובת המייל שלי והתנסיתי כמשתמש קצה בתהליך ההרשמה וההתחברות.

תוצאות הבדיקה:

- המיילים נשלחו בהצלחה עם עיצוב HTML
- קודי האימות עבדו נכון
- קודים לא תקפים נדחו

בעיות שהתגלו וכיצד נפתרו:

- בעיה: בהתחלה המיילים הגיעו לתיקיית הספאם
- פתרון: שיפור עיצוב ההודעה ושימוש ב-App Password של Gmail במקום סיסמה רגילה

בדיקה 4: תפקוד בסיס הנתונים ומניעת SQL Injection

מטרת הבדיקה: לוודא שבסיס הנתונים מוגן מפני תקיפות SQL Injection ומנהל מידע נכון.

מה בוצע בפועל:

- הזנת מחרוזות עם תווי SQL מיוחדים בשדות הממשק
- ניסיון להזין קודי SQL בשדות המייל והסיסמה
- בדיקת שמירה ושליפה של נתונים עם תווים מיוחדים

תוצאות הבדיקה:

- כל הנתונים נשמרו ונשלפו נכון
- ניסיונות SQL Injection נכשלו
- המערכת לא קרסה גם בעת הזנת נתונים חריגים

בעיות שהתגלו וכיצד נפתרו:

- בעיה: בעיות עם נתוני JSON מורכבים שהכילו תווים מיוחדים
- פתרון: שימוש בפרמטרים מופרדים בכל שאילתות ה-SQL ומנגנון fallback לפירוש JSON פגום

בדיקה 5: העלאת ועיבוד תמונות

מטרת הבדיקה: לוודא שהעלאת תמונות ל-Imgur API עובדת ותמונות מוצגות נכון.

מה בוצע בפועל:

- העלאת תמונות בפורמטים שונים (JPG, PNG)
- העלאת תמונות בגדלים שונים
- בדיקת הצגת התמונות בממשק הלקוח

**תוצאות הבדיקה:**

- תמונות הועלו בהצלחה ל-Imgur
- לינקים לתמונות נשמרו נכון במסד הנתונים
- תמונות הוצגו נכון בממשק עם קטינת גודל אוטומטית

בעיות שהתגלו וכיצד נפתרו:

- בעיה: תמונות גדולות גרמו לאיטות בממשק ובעיות בתקשורת
- פתרון: קטינת גודל אוטומטית לתמונות גדולות מ-200x200 פיקסלים

בדיקות נוספות שבוצעו למערכת:**בדיקה 6: ניהול חיבורים רב-לקוחי**

מטרת הבדיקה: לוודא שהשרת יכול לטפל במספר לקוחות בו-זמנית ללא קריסות.
מה בוצע בפועל: הרצת מספר לקוחות בו-זמנית ממחשבים שונים ובדיקת התנהגות השרת.

תוצאות הבדיקה:

- השרת טיפל בהצלחה ב-5 לקוחות בו-זמנית
- כל לקוח קיבל קוד סודי ייחודי
- הודעות הגיעו ללקוח הנכון

בעיות שהתגלו וכיצד נפתרו:

- בעיה: כאשר לקוח התנתק פתאום, השרת לא ניקה את הנתונים שלו
- פתרון: הוספת מנגנון `handle_client_disconnect` שמנקה נתונים של לקוחות שהתנתקו

בדיקה 7: טיפול בתקלות רשת

מטרת הבדיקה: לבדוק איך המערכת מתמודדת עם ניתוק פתאומי מהאינטרנט.
מה בוצע בפועל: ניתוק מכון מה-WiFi במהלך שליחת מייל ופעולות שדורשות חיבור.

תוצאות הבדיקה:

- המערכת לא קרסה כאשר לא היה חיבור לאינטרנט
- הוצגו הודעות שגיאה ברורות למשתמש
- כשהחיבור חזר, המערכת המשיכה לפעול נורמלית

בעיות שהתגלו וכיצד נפתרו:

- בעיה: שליחת מיילים קרסה כאשר לא היה חיבור לאינטרנט
- פתרון: הוספת `try-catch` לכל פעולות הרשת עם הודעות שגיאה מתאימות

**בדיקה 8: העברת נתונים גדולים**

מטרת הבדיקה: לוודא שהמערכת יכולה להעביר תמונות גדולות וכמויות גדולות של מוצרים.

מה בוצע בפועל:

- הוספת מוצרים עם תמונות גדולות (מעל 5MB)
- יצירת מסד נתונים עם מעל 100 מוצרים
- בדיקת זמני תגובה

תוצאות הבדיקה:

- תמונות גדולות הועלו בהצלחה אחרי קטינת גודל
- רשימת מוצרים גדולה נטענה תוך מספר שניות
- לא היו זליגות זיכרון

בעיות שהתגלו וכיצד נפתרו:

- בעיה: תמונות גדולות מדי גרמו לעלייה בזמן התגובה
- פתרון: הגדרת מגבלת גודל של 1MB לתמונות אחרי קידוד Base64

בדיקה 9: עמידות בפני תקלות משתמש

מטרת הבדיקה: לבדוק איך המערכת מגיבה לפעולות לא צפויות של המשתמש.

מה בוצע בפועל:

- לחיצה מהירה ומרובה על כפתורים
- סגירת חלונות באמצע פעולות
- הזנת נתונים לא תקינים בשדות שונים

תוצאות הבדיקה:

- המערכת לא קרסה בעקבות פעולות חריגות
- הוצגו הודעות שגיהא ברורות למשתמש
- כל הפעולות היו הפיכות

בעיות שהתגלו וכיצד נפתרו:

- בעיה: לחיצה מרובה על "פרסם מוצר" יצרה מוצרים כפולים
- פתרון: הוספת דגל publishing_in_progress למניעת לחיצות מרובות

בדיקה 10: אבטחת מידע ופרטיות

מטרת הבדיקה: לוודא שמידע רגיש לא נחשף ללא הרשאה.

מה בוצע בפועל:

- ניסיון גישה למוצרים של משתמשים אחרים
- בדיקת הצפנת התקשורת באמצעות Wireshark
- ניסיון עקיפת מנגנון האימות



תוצאות הבדיקה:

- כל התקשורת הייתה מוצפנת ולא ניתן היה לקרוא אותה
- לקוחות לא יכלו לגשת למידע של משתמשים אחרים
- מנגנון הקודים הסודיים מנע עקיפת האימות

בעיות שהתגלו וכיצד נפתרו:

- בעיה: בתחילה לא היה timeout על קודי האימות
- פתרון: הוספת מנגנון שמוחק קודים ישנים מהזיכרון



מדריך למשתמש

עץ קבצים

```
client_communication.py
ctk_ui.py
database.db
database.py
default profile black.png
default profile white.png
encryption_client.py
encryption_server.py
FanZone.ico
FanZone no bg.png
FanZone no bg white.png
FZ.ico
main.py
main_windows.py
server.py
server_communication.py
user_introduction.py
```

התקנת מערכת

סביבת עבודה נדרשת:

בכדי לערוך ולהריץ את הקוד נדרש שיהיה מותקן על המחשב גרסה של Python 3.13 ומעלה, ושכל הספריות המיובאות בקוד יהיו מורדות על המחשב (ניתן להוריד אותן בעזרת PIP Install ואת שם הספרייה)



הכלים הנדרשים:

בכדי לערוך את הקוד יש להשתמש בתוכנה המיועדת לכך, כמו: PyCharm, Visual Studio ועוד.
את הקוד עצמו ניתן להריץ דרך פקודה ב CDM של:

`Python server.py`

אך חשוב לוודא כי כל הקבצים של המערכת נמצאים באותה תקייה, וכי ה CMD מורץ על התיקיה הזאת.

בכדי להתבונן בטבלאות בבסיס הנתונים יש להשתמש בתוכנה המיועדת לכך, לדוגמא: DB Browser.

רשת:

כדי שהתכנה תעבוד בכל התקשורת ושליחת המיילים יקרו כמו שצריך, יש לוודא כי שני המחשבים עליהם רצים השרת והלקוח מחוברים לאינטרנט, ולאותה רשת.

מדריך להרצת המערכת

מדריך למנהל המערכת:

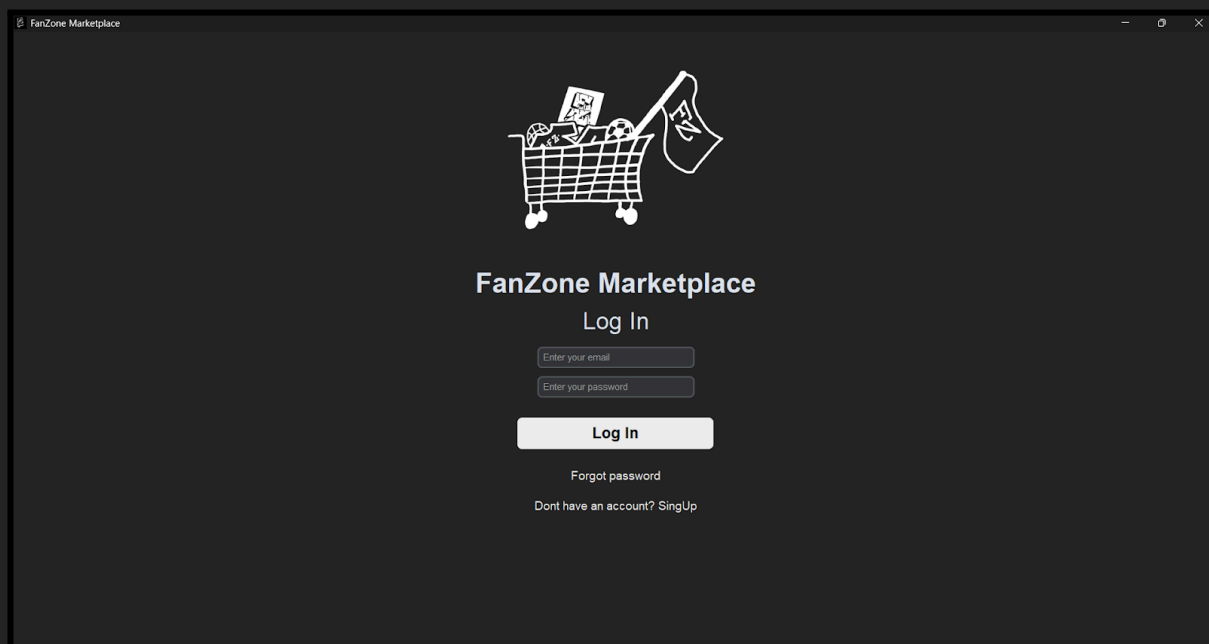
יש לוודא שהתקנת המערכת בוצעה נכונה ותקינה, ואז להריץ את הקובץ `server.py` ממחשב השרת לפני שמריצים את כל משתמשי הקצה, אחרת התוכנה של משתמש הקצה לא תעבוד כי לא תצליח להתחבר לשום שרת.

מדריך למשתמש קצה:

יש לוודא שהתקנת המערכת בוצעה נכונה ותקינה, ואז להריץ את הקובץ `main.py`.

חלק #1, כניסה לחשבון

בעת ההרצה, יפתח המסך הבא:



במסך זה תצטרך להזין את כתובת המייל של החשבון שלך ואת הסיסמא ולאחר מכן ללחוץ על הכפתור **Log In**, בכדי להכנס לחשבון שלך ולעבור למסך הבא.

שכחת את הסיסמא? תוכל ללחוץ על הכפתור **Forgot password** בכדי להכנס לחשבון בלי



סיסמא.

אין לך חשבון? תוכל ללחוץ על הכפתור **Dont have an account? Sign up** בכדי ליצור משתמש חדש.

שכחת את הסיסמא:

במידה ולחצת על **Forgot password** יופיע המסך הבא:

במסך זה תצטרך להזין את כתובת המייל שלך בכדי שישלח אליך קוד ותוכל להכנס לחשבון ללא סיסמא, ולאחר מכן ללחוץ על הכפתור **Send Code**, בכדי לעבור למסך הבא. בכדי לחזור למסך הראשי עליך ללחוץ על **Back To Login**.

אין לך חשבון:

במידה ולחצת על **Dont have an account? Sign up** יופיע המסך הבא:

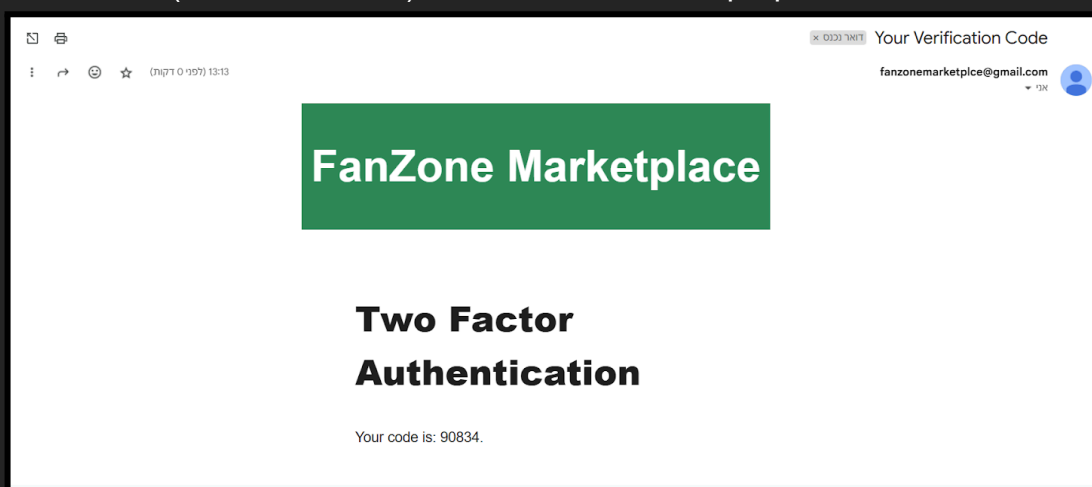
במסך זה תיצור חשבון חדש ולכך תצטרך להזין את כתובת המייל שלך, את שם המשתמש שאתה



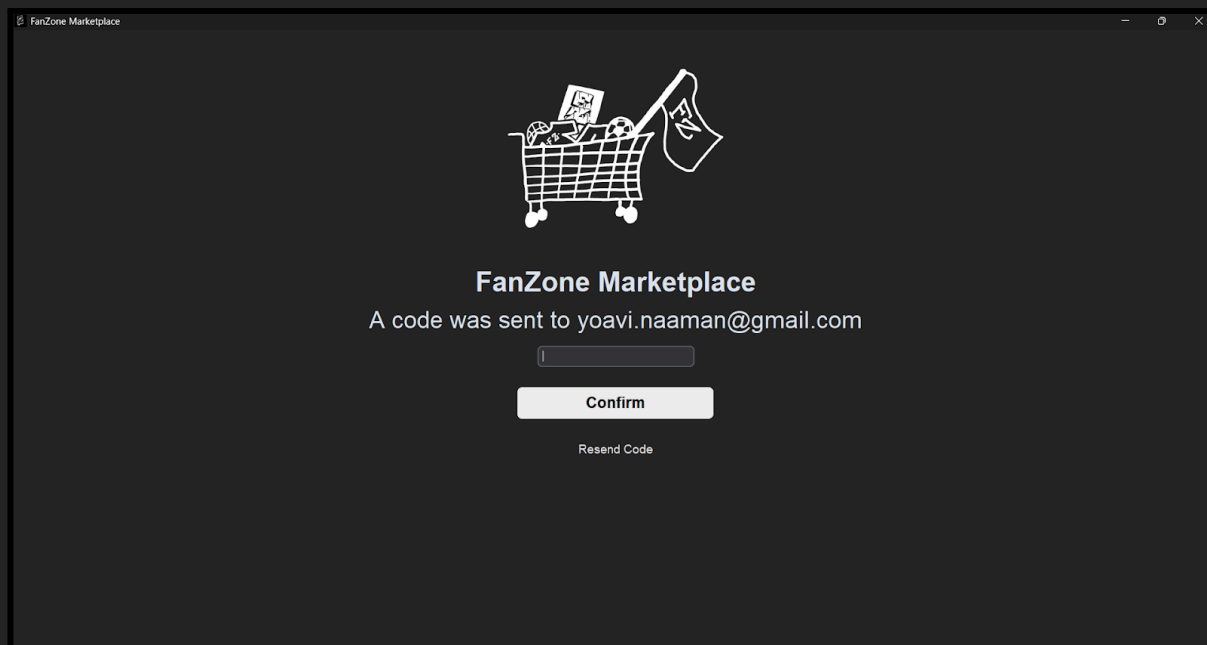
רוצה, את מספר הטלפון שלך ואת הסיסמא של החשבון (פעמיים לצורך אימות), ולאחר מכן ללחוץ על הכפתור **Sign Up**, בכדי לעבור למסך הבא.
בכדי לחזור למסך הראשי עליך ללחוץ על **Back To Login**.

חלק #2, אימות דו שלבי

לאחר שנכנסת לחשבונך (כלומר לחצת כל כל כפתור שציינתי בחלק #1 שיעביר אותך למסך הבא) ישלח לכתובת המייל שסיפקת קוד רנדומלי בעל 5 ספרות (דוגמא למייל שכזה):



ויופיע לך המסך הבא:

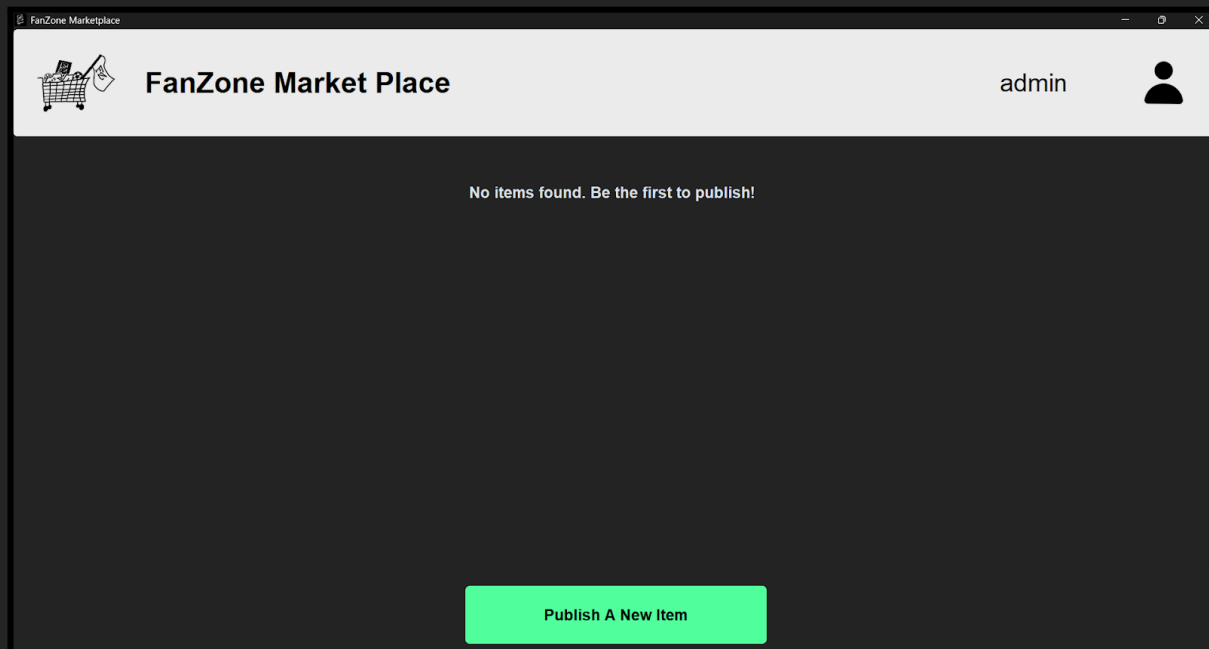


במסך זה תצטרך להזין את חמש הספרות שקיבלת במייל וללחוץ **Confirm**, בכדי לעבור למסך הראשי ולהשלים את הכניסה למשתמש.
הקוד לא הגיע אליך? דבר ראשון תבדוק את תיקיית הספאם במייל שלך ותראה אם במקרה המייל הגיע לשם. אם לא, לחץ על הכפתור **Resend Code** וישלח לך קוד חדש למייל. אם עדיין לא הגיע, וודא כי כתובת המייל המוצגת מעל תיבת הקלט נכונה ובמידה והיא לא, **התחל את התהליך ממהתחלה**.

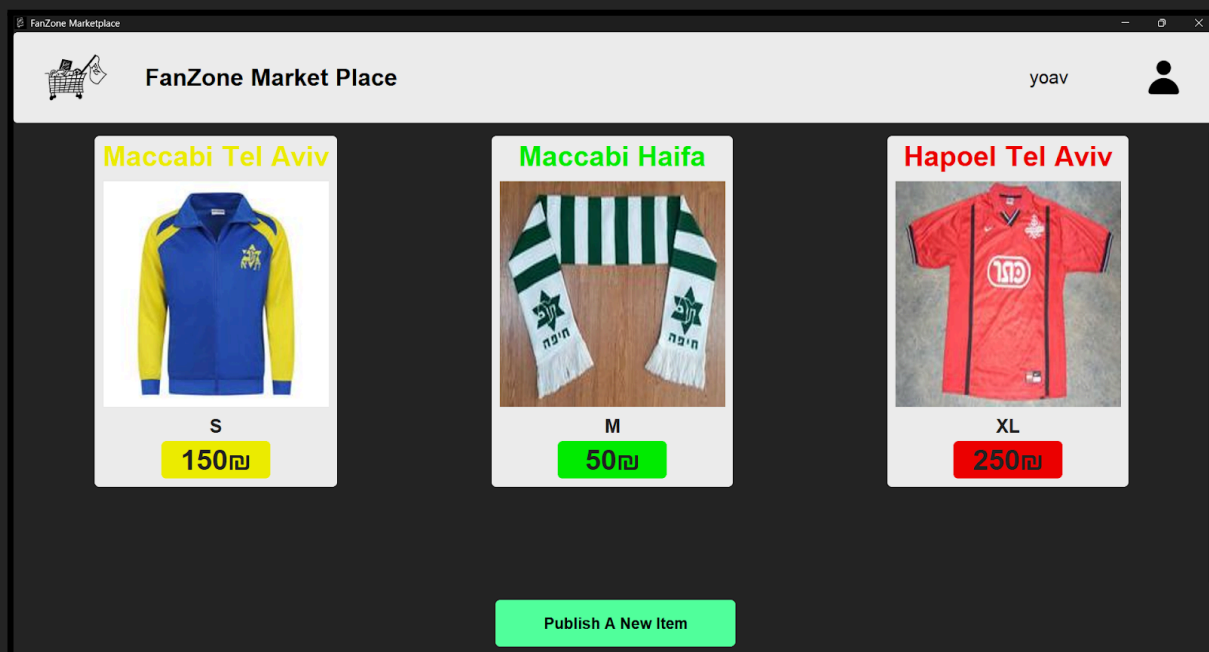


חלק #2, מסך ראשי

לאחר שעברת את מסך האימות בהצלחה יפתח לך המסך הבא:



****הבהרה**:** מסך הבית יראה ככה ^ (ללא מוצרים) אם לא פורסמו מוצרים על ידי משתמשים אחרים. מסך בית כאשר פורסמו מוצרים יראה ככה:





לאחר שנפתח המסך הזה תוכל לבצע כמה פעולות:

בכדי לראות את המוצרים אשר אתה פרסמת: יש ללחוץ על **שם המשתמש שלך** (בצד ימין

למעלה) או על **התמונת פרופיל** ליד. לאחר שלחצת יפתח המסך הבא:

admin

Hapoel Tel Aviv

XL
₪250
SOLD

Maccabi Haifa

admin

Hapoel Tel Aviv

XL
₪250
ON SALE

Maccabi Haifa

במסך זה יוצגו כל המוצרים שאי פעם פרסמת, ובכל אחד מהם יהיה כתוב **ON SALE** (במידה והמוצר עדיין זמין לקנייה) או **SOLD** (אם המוצר נקנה).



בכדי לפרסם מוצר: יש ללחוץ על הכפתור **Publish A New Item** ואז ייפתח המסך הבא:

No image selected

Select Product Type

Select Size

Select Team

Enter The Description/Name Of The

The Year The Product Was Release

Price (In shekels)

City For Pickup

Load Image

Publish Item

במסך זה תצטרך להזין את כל הפרטים של המוצר ולהעלות תמונה שלו, לדוגמא כך:

Match-Worn Shirt

XL

Hapoel Tel Aviv

Double Season 2000 Matchworn

2000

250

Tel Aviv

Load Image

Publish Item

ולאחר מכן ללחוץ על הכפתור **Publish Item**. לאחר שלחצתם תופיע לכמה שניות ההודעה: "**Item published successfully**" (במידה והמוצר התפרסם כמו שצריך, אם לא תופיע הודעת שיגאה בהתאם. **ככל הנראה מדובר תקלה בחיבור לרשת בצד שלך או של השרת**), והחלון ייסגר אוטומטית (כלומר תחזור למסך הבית).



בכדי לעשות Refresh (כלומר לבקש מהשרת מחדש את כל המוצרים הזמינים והצגתם מחדש במסך): יש ללחוץ על הטקסט **FanZone Market Place** אשר נמצא בצד שמאל למעלה או על הלוגו שלידו. לאחר הלחיצה החלון יבצע Refresh והפריטים יוצגו מחדש.

בכדי להרחיב מוצר מסויים (כלומר לראות עוד פרטים עליו): יש ללחוץ על **הכפתור הצבעוני** בו מופיע המחיר שלו (לדוגמא כדי להרחיב את המוצר של מכבי חיפה, יש ללחוץ על **הכפתור הירוק** עליו רשום **₪50**) ואז יפתח המסך הבא:

Double Season 2000 Matchworn

Team: Hapoel Tel Aviv

Size: XL

Year produced: 2000

Location of pickup: Tel Aviv

Price: 250₪**BUY**

במסך זה מוצגים כל הפרטים של המוצר ואם ברצונך לקנות את המוצר תלחץ על הכפתור **BUY**. בלחיצה על כפתור זה יישלח לשרת בקשה לקניית הפריט ובמידה והפריט ייקנה בהצלחה תוצג לך שניות ההודעה: **"Item Bought Successfully!"**, החלון ייסגר אוטומטית (כלומר תחזור למסך הבית), ויישלחו מיילים אליך ואל המוכר:



אליך:

FanZone Marketplace

Item Bought Successfully!

Your purchase of the item: Double Season 2000 Matchworn was successful!

Seller's contact information:

Email address: yoad.naaman@gmail.com

Phone number: 0543409321

We suggest you contact the seller as soon as possible for arranging the pickup.

Transaction information:

Item: Double Season 2000 Matchworn

Price: 250

Buyer: yoad

Seller: admin

Time And Date Of Transaction: 02-06-2025 13:33

Transaction ID: 541792

Item Bought Successfully!

fanzonemarketplace@gmail.com

עזרה | תנאים | פרטיות

אל הקונה:

FanZone Marketplace

Double Season 2000 Matchworn was bought!

Your item: Double Season 2000 Matchworn was bought by yoad!

Buyer's contact information:

Email address: yoyoking2110@gmail.com

Phone number: 0543409321

The buyer has received your contact information as well and will contact you soon.

250 ILS has been transferred to your bank account.

Transaction information:

Item: Double Season 2000 Matchworn

Price: 250

Buyer: yoad

Seller: admin

Time And Date Of Transaction: 02-06-2025 13:33

Transaction ID: 541792

Double Season 2000 Matchworn was bought!

fanzonemarketplace@gmail.com

עזרה | תנאים | פרטיות

(במידה וקרתה תקלה בקניית המוצר, תוצג הודעת שגיאה בהתאם. ככל הנראה מדובר תקלה בחיבור לרשת בצד שלך או של השרת)



רפלקציה

במהלך כל שלושת שנות הלימוד שלי בתיכון במגמת הסייבר, ידעתי שבסופו של דבר הכל מתנקז לפרוייקט בכיתה יב' ובעצם ציפיתי מאוד לפרוייקט הזה, גם אם לא באמת ידעתי מה יהיה בו. לאחר שסיימתי את חמש היחידות של מדעי המחשב ביא' היה לי רגע פתאום לנסות להבין מה זה הפרוייקט הזה שאני צריך להגיש בשנה הבאה ולחשוב מה אני ארצה לעשות. בהתחלה עלה לי הרעיון של הגנת סייבר בנושא באופן כללי כי בסופו של דבר רציתי להתנסות בשם המגמה - סייבר. אך מהר מאוד הבנתי שגם לא עולים לי רעיונות בנושא ושגם ניתן לעשות משהו אחר שישלב את הרעיון של הגנת הסייבר בתוכו. עבר זמן, וכבר התחיל יב' ולא עלה לי שום רעיון, עד שיום אחד קניתי ממישהו דרך קבוצה בווצאפ חולצת כדורגל של המועדון שאני אוהד, וחשבתי לעצמי כמה זה מסורבל לקנות ולמכור דרך קבוצה בווצאפ, שזוהי פלטפורמה שלא מיועדת לזה בכלל. ואז עלה לי הרעיון של FanZone Marketplace, גם אפליקציה שאין כמוה (לפחות בארץ) וגם פרוייקט שישלב בתוכו הרבה נושאים של תכנות תקשורת וסייבר. כשניגשתי להתחיל את הפרוייקט, אני יכול להודות שלא היה לי מושג מאיפה להתחיל. רשמתי לעצמי את כל מה שאני צריך שיהיה לי בצד של הלקוח ושל השרת, אך לקח לי זמן לחשוב מה אני חושב שהכי צריך לבנות קודם. לבסוף הבנתי שאני מבזבז על המחשבה הזאת יותר מדי זמן והתחלתי פשוט עם הממשק הגרפי של הלקוח - כי הבנתי שבכל מקרה אבנה אותו תחילה מבלי לשלב תקשורת כדי שאוכל לשנות דברים ולבנות דברים חדשים יותר בקלות וביעילות. כאשר סיימתי עם מסך ההרשמה הבנתי שהמסך הראשי מחייב שיהיו לי תקשורת ובסיס נתונים בנויים ולכן התחלתי לעבוד על תקשורת הלקוח והשרת. החלק הזה לקח לי הרבה זמן מכיוון שגם הייתי צריך לחשוב איך אני רוצה שמידע יעבור וינותח, גם הייתי צריך פיזית להכניס את כל האלמנטים התקשורתיים לקוד הקיים של הממשק שהיה אז מאוד ארוך, וגם לא הייתה לי ממש דרך לבדוק את הכל עד שהתקשורת הייתה ללא כל תקלות והיה אפשר לחבר בין הלקוח לשרת, אז כל דבר שעשיתי הייתי חייב לעשות בצורה מושלמת שלא תפגע בקוד הקיים או שלא תהיה פגומה בעצמה. לבסוף ולאחר תקופה מאוד מתישה וקשה מבחינת הפרוייקט סיימתי את החלק הזה והרגשתי הקלה מסוימת, רק שמהר מאוד הבנתי שזה היה רק בקושי שלישי ממה שהייתי צריך בשביל פרוייקט שלם ושסך הכל סיימתי רק התחברות לחשבון. אז שוב חזרתי לעבוד רק שהפעם בגלל שהיה כבר הבסיס לתקשורת ולממשק, היה יותר קל לכתוב את כל החלק של המסך הראשי ומן הסתם היו תקלות ורגעי משבר אך לבסוף הצלחתי לסיים את החלון הראשי יותר ויותר משסיימתי את חלון ההרשמה. כנראה שפשוט אחרי זמן רב שעבדתי על הפרוייקט נהייתי יותר ויותר יעיל ומדויק, או שפשוט נהייתה לי יותר מוטיבציה, בכל מקרה הרגע בו סיימתי לגמרי את הפרוייקט היה הרגע שבו הבנתי ונזכרתי למה באתי למגמה הזאת מלכתחילה - כדי לרכוש ידע שחשוב מאוד בעידן הנוכחי ויותר חשוב, לדעת איך להשתמש בכל הידע הזה ולתעל אותו לטובת יצירה שאני חשבתי ועבדתי עליה. אני חושב שבמהלך הפרוייקט למדתי הרבה דברים על עצמי:

על יכולת ההתמדה שלי, יכולת הלמידה העצמית שלי, היצירתיות שלי והאופן שבו אני מתנהל סביב פרוייקט גדול (מה אני צריך לשפר/מה לשמר)

וגם למדתי דברים על איך לעבוד בפרוייקט שכזה ובפרוייקט גדול בכללי: איך לפתור בעיות, לדעת לבקש עזרה ממומחים או הורים בעת הצורך ואפילו סתם חברים כשלא עולה רעיון לגבי משהו עיצובי/רעיוני, איך לעבוד בצורה מסודרת ולמנוע בעיות תוך כדי העבודה ובכללי איך מתנהלים לבד בפרוייקט שדורש כל כך הרבה.

בראייה לאחור לא הייתי משנה כלום. אני יכול עכשיו כשסיימתי להגיד אם הייתי עושה ככה אז הייתי פותר את הדבר הזה יותר מוקדם או שאם הייתי מוסיף משהו אז הפרוייקט היה יותר טוב, אבל אני כרגע נמצא במצב שיש לי פרוייקט גמור בזמן שאני גאה בו ובסופו של דבר כל דבר שעשיתי, כל תקלה שקרתה וכל משבר שהיה ועבר הוביל אותי לאיפה שאני היום, ואם הייתי משנה או מוסיף



אפילו דבר אחד, אולי הייתי נמצא עכשיו במצב אחר. אם היו לי משאבים נוספים וזמן נוסף הייתי מוסיף בראש ובראשונה פונקציונאליות לרכישה אמיתית עם כרטיס אשראי אמיתי כאשר האפליקציה (כלומר אני) תחזיק בכסף אצלה עד ש"תושלם" הרכישה ובעצם המוצר יעבור מהמוכר לקונה ורק אז הכסף יעבור לקונה בכדי למנוע הונאות, ובנוסף הייתי מוסיף שיטת דירוג למוכרים כך שלפני שאנשים יקנו הם יוכלו לראות כמה אמין המוכר, כמה איכותיים המוצרים שלו וכו'. כנראה שגם בזמן הזה היו עולים לי עוד אלף ואחד רעיונות אבל אלה דברים שאני יודע וחשבתי עליהם שהייתי מוסיף אם היה לי עוד משאבים. אני מעוניין להודות לכל צוות המורים של המגמה בתיכון אלון שליוו אותי ותמכו בי כשנתקלתי בבעיות או כשהייתי צריך הכוונה בכלליות, לחברי לכיתה שנתנו לי עצות והיו מוכנים תמיד לעזור לי, ולהודות להורים שלי שניסו לעזור לי כמה שיותר בין אם זה לקשר אותי לאנשים שידעו איך לעזור לי או סתם להביא לי מים כשאני עבדתי על הקוד על כל היום.

ביבליוגרפיה

רוב הנושאים ששילבתי בפרוייקט בין אם זה תקשורת, Tkinter, Threading, והצפנה, למדתי לפני בשיעורים שהיו לנו בתחילת שנה ולכן לא הייתי צריך לחקור עליהם יותר מדי. בנוסף, להרבה מהנושאים שלא ידעתי או לא זכרתי, נעזרתי במודל הבינה המלאכותית Claude שידע להסביר לי בצורה ברורה ומפורטת (ולרוב בצורה הרבה יותר טובה מכל מקום אחר) לגבי כל נושא שנתתי לו והוא גם ידע להסביר לי בצורה מקושרת לפרוייקט שלי ולמה שניסיתי להשיג עם אותו נושא. אז לכן אין בביבליוגרפיה הרבה קישורים מהסיבה הפשוטה שרוב הנושאים לא הצריכו ממני לחפש באתרים בגוגל.

Imgur API:

<https://api.imgur.com/oauth2/addclient>

Datetime module:

https://www.w3schools.com/python/python_datetime.asp

Hashing:

<https://stackoverflow.com/questions/9594125/salt-and-hash-a-password-in-python>

Claude AI:

<https://claude.ai>



קוד הפרויקט

main.py

```
import threading

import time

import logging

import customtkinter as ctk

from client_communication import Communication

from ctk_ui import App


logging.basicConfig(level=logging.INFO, format='%(asctime)s -
%(levelname)s - %(message)s')


def communication_thread():

    global client_com

    global app

    connection_retry_count = 0

    max_connection_retries = 5

    while True:

        try:

            client_com.run()

            if client_com.in_msg != '':

                logging.info(f"Processing incoming message:
{client_com.in_msg}")

                app.handle_in_msg(client_com.in_msg)
```



```
        client_com.in_msg = ''

        if hasattr(app, 'u'):

            if hasattr(app.u, 'out_message') and
app.u.out_message != '':

                logging.info(f"Sending UI message:
{app.u.out_message}")

                client_com.out_msg = app.u.out_message

                app.u.out_message = ''

            if hasattr(app.u, 'm') and app.u.m is not None:

                if hasattr(app.u.m, 'out_message') and
app.u.m.out_message != '':

                    logging.info(f"Sending main window
message: {app.u.m.out_message}")

                    client_com.out_msg =
app.u.m.out_message

                    app.u.m.out_message = ''

        connection_retry_count = 0

        time.sleep(0.1)

    except ConnectionError as e:

        logging.error(f"Connection error: {e}")

        connection_retry_count += 1

        if connection_retry_count >=
max_connection_retries:

            logging.critical("Maximum connection retries
reached")
```



```
        show_error_popup("Connection Error",
                           "Cannot connect to the server.
Please check your network connection.")

        time.sleep(5)

        connection_retry_count = 0

    else:

        time.sleep(1)

except Exception as e:

    logging.error(f"Error in communication thread:
{e}")

    time.sleep(1)

def show_error_popup(title, message):

    try:

        root.after(0, lambda: create_error_popup(title,
message))

    except Exception as e:

        logging.error(f"Error showing popup: {e}")

def create_error_popup(title, message):

    popup = ctk.CTkToplevel(root)

    popup.title(title)

    popup.geometry("400x200")

    popup.grab_set()

    # Center the popup on the screen
```



```
popup.update_idletasks()

width = popup.winfo_width()

height = popup.winfo_height()

x = (popup.winfo_screenwidth() // 2) - (width // 2)

y = (popup.winfo_screenheight() // 2) - (height // 2)

popup.geometry(f"{x}+{y}")


label = ctk.CTkLabel(popup, text=message, font=("Arial",
14))

label.pack(pady=20, padx=20, expand=True)


ok_button = ctk.CTkButton(popup, text="OK",
command=popup.destroy)

ok_button.pack(pady=20)


ctk.set_appearance_mode("system") # Options: "dark", "light",
"system"

ctk.set_default_color_theme("blue")


# Initialize application

try:

    logging.info("Starting FanZone Marketplace client")


    client_com = Communication()


    root = ctk.CTk()

    app = App(root)
```



```
if hasattr(client_com, 'connection_active') and
client_com.connection_active:

    comm_thread =
threading.Thread(target=communication_thread, daemon=True)

    comm_thread.start()

    logging.info("Communication thread started")

else:

    logging.warning("Starting without server connection")

    show_error_popup("Connection Warning",

                    "Could not connect to server. Some
features may be unavailable.")

    logging.info("Sample credentials: yoav@gmail.com / 12345")

    root.mainloop()

    client_com.close_com()

except Exception as e:

    logging.critical(f"Fatal error starting application: {e}")

    try:

        import tkinter as tk

        from tkinter import messagebox

        tk.messagebox.showerror("Error", f"Failed to start
application: {e}")

    except:

        print(f"CRITICAL ERROR: {e}")
```



ctk_ui.py

```
from user_introduction import UserIntro

class App():

    def __init__(self, root):

        root.title("FanZone Marketplace")

        root.iconbitmap(r"FZ.ico")

        root.after(1, lambda: root.state('zoomed'))

        defaultbg = root.cget('bg')

        opposite_color = ""

        white_or_black = ""

        if defaultbg == "gray14":

            opposite_color = "gray92"

            white_or_black = "black"

        else:

            opposite_color = "gray14"

            white_or_black = "white"

        self.u = UserIntro(root, defaultbg, opposite_color,
white_or_black)

    def handle_in_msg(self, msg):

        if msg and len(msg.split('#')) > 0 and
msg.split('#')[0] == 'USER_INTRO':

            self.u.in_message = msg

        elif (msg and len(msg.split('#')) > 0 and
msg.split('#')[0] == 'MAIN'):

            self.u.m.in_message = msg
```



client_communication.py

```
import socket

import threading

import time

import logging

import os

from encryption_client import Client_Encryption

logging.basicConfig(level=logging.INFO, format='%(asctime)s -
%(levelname)s - %(message)s')

class Communication:

    def __init__(self):

        self.MAX_MSG_LENGTH = 10485760

        SERVER_PORT = 5678

        SERVER_IP = '127.0.0.1'

        self.my_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)

        self.my_socket.settimeout(10)

        self.iv = None

        self.en = Client_Encryption()

        try:

            self.my_socket.connect((SERVER_IP, SERVER_PORT))

self.en.generate_secret_key(self.my_socket.recv(1024).decode()
)

self.my_socket.send(str(self.en.public_parameter).encode())

            logging.info("Connected to server successfully")
```



```
        self.connection_active = True

    except (socket.timeout, ConnectionRefusedError) as e:
        logging.error(f"Failed to connect to server: {e}")
        self.connection_active = False
        self.my_socket = None

    self.in_msg = ''
    self.out_msg = ''
    self.l = threading.Lock()

def run(self):
    if not self.connection_active:
        try:
            if self.my_socket:
                self.my_socket.close()

            self.my_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)

            self.my_socket.settimeout(10)
            self.my_socket.connect(('127.0.0.1', 5678))
            self.connection_active = True

self.en.generate_secret_key(self.my_socket.recv(1024).decode()
)

self.my_socket.send(str(self.en.public_parameter).encode())
        logging.info("Reconnected to server")
    except (socket.timeout, ConnectionRefusedError) as
e:
        logging.error(f"Failed to reconnect: {e}")
        time.sleep(2)
```



```
        return

    if not self.out_msg:
        return

    with self.l:
        if self.out_msg != '':
            msg_length = len(self.out_msg)

            logging.info(f"Sending message of length
{msg_length}: {self.out_msg[:100]}...")

            try:
                self.my_socket.settimeout(15)

                self.iv = os.urandom(16)

                msg_bytes = self.en.encrypt(self.out_msg,
self.iv)

                self.my_socket.send(self.iv)

                self.my_socket.send(msg_bytes)

                logging.info("Message sent successfully")

                self.iv = self.my_socket.recv(16)

                response_bytes =
self.my_socket.recv(self.MAX_MSG_LENGTH)

                if response_bytes:
                    self.in_msg =
self.en.decrypt(response_bytes, self.iv)

                    in_length = len(self.in_msg)

                    logging.info(f"Received response of
length {in_length}: {self.in_msg[:100]}...")

            else:
```



```
        logging.warning("Server closed
connection - empty response received")

        self.connection_active = False

        return

    self.out_msg = ''

except socket.timeout:

    logging.error("Socket timeout - server not
responding")

    self.connection_active = False

    self.out_msg = ''

except ConnectionResetError as e:

    logging.error(f"Connection reset by server:
{e}")

    self.connection_active = False

    self.out_msg = ''

except Exception as e:

    logging.error(f"Communication error: {e}")

    self.connection_active = False

    self.out_msg = ''

def close_com(self):

    try:

        if self.my_socket:

            self.my_socket.close()

            logging.info("Connection closed")

    except Exception as e:

        logging.error(f"Error closing socket: {e}")
```



encryption_client.py

```
import random

import logging

from hashlib import sha256

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms,
modes

import os

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s
- %(message)s')

class Client_Encryption:

    def __init__(self):

        self.p = 23

        self.g = 5

        self.private_parameter = random.randint(1, self.p-1)

        self.public_parameter = pow(self.g, self.private_parameter, self.p)

        self.secret = None

        self.key = None

    def generate_secret_key(self, client_parameter):

        self.secret = pow(int(client_parameter), self.private_parameter,
self.p)

        self.key = sha256(str(self.secret).encode()).digest()[:16]

    def encrypt(self, message, iv):

        if isinstance(message, str):

            message = message.encode('utf-8')

        from cryptography.hazmat.primitives import padding
```



```
    padder = padding.PKCS7(128).padder()
    padded_data = padder.update(message)
    padded_data += padder.finalize()

    cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv))
    encryptor = cipher.encryptor()

    return encryptor.update(padded_data) + encryptor.finalize()

def decrypt(self, message, iv):
    cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv))
    decryptor = cipher.decryptor()

    padded_data = decryptor.update(message) + decryptor.finalize()

    from cryptography.hazmat.primitives import padding
    unpadder = padding.PKCS7(128).unpadder()

    data = unpadder.update(padded_data)
    data += unpadder.finalize()

    return data.decode('utf-8')
```



user_introduction.py

```
import customtkinter as ctk

from PIL import Image

import time


class UserIntro():

    def __init__(self, root, defaultbg, opposite_color, white_or_black):

        self.root = root

        self.defaultbg = defaultbg

        self.opposite_color = opposite_color

        self.white_or_black = white_or_black

        self.forgot_password = False

        self.signup = False

        self.email_error = False

        self.phone_error = False

        self.code_error = False

        self.code_sent = False

        self.user = ""

        self.password = ""

        self.secret_code = None

        self.e_or_p = False

        self.doesnt_match = False

        self.usr_exists = False

        self.pass_dont_match_label = ctk.CTkLabel(self.root, text="password
doesn't match",

                                                    font=("Arial", 15,
"normal"), text_color="red")

        self.email_not_valid_label = ctk.CTkLabel(self.root, text="Incorrect
Email Adress",

                                                    font=("Arial", 15,
"normal"), text_color="red")
```



```
self.phone_not_valid_label = ctk.CTkLabel(self.root, text="Invalid
phone number",

font=("Arial", 15,
"normal"), text_color="red")

self.e_or_p_incorrect = ctk.CTkLabel(self.root, text="Email or
Password are incorrect",

font=("Arial", 15, "normal"),
text_color="red")

self.user_exists = ctk.CTkLabel(self.root, text="User already exists
for this email",

font=("Arial", 15, "normal"),
text_color="red")

self.m = None

self.in_message = ''

self.out_message = ''

self.email = ''

self.image_label = None

self.LogIn_label = None

self.Fanzone_label = None

self.email_entry = None

self.password_entry = None

self.login_btn = None

self.forgot_btn = None

self.signup_btn = None

self.forgot_label = None

self.enter_email = None

self.forgot_email = None

self.send_code = None

self.back_btn = None

self.signUp_label = None

self.su_email_entry = None

self.su_phone_entry = None

self.su_pass_entry = None
```



```
self.su_username_entry = None

self.re_password_entry = None

self.code_label = None

self.code_entry = None

self.confirm_btn = None

self.resend_btn = None

self.login()

def Send_Message(self, msg, msg_type):

    self.out_message = f"USER_INTRO#{msg_type}#{msg}"

    print(self.out_message)

def login(self):

    image = ctk.CTkImage(light_image=Image.open(r"FanZone no bg.png"),
                        dark_image=Image.open(r"FanZone no bg
white.png"),

                        size=(300, 300))

    self.image_label = ctk.CTkLabel(self.root, image=image, text="")

    self.image_label.pack(pady=0)

    self.Fanzone_label = ctk.CTkLabel(self.root, text="FanZone
Marketplace", font=("Arial", 35, "bold"))

    self.Fanzone_label.pack(pady=0)

    self.LogIn_label = ctk.CTkLabel(self.root, text="Log In",
font=("Arial", 30, "normal"))

    self.LogIn_label.pack(pady=10)

    self.email_entry = ctk.CTkEntry(self.root, font=("Arial", 12),
width=200, placeholder_text="Enter your email")

    self.email_entry.pack(pady=5)

    self.password_entry = ctk.CTkEntry(self.root, font=("Arial", 12),
width=200,

                                placeholder_text="Enter your
password",

                                show="*")
```



```
self.password_entry.pack(pady=5)

self.login_btn = ctk.CTkButton(self.root, text="Log In",
font=("Arial", 20, "bold"),
                                fg_color=self.opposite_color,
                                hover_color="SeaGreen1", height=40,
width=250, text_color=self.white_or_black,
                                command=self.log_pressed)

self.login_btn.pack(pady=20)

self.forgot_btn = ctk.CTkButton(self.root, text="Forgot password",
font=("Arial", 15), fg_color=self.defaultbg,
                                hover_color=self.white_or_black,
text_color=self.opposite_color,
                                command=self.forgot_pass)

self.forgot_btn.pack(pady=0)

self.signup_btn = ctk.CTkButton(self.root, text="Dont have an
account? SingUp", font=("Arial", 15),
                                fg_color=self.defaultbg,
                                hover_color=self.white_or_black,
text_color=self.opposite_color,
                                command=self.sign_up)

self.signup_btn.pack(pady=10)

def forgot_pass(self):

    self.LogIn_label.pack_forget()

    self.email_entry.pack_forget()

    self.password_entry.pack_forget()

    self.login_btn.pack_forget()

    self.forgot_btn.pack_forget()

    self.signup_btn.pack_forget()

    if self.e_or_p:

        self.e_or_p_incorrect.pack_forget()

    self.forgot()
```



```
def forgot(self):

    self.forgot_password = True

    self.forgot_label = ctk.CTkLabel(self.root, text="Forgot Password",
font=("Arial", 30, "normal"))

    self.forgot_label.pack(pady=10)

    self.enter_email = ctk.CTkLabel(self.root, text="Choose an email to
which a verification code will be sent:",

                                font=("Arial", 20, "normal"))

    self.enter_email.pack(pady=20)

    self.forgot_email = ctk.CTkEntry(self.root, font=("Arial", 12),
height=40, width=300,

                                placeholder_text="Enter your
email")

    self.forgot_email.pack(pady=10)

    self.send_code = ctk.CTkButton(self.root, text="Send Code",
font=("Arial", 20, "bold"),

                                fg_color=self.opposite_color,

                                hover_color="SeaGreen1", height=40,
width=250, text_color=self.white_or_black,

                                command=self.code_send)

    self.send_code.pack(pady=20)

    self.back_btn = ctk.CTkButton(self.root, text="Back To LogIn",
font=("Arial", 15, "normal"),

                                fg_color=self.defaultbg,

                                hover_color=self.white_or_black,
text_color=self.opposite_color,

                                command=self.get_back)

    self.back_btn.pack(pady=5)

def sign_up(self):

    self.LogIn_label.pack_forget()

    self.email_entry.pack_forget()

    self.password_entry.pack_forget()
```



```
self.login_btn.pack_forget()

self.forgot_btn.pack_forget()

self.signup_btn.pack_forget()

if self.e_or_p:
    self.e_or_p_incorrect.pack_forget()

self.sign()

def sign(self):

    self.signup = True

    doesnt_match = False

    not_enough_letters = False

    too_much_letters = False

    no_capital = False

    no_special = False

    self.signUp_label = ctk.CTkLabel(self.root, text="Sign Up",
font=("Arial", 30, "normal"))

    self.signUp_label.pack(pady=10)

    self.su_email_entry = ctk.CTkEntry(self.root, font=("Arial", 12),
width=200,

placeholder_text="Enter your
email")

    self.su_email_entry.pack(pady=5)

    self.su_username_entry = ctk.CTkEntry(self.root, font=("Arial", 12),
width=200,

placeholder_text="Enter your
username")

    self.su_username_entry.pack(pady=5)

    self.su_phone_entry = ctk.CTkEntry(self.root, font=("Arial", 12),
width=200,

placeholder_text="Enter your
phone number")

    self.su_phone_entry.pack(pady=5)

    self.su_pass_entry = ctk.CTkEntry(self.root, font=("Arial", 12),
width=200,
```



```
placeholder_text="Enter your
password")

self.su_pass_entry.pack(pady=5)

self.re_password_entry = ctk.CTkEntry(self.root, font=("Arial", 12),
width=200,

placeholder_text="Re-Enter
your password",

show="*")

self.re_password_entry.pack(pady=5)

self.signup_btn = ctk.CTkButton(self.root, text="Sign Up",
font=("Arial", 20, "bold"),

fg_color=self.opposite_color,

hover_color="SeaGreen1", height=40,
width=250, text_color=self.white_or_black,

command=self.Signedup)

self.signup_btn.pack(pady=20)

self.back_btn = ctk.CTkButton(self.root, text="Back To LogIn",
font=("Arial", 15, "normal"),

fg_color=self.defaultbg,

hover_color=self.white_or_black,
text_color=self.opposite_color,

command=self.get_back)

self.back_btn.pack(pady=5)

def Signedup(self):

import bcrypt

self.signup_btn.pack_forget()

self.back_btn.pack_forget()

if self.su_pass_entry.get() == self.re_password_entry.get():

salt = bcrypt.gensalt()

self.password =
bcrypt.hashpw(self.su_pass_entry.get().encode('utf-8'), salt)

self.Send_Message(
```



```
f'{self.su_email_entry.get()}@{self.password}@{self.su_username_entry.get()}@{self.su_phone_entry.get()}',

        'SIGNUP_REQUEST')

        self.wait_for_signup_response()

    else:

        if self.su_pass_entry.get() != self.re_password_entry.get() and not self.doesnt_match:

            self.pass_dont_match_label = ctk.CTkLabel(self.root,
text="password doesn't match",

                                                    font=("Arial", 15,
"normal"), text_color="red")

            self.pass_dont_match_label.pack(pady=0)

            self.doesnt_match = True

            self.signup_btn.pack(pady=20)

            self.back_btn.pack(pady=5)

def wait_for_signup_response(self):

    if self.in_message != '':

        parts = self.in_message.split('#')

        if len(parts) >= 3 and parts[1] == 'SIGNUP_RESPONSE':

            if parts[2] == 'OK':

                print("Signup successful, sending code")

                self.in_message = ''

                self.code_send()

            elif parts[2] == 'USER_EXISTS':

                print("User already exists")

                self.in_message = ''

                self.user_exists.pack(pady=0)

                self.signup_btn.pack(pady=20)

                self.back_btn.pack(pady=5)

                self.usr_exists = True

            else:
```



```
        print(f"Signup failed: {parts[2]}")

        self.in_message = ''

        self.signup_btn.pack(pady=20)

        self.back_btn.pack(pady=5)

    return

self.root.after(100, self.wait_for_signup_response)

def get_back(self):

    self.image_label.pack_forget()

    self.LogIn_label.pack_forget()

    self.Fanzone_label.pack_forget()

    if self.forgot_password:

        self.forgot_label.pack_forget()

        self.enter_email.pack_forget()

        self.forgot_email.pack_forget()

        self.send_code.pack_forget()

        self.back_btn.pack_forget()

        if self.email_error:

            self.email_not_valid_label.pack_forget()

        self.forgot_password = False

    elif self.signup:

        self.signUp_label.pack_forget()

        self.su_email_entry.pack_forget()

        self.su_username_entry.pack_forget()

        self.su_phone_entry.pack_forget()

        self.su_pass_entry.pack_forget()

        self.re_password_entry.pack_forget()

        self.signup_btn.pack_forget()
```



```
        self.back_btn.pack_forget()

        if self.email_error:

            self.email_not_valid_label.pack_forget()

            self.signup = False

        self.email_error = False

        self.login()

def log_pressed(self):

    import bcrypt

    self.user = self.email_entry.get()

    self.password = self.password_entry.get()

    self.Send_Message(f"{self.user} {self.password}", "LOGIN_REQUEST")

    self.wait_for_login_response()

def wait_for_login_response(self):

    if self.in_message:

        parts = self.in_message.split('#')

        if len(parts) >= 3 and parts[1] == 'LOGIN_RESPONSE':

            if parts[2] == 'LOGIN_SUCCESS':

                print('login success')

                self.in_message = ''

                self.code_send()

                return

            elif parts[2] == 'USER_OR_PASS_INVALID':

                self.in_message = ''

                self.e_or_p = True

                self.login_btn.pack_forget()

                self.forgot_btn.pack_forget()

                self.signup_btn.pack_forget()

                self.e_or_p_incorrect.pack(pady=0)
```



```
        self.login_btn.pack(pady=20)

        self.forgot_btn.pack(pady=0)

        self.signup_btn.pack(pady=10)

        return

    self.root.after(100, self.wait_for_login_response)

def code_send(self):

    import re

    if self.forgot_password:

        self.email = self.forgot_email.get()

        if re.match(r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$',
self.email) is not None:

            self.transition_to_code()

            self.Send_Message(self.email, "CODE_REQUEST")

            self.email_error = False

            self.wait_for_code_response()

        elif not self.email_error:

            self.email_error = True

            self.send_code.pack_forget()

            self.back_btn.pack_forget()

            self.email_not_valid_label.pack(pady=0)

            self.send_code.pack(pady=20)

            self.back_btn.pack(pady=5)

        elif self.signup:

            self.email = self.su_email_entry.get()

            if re.match(r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$',
self.email) is not None:

                if len(self.su_phone_entry.get()) == 10 and
self.su_phone_entry.get().isdigit():
```



```
        self.transition_to_code()

        self.Send_Message(self.email, "CODE_REQUEST")

        self.wait_for_code_response()

    elif not self.phone_error:

        self.phone_error = True

        self.signup_btn.pack_forget()

        self.back_btn.pack_forget()

        if self.email_error:

            self.email_not_valid_label.pack_forget()

            self.phone_not_valid_label.pack(pady=0)

            self.signup_btn.pack(pady=20)

            self.back_btn.pack(pady=5)

    elif not self.email_error:

        self.email_error = True

        self.signup_btn.pack_forget()

        self.back_btn.pack_forget()

        self.email_not_valid_label.pack(pady=0)

        if len(self.su_phone_entry.get()) == 10 and self.su_phone_entry.get().isdigit():

            if self.phone_error:

                self.phone_not_valid_label.pack_forget()

            elif not self.phone_error:

                self.phone_error = True

                self.phone_not_valid_label.pack(pady=0)

            self.signup_btn.pack(pady=20)

            self.back_btn.pack(pady=5)

    else:

        print('logcode')

        self.email = self.email_entry.get()

        self.transition_to_code()
```



```
self.Send_Message(self.email, "CODE_REQUEST")

self.wait_for_code_response()

def wait_for_code_response(self):

    if self.in_message:

        parts = self.in_message.split('#')

        if len(parts) >= 3 and parts[1] == 'CODE_RESPONSE':

            if parts[2] == 'OK':

                self.in_message = ''

                self.code_sent = True

                if not self.email_error:

                    print('trans')

                    return

            elif parts[2] == 'FAIL' and not self.email_error:

                print("code error")

                self.in_message = ''

                self.email_error = True

                if hasattr(self, 'send_code') and self.send_code:

                    self.send_code.pack_forget()

                if hasattr(self, 'back_btn') and self.back_btn:

                    self.back_btn.pack_forget()

                self.email_not_valid_label.pack(pady=0)

                if hasattr(self, 'send_code') and self.send_code:

                    self.send_code.pack(pady=20)

                if hasattr(self, 'back_btn') and self.back_btn:

                    self.back_btn.pack(pady=5)

            return

self.root.after(100, self.wait_for_code_response)
```



```
def transition_to_code(self):  
  
    if self.forgot_password:  
  
        self.user = self.forgot_email.get()  
  
        self.forgot_label.pack_forget()  
  
        self.enter_email.pack_forget()  
  
        self.forgot_email.pack_forget()  
  
        self.send_code.pack_forget()  
  
        self.back_btn.pack_forget()  
  
        if self.email_error:  
  
            self.email_not_valid_label.pack_forget()  
  
        self.forgot_password = False  
  
    elif self.signup:  
  
        self.user = self.su_email_entry.get()  
  
        self.signUp_label.pack_forget()  
  
        self.su_email_entry.pack_forget()  
  
        self.su_username_entry.pack_forget()  
  
        self.su_phone_entry.pack_forget()  
  
        self.su_pass_entry.pack_forget()  
  
        self.re_password_entry.pack_forget()  
  
        self.signup_btn.pack_forget()  
  
        self.back_btn.pack_forget()  
  
        if self.email_error:  
  
            self.email_not_valid_label.pack_forget()  
  
        if self.phone_error:  
  
            self.phone_not_valid_label.pack_forget()  
  
        if self.doesnt_match:  
  
            self.pass_dont_match_label.pack_forget()  
  
        if self.usr_exists:  
  
            self.user_exists.pack_forget()  
  
        self.signup = False
```



```
else:

    print('login2')

    self.LogIn_label.pack_forget()

    self.email_entry.pack_forget()

    self.password_entry.pack_forget()

    self.login_btn.pack_forget()

    self.forgot_btn.pack_forget()

    self.signup_btn.pack_forget()

    if self.e_or_p:

        self.e_or_p_incorrect.pack_forget()

    self.code_screen()

def code_screen(self):

    self.code_label = ctk.CTkLabel(self.root, text=f"A code was sent to {self.user}", font=("Arial", 30, "normal"))

    self.code_label.pack(pady=10)

    self.code_entry = ctk.CTkEntry(self.root, font=("Arial", 12),
width=200,

                                placeholder_text="Enter the code")

    self.code_entry.pack(pady=5)

    self.confirm_btn = ctk.CTkButton(self.root, text="Confirm",
font=("Arial", 20, "bold"),

                                fg_color=self.opposite_color,

                                hover_color="SeaGreen1", height=40,
width=250, text_color=self.white_or_black,

                                command=self.check_code)

    self.confirm_btn.pack(pady=20)

    self.resend_btn = ctk.CTkButton(self.root, text="Resend Code",
font=("Arial", 15, "normal"),

                                fg_color=self.defaultbg,

                                hover_color=self.white_or_black,
text_color=self.opposite_color,

                                command=self.code_resend)
```



```
self.resend_btn.pack(pady=5)

def code_resend(self):

    self.code_label.pack_forget()

    self.code_entry.pack_forget()

    self.confirm_btn.pack_forget()

    self.resend_btn.pack_forget()

    self.Send_Message(self.email, "CODE_REQUEST")

    self.wait_for_code_resend()

def wait_for_code_resend(self):

    if self.in_message:

        parts = self.in_message.split('#')

        if len(parts) >= 3 and parts[1] == 'CODE_RESPONSE' and parts[2]
== 'OK':

            self.in_message = ''

            self.code_sent = True

            self.code_screen()

            return

    self.root.after(100, self.wait_for_code_resend)

def check_code(self):

    self.Send_Message(self.email + ":" + self.code_entry.get(),
"CODE_VALIDATION")

    self.wait_for_code_validation()

def wait_for_code_validation(self):

    if self.in_message:

        parts = self.in_message.split('#')

        if len(parts) >= 3 and parts[1] == 'CODE_VALIDATION':
```



```
if parts[2].split(':')[0] == 'OK':

    print('valid')

    self.secret_code = parts[2].split(':')[1]

    print(self.secret_code)

    self.in_message = ''

    self.image_label.pack_forget()

    self.Fanzone_label.pack_forget()

    self.code_label.pack_forget()

    self.code_entry.pack_forget()

    self.confirm_btn.pack_forget()

    self.resend_btn.pack_forget()

    from main_windows import MainWin

    self.m = MainWin(self.root, self.defaultbg,
self.opposite_color, self.white_or_black,

                        self.secret_code)

    return

else:

    self.in_message = ''

    if not self.code_error:

        self.confirm_btn.pack_forget()

        self.resend_btn.pack_forget()

        code_dont_match_label = ctk.CTkLabel(self.root,
text="code doesn't match",

                                                font=("Arial",
15, "normal"), text_color="red")

        code_dont_match_label.pack(pady=0)

        self.confirm_btn.pack(pady=20)

        self.resend_btn.pack(pady=5)

        self.code_error = True

    return

self.root.after(100, self.wait_for_code_validation)
```



main_windows.py

```
import customtkinter as ctk

import tkinter

from PIL import Image, ImageTk

import time

import random

import json

import base64

import io

import ast

import requests

from io import BytesIO


class MainWin():

    def __init__(self, root, defaultbg, opposite_color, white_or_black,
secret_code):

        self.user = None

        self.secret_code = secret_code

        self.header_frame = None

        self.scrollable_frame = None

        self.scrollable_frame_items = []

        self.profile_scrollable_frame_items = []

        self.frame = None

        self.publish_frame = None

        self.menu_logo = None

        self.menu_text = None

        self.menu_profile_name = None

        self.publish_button = None

        self.pfp_label = None

        self.root = root
```



```
self.pfroot = None

self.publish_team_name_entry = None

self.publish_type_entry = None

self.publish_size_entry = None

self.publish_year_entry = None

self.publish_price_entry = None

self.publish_city_entry = None

self.publish_description_entry = None

self.image_field_frame = None

self.image_field = None

self.image_file_path = None

self.image_pil = None

self.binary_data = None

self.defaultbg = defaultbg

self.opposite_color = opposite_color

self.white_or_black = white_or_black

self.in_message = ''

self.out_message = ''

self.item_width = None

self.row = 0

self.col = 0

self.screen_width = self.root.winfo_screenwidth()

self.screen_height = self.root.winfo_screenheight()

logo_size = int(min(self.screen_width, self.screen_height) * 0.1) #
10% of screen min dimension

pfp_size = int(min(self.screen_width, self.screen_height) * 0.08) #
8% of screen min dimension

self.image = ctk.CTkImage(light_image=Image.open(r"FanZone no bg
white.png"),
```



```
        dark_image=Image.open(r"FanZone no
bg.png"),

        size=(logo_size, logo_size))

    self.pfp = ctk.CTkImage(light_image=Image.open(r"default profile
white.png"),

        dark_image=Image.open(r"default profile
black.png"),

        size=(pfp_size, pfp_size))

    self.product_price_example = None

    self.root.grid_columnconfigure(0, weight=1)

    self.root.grid_rowconfigure(0, weight=0)

    self.root.grid_rowconfigure(1, weight=1)

    self.root.bind("<Configure>", self.on_resize)

    if self.secret_code != 21:

        self.Send_Message(f"USERNAME_REQUEST#{self.secret_code}")

        self.check_for_username()

    else:

        self.user = 'admin'

        self.landing_page()

def check_for_username(self):

    if self.in_message != '':

        print(f"Checking username message: {self.in_message}")

        parts = self.in_message.split('#')

        if parts[0] == 'MAIN' and parts[1] == 'USERNAME' and len(parts)
>= 3:

            self.user = parts[2]

            print(f"Username received: {self.user}")

            self.in_message = ''

            self.landing_page()
```



```
        return

    if not hasattr(self, '_username_check_count'):
        self._username_check_count = 0

    self._username_check_count += 1

    if self._username_check_count > 600:
        print("Timeout waiting for username")
        self.user = "Guest"
        self.landing_page()
    else:
        self.root.after(100, self.check_for_username)

def is_json(self, value):
    if not isinstance(value, str):
        return False

    try:
        # Attempt to parse the string as JSON
        json.loads(value)
        return True
    except json.JSONDecodeError:
        return False

def Send_Message(self, data):
    print(f"Sending message: {data}")

    if self.is_json(data):
        self.out_message =
f"MAIN#PUBLISH_ITEM#{data}#{self.secret_code}"
    else:
```



```
        if data.startswith("DATA_REQUEST") :

            parts = data.split('#')

            if len(parts) == 2:

                self.out_message = f"MAIN#{parts[0]}#{parts[1]}#1"

            else:

                self.out_message = f"MAIN#{data}"

        else:

            self.out_message = f"MAIN#{data}"

    print(f"Formatted output message: {self.out_message}")

def on_resize(self, event):

    if event.widget == self.root:

        self.screen_width = event.width

        self.screen_height = event.height

        self.header_frame.configure(width=self.screen_width)

        self.scrollable_frame.configure(width=self.screen_width)

        self.publish_frame.configure(width=self.screen_width)

def landing_page(self):

    self.scrollable_frame_items = []

    self.header_frame = self.Menu(self.root,
height=int(self.screen_height * 0.12), fg_color=self.opposite_color)

    self.header_frame.grid(row=0, column=0, sticky="ew")

    self.header_frame.grid_columnconfigure(0, weight=0) # Logo (fixed
size)

    self.header_frame.grid_columnconfigure(1, weight=0) # Title (fixed
size)
```



```
        self.header_frame.grid_columnconfigure(2, weight=1) # Flexible
space

        self.header_frame.grid_columnconfigure(3, weight=0) # Profile name
(fixed size)

        self.header_frame.grid_columnconfigure(4, weight=0) # Profile pic
(fixed size)

# Logo

        self.menu_logo = ctk.CTkButton(self.header_frame, image=self.image,
fg_color=self.opposite_color,

        hover_color=self.opposite_color, text="",
command=self.logo_pressed)

        self.menu_logo.grid(row=0, column=0, padx=(10, 0), pady=10)

title_font_size = int(min(self.screen_width, self.screen_height) *
0.035)

        self.menu_text = ctk.CTkButton(

            self.header_frame,

            text="FanZone Market Place",

            font=("Arial", title_font_size, "bold"),

            text_color=self.white_or_black,

            fg_color=self.opposite_color,

            hover_color=self.opposite_color,

            command=self.logo_pressed

        )

        self.menu_text.grid(row=0, column=1, padx=10, pady=10)

profile_name_width = int(self.screen_width * 0.10)

profile_font_size = int(min(self.screen_width, self.screen_height) *
0.030)

        self.menu_profile_name = ctk.CTkButton(

            self.header_frame,

            text=f"{self.user}",
```



```
        font=("Arial", profile_font_size),

        hover_color=self.opposite_color,

        fg_color=self.opposite_color,

        text_color=self.white_or_black,

        width=profile_name_width,

        command = self.profile_pressed

    )

    self.menu_profile_name.grid(row=0, column=3, padx=0, pady=10,
sticky="e")

    self.pfp_label = ctk.CTkButton(

        self.header_frame,

        image=self.pfp,

        text="",

        fg_color=self.opposite_color,

        hover_color=self.opposite_color,

        command=self.profile_pressed

    )

    self.pfp_label.grid(row=0, column=4, padx=0, pady=10, sticky="e")

    self.scrollable_frame = ctk.CTkScrollableFrame(self.root,
fg_color=self.defaultbg)

    self.scrollable_frame.grid(row=1, column=0, padx=(0, 0), pady=(0,
0), sticky="nsew")

    self.scrollable_frame.grid_columnconfigure(0, weight=1)

    self.scrollable_frame.grid_columnconfigure(1, weight=1)

    self.scrollable_frame.grid_columnconfigure(2, weight=1)

    self.load_items()

    self.publish_frame = ctk.CTkFrame(self.root,
fg_color=self.defaultbg)
```



```
self.publish_frame.grid(row=2, column=0, padx=(0, 0), pady=(0, 0),
sticky="nsew")

self.publish_frame.grid_columnconfigure(0, weight=1) # Flexible
left margin

self.publish_frame.grid_columnconfigure(1, weight=0) # Fixed center
column for button

self.publish_frame.grid_columnconfigure(2, weight=1) # Flexible
right margin


self.publish_frame.grid_rowconfigure(0, weight=1) # Top margin

self.publish_frame.grid_rowconfigure(1, weight=0) # Button row

self.publish_frame.grid_rowconfigure(2, weight=1) # Bottom margin


button_width = int(self.screen_width * 0.2)

button_height = int(self.screen_height * 0.07)

self.publish_button = ctk.CTkButton(

    self.publish_frame,

    fg_color="SeaGreen1",

    text_color=self.white_or_black,

    font=("Arial", 20, "bold"),

    text="Publish A New Item",

    width=button_width,

    height=button_height,

    hover_color=self.opposite_color,

    command=self.publish

)

self.publish_button.grid(

    row=1,

    column=1,

    padx=(20, 20),

    pady=(10, 10)
```



```
)

def load_items(self):

    loading_label = ctk.CTkLabel(

        self.scrollable_frame,

        text="Loading items...",

        font=("Arial", 20, "bold")

    )

    loading_label.grid(row=0, column=0, columnspan=3, pady=50)

    print("Requesting data from server...")

    self.Send_Message(f"DATA_REQUEST#{self.secret_code}")

    if hasattr(self, '_data_request_timeout'):

        self.root.after_cancel(self._data_request_timeout)

    self._check_attempts = 0

    self.root.after(500, lambda:

self.check_for_items_data(loading_label))

    self._data_request_timeout = self.root.after(10000, lambda:

self.data_request_timeout(loading_label))

def data_request_timeout(self, loading_label):

    print("Timeout waiting for data response")

    loading_label.destroy()

    error_label = ctk.CTkLabel(

        self.scrollable_frame,

        text="Could not load items. Server not responding.",

        font=("Arial", 20, "bold"),
```



```
        text_color="red"

    )

    error_label.grid(row=0, column=0, columnspan=3, pady=50)

    retry_button = ctk.CTkButton(

        self.scrollable_frame,

        text="Retry",

        command=lambda: self.retry_load_items(error_label, retry_button)

    )

    retry_button.grid(row=1, column=1, pady=20)

def retry_load_items(self, error_label, retry_button):

    error_label.destroy()

    retry_button.destroy()

    self.load_items()

def check_for_items_data(self, loading_label):

    if self.in_message != '':

        print(f"Checking for data in message of length:

{len(self.in_message)}")

        if hasattr(self, '_data_request_timeout'):

            self.root.after_cancel(self._data_request_timeout)

        if not self.in_message.startswith('MAIN#DATA#'):

            print(f"Message doesn't start with MAIN#DATA#:

{self.in_message[:20]}...")

            self._check_attempts += 1

            if self._check_attempts < 10:
```



```
        self.root.after(1000, lambda:
self.check_for_items_data(loading_label))

        else:

            self.data_request_timeout(loading_label)

        return

data_str = self.in_message[10:]

self.in_message = ''

loading_label.destroy()

if data_str == "NO_DATA":

    empty_label = ctk.CTkLabel(

        self.scrollable_frame,

        text="No items found. Be the first to publish!",

        font=("Arial", 20, "bold")

    )

    empty_label.grid(row=0, column=0, columnspan=3, pady=50)

else:

    try:

        import json

        items = []

        try:

            # Try to parse the entire data string as JSON

            json_data = json.loads(data_str)

            if isinstance(json_data, list):

                items = json_data

            else:

                print(f"Received non-list JSON:

{type(json_data)}")
```



```
except json.JSONDecodeError as e:

    print(f"JSON decode error: {e}")

    if data_str.startswith '[' and
data_str.endswith(']'):

        import re

        # Find each item - look for patterns with
item_id@{json}

        item_matches = re.findall(r'\d+@{[^}]+\}',
data_str)

        if item_matches:

            for item in item_matches:

                items.append(item)

        else:

            try:

                item_objects =
re.findall(r'\{[^}]*(?:\{[^}]*\}[^}]*)*\}', data_str)

                for item_obj in item_objects:

                    items.append(item_obj)

            except Exception as inner_e:

                print(f"Error in fallback parsing:
{inner_e}")

    if not items:

        empty_label = ctk.CTkLabel(

            self.scrollable_frame,

            text="No items found.",

            font=("Arial", 20, "bold")

        )

        empty_label.grid(row=0, column=0, columnspan=3,
pady=50)
```



```
        else:

            self.process_items_list(items)

    except Exception as e:

        print(f"Error processing items: {e}")

        import traceback

        traceback.print_exc()

    else:

        self._check_attempts += 1

        delay = min(500 * (1.5 ** min(self._check_attempts, 5)), 3000)

        if self._check_attempts < 20:

            self.root.after(int(delay), lambda:
self.check_for_items_data(loading_label))

        else:

            self.data_request_timeout(loading_label)

def process_items_list(self, json_list):

    for widget in self.scrollable_frame_items:

        if widget.winfo_exists():

            widget.destroy()

    self.scrollable_frame_items = []

    self.row = 0

    self.col = 0

    max_cols = 3

    print(f"Processing {len(json_list)} items")

    for i, json_item in enumerate(json_list):

        try:
```



```
# Skip empty items

if not json_item:

    continue

# Handle different item formats

if isinstance(json_item, str):

    if '@' in json_item:

        # This is the item_id@json_data format

        parts = json_item.split('@', 1)

        if len(parts) == 2:

            item_id = parts[0].strip()

            json_data = parts[1].strip()

        else:

            json_data = json_item

    else:

        json_data = json_item

    json_data = json_data.strip()

    if (json_data.startswith('"') and
        json_data.endswith('"')) or \
        (json_data.startswith("'") and
        json_data.endswith("'")):

        json_data = json_data[1:-1]

    json_data = json_data.replace('\\"', '"').replace("\'",
    "'")

    else:

        json_data = json_item

    self.pack_items_to_frame(json_data, item_id, self.row,
    self.col)
```



```
        self.col += 1

        if self.col >= max_cols:

            self.col = 0

            self.row += 1

    except Exception as e:

        print(f"Error processing item {i}: {e}")

        import traceback

        traceback.print_exc()

        error_frame = ctk.CTkFrame(self.scrollable_frame,
fg_color="red")

        error_frame.configure(width=int(self.screen_width * 0.2),
height=int(self.screen_width * 0.2 * 0.5))

        error_frame.grid(row=self.row, column=self.col, padx=10,
pady=10, sticky="n")

        error_label = ctk.CTkLabel(

            error_frame,

            text="Error loading item",

            font=("Arial", 14, "bold"),

            text_color="white"

        )

        error_label.place(relx=0.5, rely=0.5, anchor="center")

        self.scrollable_frame_items.append(error_frame)

        self.col += 1

        if self.col >= max_cols:

            self.col = 0

            self.row += 1
```



```
def profile_pressed(self):  
  
    print("profile")  
  
    self.profile_root = ctk.CTkToplevel(self.root)  
  
    self.profile_root.title("FanZone Marketplace")  
  
    self.profile_root.iconbitmap(r'FZ.ico')  
  
    self.profile_root.geometry("405x720+-10+-10")  
  
    self.profile_root.grab_set()  
  
    self.profile_root.grid_columnconfigure(0, weight=1)  
  
    pf_header = ctk.CTkFrame(self.profile_root, width=405,  
height=int(self.screen_height * 0.12), fg_color=self.opposite_color)  
  
    pf_header.grid(row=0, column=0, sticky="nsew")  
  
    pf_header.grid_columnconfigure(0, weight=1)  
  
    pf_header.grid_columnconfigure(1, weight=1)  
  
    profile_name_width = int(self.screen_width * 0.10)  
  
    profile_font_size = int(min(self.screen_width, self.screen_height) *  
0.030)  
  
    profile_name_label = ctk.CTkLabel(  
  
        pf_header,  
  
        text=f"{self.user}",  
  
        font=("Arial", profile_font_size),  
  
        fg_color=self.opposite_color,  
  
        text_color=self.white_or_black,  
  
        width=profile_name_width  
  
    )  
  
    profile_name_label.grid(row=0, column=0, padx=0, pady=10,  
sticky='w')  
  
  
    pf_pic_label = ctk.CTkLabel(  
  
        pf_header,  
  
        image=self.pfp,
```



```
        text="",

        fg_color=self.opposite_color

    )

    pf_pic_label.grid(row=0, column=1, padx=0, pady=10)


    self.pf_scrollable_frame = ctk.CTkScrollableFrame(self.profile_root,
fg_color=self.defaultbg, height=720-int(self.screen_height * 0.12))

    self.pf_scrollable_frame.grid(row=1, column=0, padx=(0, 0), pady=(0,
0), sticky="nsew")

    self.pf_scrollable_frame.grid_columnconfigure(0, weight=1)

    self.load_user_items()


def load_user_items(self):

    loading_label = ctk.CTkLabel(

        self.pf_scrollable_frame,

        text="Loading items...",

        font=("Arial", 20, "bold")

    )

    loading_label.grid(row=0, column=0, columnspan=3, pady=50)

    print(f"Requesting data for user: {self.user} from server...")

    self.Send_Message(f"DATA_FOR_USER_REQUEST#{self.secret_code}")

    if hasattr(self, '_data_for_user_request_timeout'):

self.profile_root.after_cancel(self._data_for_user_request_timeout)

        self._check_attempts = 0

        self.root.after(500, lambda:
self.check_for_profile_data(loading_label))
```



```
self._data_for_user_request_timeout = self.profile_root.after(10000,
lambda: self.user_data_request_timeout/loading_label))

def user_data_request_timeout(self, loading_label):

    print("Timeout waiting for data response")

    loading_label.destroy()

error_label = ctk.CTkLabel(

    self.pf_scrollable_frame,

    text="Could not load items. Server not responding.",

    font=("Arial", 20, "bold"),

    text_color="red"

)

error_label.grid(row=0, column=0, columnspan=3, pady=50)

retry_button = ctk.CTkButton(

    self.pf_scrollable_frame,

    text="Retry",

    command=lambda: self.retry_load_user_items(error_label,
retry_button)

)

retry_button.grid(row=1, column=1, pady=20)

def retry_load_user_items(self, error_label, retry_button):

    error_label.destroy()

    retry_button.destroy()

    self.load_user_items()

def check_for_profile_data(self, loading_label):

    if self.in_message != '':
```



```
        print(f"Checking for data in message of length:
{len(self.in_message)}")

        if hasattr(self, '_data_request_timeout'):

self.profile_root.after_cancel(self._data_for_user_request_timeout)

        if not self.in_message.startswith('MAIN#DATA_FOR_USER#'):

            print(f"Message doesn't start with MAIN#DATA_FOR_USER#:
{self.in_message[:20]}...")

            self._check_attempts += 1

            if self._check_attempts < 10:

                self.profile_root.after(1000, lambda:
self.check_for_profile_data(loading_label))

            else:

                self.user_data_request_timeout(loading_label)

            return

data_str = self.in_message[19:]

loading_label.destroy()

if data_str == "NO_DATA":

    empty_label = ctk.CTkLabel(

        self.pf_scrollable_frame,

        text="All items you'll publish will appear here",

        font=("Arial", 20, "bold")

    )

    empty_label.grid(row=0, column=0, columnspan=3, pady=50)

else:

    try:

        import json

        items = []
```



```
try:

    json_data = json.loads(data_str)

    if isinstance(json_data, list):

        items = json_data

    else:

        print(f"Received non-list JSON:
{type(json_data)}")

except json.JSONDecodeError as e:

    print(f"JSON decode error: {e}")

    if data_str.startswith '[' and
data_str.endswith(']'):

        import re

        # Find each item - look for patterns with
item_id@{json}

        item_matches = re.findall(r'\d+@{[^}]+\}',
data_str)

        if item_matches:

            for item in item_matches:

                items.append(item)

        else:

            try:

                item_objects =
re.findall(r'\{[^}]*(?:\{[^}]*\}[^}]*)*\}', data_str)

                for item_obj in item_objects:

                    items.append(item_obj)

            except Exception as inner_e:

                print(f"Error in fallback parsing:
{inner_e}")
```



```
        if not items:

            empty_label = ctk.CTkLabel(

                self.pf_scrollable_frame,

                text="No items found.",

                font=("Arial", 20, "bold")

            )

            empty_label.grid(row=0, column=0, columnspan=3,

pady=50)

        else:

            self.process_user_items_list(items)

    except Exception as e:

        print(f"Error processing items: {e}")

        import traceback

        traceback.print_exc()

    else:

        self._check_attempts += 1

        delay = min(500 * (1.5 ** min(self._check_attempts, 5)), 3000)

        if self._check_attempts < 20:

            self.profile_root.after(int(delay), lambda:

self.check_for_profile_data(loading_label))

        else:

            self.user_data_request_timeout(loading_label)

def process_user_items_list(self, json_list):

    for widget in self.profile_scrollable_frame_items:

        if widget.winfo_exists():

            widget.destroy()

    self.profile_scrollable_frame_items = []

    self.row = 0
```



```
status = ''

print(f"Processing {len(json_list)} items")

for i, json_item in enumerate(json_list):
    try:
        # Skip empty items
        if not json_item:
            continue

        # Handle different item formats
        if isinstance(json_item, str):
            if '@' in json_item:
                parts = json_item.split('@')
                print(parts)

                if len(parts) == 3:
                    item_id = parts[0].strip()
                    json_data = parts[1].strip()
                    status = parts[2]
                    print(status)

                else:
                    json_data = json_item

            else:
                json_data = json_item

        json_data = json_data.strip()

        if (json_data.startswith('"') and
            json_data.endswith('"')) or \
            (json_data.startswith("'") and
            json_data.endswith("'")):
            json_data = json_data[1:-1]
```



```
        json_data = json_data.replace('\\"', '"').replace("\'", "'"),
    """)

    else:

        json_data = json_item

    self.pack_user_items_to_frame(json_data, self.row, status)

    self.row += 1

except Exception as e:

    print(f"Error processing item {i}: {e}")

    import traceback

    traceback.print_exc()

    error_frame =
    ctk.CTkFrame(self.profile_scrollable_frame_items, fg_color="red")

    error_frame.configure(width=int(self.screen_width * 0.2),
height=int(self.screen_width * 0.2 * 0.5))

    error_frame.grid(row=self.row, column=self.col, padx=10,
pady=10, sticky="n")

    error_label = ctk.CTkLabel(

        error_frame,

        text="Error loading item",

        font=("Arial", 14, "bold"),

        text_color="white"

    )

    error_label.place(relx=0.5, rely=0.5, anchor="center")

    self.profile_scrollable_frame_items.append(error_frame)
```



```
        self.row += 1

def pack_user_items_to_frame(self, json_string, row, status):

    try:

        item_id = "unknown"

        original_json_data = json_string

        json_data = None

        if '@' in json_string:

            parts = json_string.split('@', 1)

            if len(parts) == 2:

                item_id = parts[0].strip()

                json_data = parts[1].strip()

            else:

                json_data = json_string

        else:

            json_data = json_string

        import json

        data = {}

        try:

            try:

                if json_data.startswith('"') and
json_data.endswith('"'):

                    json_data = json_data[1:-1]

                    json_data = json_data.replace("\\'", "'").replace('\'\\',
'')

                    data = json.loads(json_data)
```



```
except json.JSONDecodeError:

    cleaned = json_data.replace('\\"', '').replace("\\'",
    '')

    if not cleaned.startswith('{'):

        cleaned = '{' + cleaned

    if not cleaned.endswith('}'):

        cleaned = cleaned + '}'

    try:

        data = json.loads(cleaned)

    except json.JSONDecodeError:

        import re

        data = {}

        team_match = re.search(r'"team"\s*:\s*"([^"]+)"',
    json_data)

        if team_match:

            data['team'] = team_match.group(1)

            team_color_match =
    re.search(r'"team_color"\s*:\s*"([^"]+)"', json_data)

            if team_color_match:

                data['team_color'] = team_color_match.group(1)

            size_match = re.search(r'"size"\s*:\s*"([^"]+)"',
    json_data)

            if size_match:

                data['size'] = size_match.group(1)

            price_match = re.search(r'"price"\s*:\s*"([^"]+)"',
    json_data)

            if price_match:

                data['price'] = price_match.group(1)
```



```
        image_match = re.search(r'"image"\s*:\s*"([^"]*)"',
json_data)

        if image_match:

            data['image'] = image_match.group(1)

    except Exception as e:

        print(f"Error parsing item JSON: {e}")

    data = {

        "team": "Unknown Team",

        "size": "Unknown Size",

        "price": "0",

        "team_color": self.white_or_black

    }

    frame = ctk.CTkFrame(self.pf_scrollable_frame,
fg_color=self.opposite_color)

    self.item_width = 340

    frame.configure(width=self.item_width, height=self.item_width *
1.3)

    frame.grid(row=row, column=0, padx=(10, 10), pady=(10, 10),
sticky="n")

    self.profile_scrollable_frame_items.append(frame)

    frame.grid_columnconfigure(0, weight=1)

    team = data.get("team", "Unknown Title")

    team_color = data.get("team_color", self.white_or_black)

    size = data.get("size", "Unknown Size")

    price = data.get("price", "0")

    item_id = data.get("submission_id")

    from PIL import Image as PILImage

    placeholder = PILImage.new('RGB', (self.item_width - 20,
self.item_width - 20), color=(73, 109, 137))
```



```
img = ctk.CTkImage(  
    light_image=placeholder,  
    dark_image=placeholder,  
    size=(self.item_width - 20, self.item_width - 20)  
)  
  
if "image" in data and data["image"]:  
    try:  
        import requests  
        from io import BytesIO  
        from PIL import Image  
        import time  
  
        # Add user agent to mimic a browser  
        headers = {  
            'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64;  
x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124  
Safari/537.36'  
        }  
  
        print(data["image"])  
        max_retries = 3  
        retry_delay = 2  
  
        for attempt in range(max_retries):  
            try:  
                response = requests.get(data["image"],  
headers=headers)  
                response.raise_for_status()  
                pil_img = Image.open(BytesIO(response.content))
```



```
img = ctk.CTkImage(  
    light_image=pil_img,  
    dark_image=pil_img,  
    size=(self.item_width - 20, self.item_width  
- 20)  
  
    )  
  
    break  
  
except requests.exceptions.HTTPError as err:  
    if err.response.status_code == 429:  
        print(  
            f"Rate limited (attempt {attempt +  
1}/{max_retries}), waiting {retry_delay} seconds...")  
        time.sleep(retry_delay)  
        retry_delay *= 2  
    else:  
        raise  
  
except Exception as e:  
    print(f"Error loading image data: {e}")  
  
team_label = ctk.CTkLabel(  
    frame,  
    text=team,  
    font=("Arial", int(self.item_width * 0.12), "bold"),  
    text_color=team_color  
)  
team_label.grid(row=0, column=0, padx=10, pady=(5, 0))  
  
image_label = ctk.CTkLabel(frame, image=img, text="")  
image_label.grid(row=1, column=0, padx=10, pady=10)
```



```
size_label = ctk.CTkLabel(
    frame,
    text=size,
    font=("Arial", int(self.item_width * 0.08), "bold"),
    text_color=self.defaultbg
)
size_label.grid(row=2, column=0, padx=10, pady=(0, 5))
price_label = ctk.CTkLabel(
    frame,
    text=f'{price}₪',
    font=("Arial", int(self.item_width * 0.08), "bold"),
    text_color=self.defaultbg
)
price_label.grid(row=3, column=0, padx=10, pady=(0, 5))
print(status)

if status == 'Available':
    buy_button = ctk.CTkButton(
        frame,
        text='ON SALE',
        font=("Arial", int(self.item_width * 0.12), "bold"),
        fg_color='SeaGreen1',
        hover_color='SeaGreen1',
        text_color=self.defaultbg
    )
else:
    buy_button = ctk.CTkButton(
        frame,
        text=f'SOLD',
        font=("Arial", int(self.item_width * 0.12), "bold"),
```



```
        fg_color='gray50',

        hover_color='gray50',

        text_color='black'

    )

    buy_button.grid(row=4, column=0, padx=10, pady=(0, 10))

except Exception as e:

    print(f"Error displaying item: {e}")

    import traceback

    traceback.print_exc()

    error_frame = ctk.CTkFrame(self.pf_scrollable_frame,
fg_color="gray")

    error_frame.configure(width=int(self.screen_width * 0.2),
height=int(self.screen_width * 0.2 * 1.3))

    error_frame.grid(row=row, column=0, padx=10, pady=10,
sticky="n")

    error_frame.grid_propagate(False)

    self.scrollable_frame_items.append(error_frame)

    error_label = ctk.CTkLabel(

        error_frame,

        text="Error loading item",

        font=("Arial", 14, "bold"),

        text_color="white"

    )

    error_label.place(relx=0.5, rely=0.5, anchor="center")

def logo_pressed(self):

    self.clear_all_widgets()
```



```
self.landing_page()

def publish(self):

    print("publish")

    self.publish_root = ctk.CTkToplevel(self.root)

    self.publish_root.title("FanZone Marketplace")

    self.publish_root.iconbitmap(r'FZ.ico')

    self.publish_root.geometry("405x720+-10+-10")

    self.publish_root.grab_set()

    import tkinter as tk

    self.image_field_frame = tk.Frame(self.publish_root, width=250,
height=250,

                                borderwidth=2, relief="groove")

    self.image_field_frame.pack(pady=(20, 5))

    # Fix the size of the frame

    self.image_field_frame.pack_propagate(False)

    # Create a label to show "No image" until one is selected

    self.image_field = tk.Label(self.image_field_frame, text="No image
selected",

                                width=30, height=10)

    self.image_field.pack(expand=True, fill="both")

    # Continue with the rest of the UI elements

    self.publish_type_entry = ctk.CTkOptionMenu(self.publish_root,
width=200, fg_color=self.opposite_color,

                                dynamic_resizing=False,
button_color=self.opposite_color,

text_color=self.white_or_black,
```



```
button_hover_color=self.opposite_color,

values=["Match-Worn
Shirt", "T-Shirt", "Long-Sleeve",
"Jacket",
"Fleece", "Shorts", "Sweatpants",
"Scarf",
"Stickers", "Hat"])

self.publish_type_entry.set("Select Product Type")

self.publish_type_entry.pack(pady=5)

self.publish_size_entry = ctk.CTkOptionMenu(self.publish_root,
width=200, fg_color=self.opposite_color,
dynamic_resizing=False,
button_color=self.opposite_color,
text_color=self.white_or_black,

button_hover_color=self.opposite_color,

values=["XXS", "XS",
"S", "M", "L", "XL", "XXL", "XXXL+"])

self.publish_size_entry.set("Select Size")

self.publish_size_entry.pack(pady=5)

self.publish_team_name_entry = ctk.CTkOptionMenu(self.publish_root,
width=200, fg_color=self.opposite_color,
dynamic_resizing=False, button_color=self.opposite_color,
text_color=self.white_or_black,

button_hover_color=self.opposite_color,

values=["Hapoel Tel
Aviv", "Maccabi Tel Aviv",
"Maccabi
Haifa", "Hapoel Beer Sheva",
"Beitar
Jerusalem", "Hapoel Jerusalem",
```



```
Haifa", "Maccabi Natanya"]])

        self.publish_team_name_entry.set("Select Team")

        self.publish_team_name_entry.pack(pady=5)

        self.publish_description_entry = ctk.CTkEntry(self.publish_root,
font=("Arial", 12), width=200,

placeholder_text="Enter The Description/Name Of The Product",

fg_color=self.opposite_color, text_color=self.white_or_black,

placeholder_text_color=self.white_or_black,

border_color=self.opposite_color)

        self.publish_description_entry.pack(pady=5)

        self.publish_year_entry = ctk.CTkEntry(self.publish_root,
font=("Arial", 12), width=200,

placeholder_text="The Year

The Product Was Released",

fg_color=self.opposite_color,

text_color=self.white_or_black,

placeholder_text_color=self.white_or_black,

border_color=self.opposite_color)

        self.publish_year_entry.pack(pady=5)

        self.publish_price_entry = ctk.CTkEntry(self.publish_root,
font=("Arial", 12), width=200,

placeholder_text="Price (In

shekels)",

fg_color=self.opposite_color, text_color=self.white_or_black,

placeholder_text_color=self.white_or_black,
```



```
border_color=self.opposite_color)

        self.publish_price_entry.pack(pady=5)

        self.publish_city_entry = ctk.CTkEntry(self.publish_root,
font=("Arial", 12), width=200,

placeholder_text="City For

Pickup",

fg_color=self.opposite_color,

text_color=self.white_or_black,

placeholder_text_color=self.white_or_black,

border_color=self.opposite_color)

        self.publish_city_entry.pack(pady=5)

        upload_photo_button = ctk.CTkButton(

            self.publish_root,

            text="Load Image",

            fg_color=self.opposite_color,

            text_color=self.defaultbg,

            font=("Arial", 20),

            height=28, width=140,

            command=self.load_image

        )

        upload_photo_button.pack(pady=5)

        submit_button = ctk.CTkButton(

            self.publish_root,

            text="Publish Item",

            fg_color="SeaGreen1",

            text_color=self.white_or_black,

            font=("Arial", 30, "bold"),
```



```
        height=40, width=250,

        command=lambda: self.submit_publish_form(self.publish_root)

    )

    submit_button.pack(pady=20)

def load_image(self):

    try:

        import tkinter as tk

        from tkinter import filedialog

        file_path = filedialog.askopenfilename(

            filetypes=[("Image files", "*.jpg *.jpeg *.png *.gif *.bmp

*.tiff")]

        )

        if not file_path:

            print("No file selected")

            return

        self.image_file_path = file_path

        print(f"Selected image: {file_path}")

        # CRITICAL: Make sure the image field frame exists and is

visible

        if self.image_field_frame is None:

            self.image_field_frame = tk.Frame(self.publish_root,

width=250, height=250)

            self.image_field_frame.pack(pady=(20, 5))

        # Create the label if it doesn't exist yet

        if not hasattr(self, 'image_field') or self.image_field is None:
```



```
self.image_field = tk.Label(self.image_field_frame)

self.image_field.pack(padx=10, pady=10)

from PIL import Image, ImageTk

pil_image = Image.open(file_path)

print(f"Image size: {pil_image.width}x{pil_image.height},
format: {pil_image.format}")

# Resize the image to fit display
max_size = 200

if pil_image.width > max_size or pil_image.height > max_size:
    ratio = min(max_size / pil_image.width, max_size /
pil_image.height)

    new_width = int(pil_image.width * ratio)
    new_height = int(pil_image.height * ratio)

    pil_image = pil_image.resize((new_width, new_height),
Image.LANCZOS)

    print(f"Resized to: {new_width}x{new_height}")

# Save the resized image for later use
self.image_pil = pil_image

photo = ImageTk.PhotoImage(pil_image)

# CRITICAL: Store a reference to the image to prevent garbage
collection
self.photo_reference = photo

self.image_field.config(image=photo)

# Force update the UI to show the image immediately
```



```
self.image_field.update()

self.image_field_frame.update()


import io

buffer = io.BytesIO()

img_format = pil_image.format if pil_image.format else 'JPEG'

pil_image.save(buffer, format=img_format)

self.binary_data = buffer.getvalue()


print("Image loaded and displayed successfully")


except Exception as e:

    print(f"Error loading image: {e}")

    import traceback

    traceback.print_exc()


def submit_publish_form(self, window):

    try:

        if hasattr(self, '_publishing_in_progress') and
self._publishing_in_progress:

            return

        self._publishing_in_progress = True


        item_type = self.publish_type_entry.get()

        item_description = self.publish_description_entry.get()

        size = self.publish_size_entry.get()

        team = self.publish_team_name_entry.get()

        team_color = self.DetermineTeamColor(team)

        year = self.publish_year_entry.get()

        price = self.publish_price_entry.get()
```



```
city = self.publish_city_entry.get()

# Validate form before proceeding

if (item_type == "Select Product Type" or size == "Select Size"
or team == "Select Team"
    or item_description == '' or price == '' or year == ''
or city == ''):

    error_label = ctk.CTkLabel(window, text="Please fill in all
fields", text_color="red")

    error_label.pack(pady=5)

    window.after(3000, error_label.destroy)

    self._publishing_in_progress = False

    return

# Check if an image was uploaded

base64_data = ""

try:

    if hasattr(self, 'binary_data') and self.binary_data:

        import base64

        base64_data =
base64.b64encode(self.binary_data).decode('utf-8')

        print(f"Image encoded to base64, size:
{len(base64_data)} bytes")

    else:

        raise Exception("No image selected")

except Exception as e:

    error_label = ctk.CTkLabel(window, text=f"Please upload a
photo of the product: {e}", text_color="red")

    error_label.pack(pady=5)

    window.after(3000, error_label.destroy)

    self._publishing_in_progress = False

    return
```



```
import json

import uuid

# Generate a unique ID for this submission to prevent duplicates
submission_id = str(uuid.uuid4())[:8]

json_dict = {

    "item_type": item_type,

    "image": base64_data,

    "item_title": item_description,

    "size": size,

    "price": price,

    "team": team,

    "year": year,

    "city_of_pickup": city,

    "team_color": team_color,

    "secret_code": str(self.secret_code),

    "submission_id": submission_id

}

print(f"Publishing item with title: {item_description}")

json_string = json.dumps(json_dict, ensure_ascii=False)

# Verify it's valid JSON by parsing it back
try:

    # This will throw an error if the JSON is invalid
    test_parse = json.loads(json_string)

except json.JSONDecodeError as e:

    print(f"Generated invalid JSON: {e}")
```



```
        error_label = ctk.CTkLabel(window, text=f"Error creating
item data. Please try again.",
                                   text_color="red")

        error_label.pack(pady=5)

        window.after(3000, error_label.destroy)

        self._publishing_in_progress = False

        return

    print(f"JSON data size: {len(json_string)} bytes")

    if len(json_string) > 1000000:

        error_label = ctk.CTkLabel(window, text="Image is too large.
Please use a smaller image.",
                                   text_color="red")

        error_label.pack(pady=5)

        window.after(3000, error_label.destroy)

        self._publishing_in_progress = False

        return

    self.Send_Message(json_string)

    status_label = ctk.CTkLabel(window, text="Publishing item,
please wait...",
                                text_color="green", font=("Arial",
16, "bold"))

    status_label.pack(pady=5)

    self._publish_check_count = 0

    self.root.after(1000, lambda:
self.check_publish_response(window, status_label, submission_id))
```



```
        self._publish_timeout = window.after(15000, lambda:
self.publish_timeout(window, status_label))

    except Exception as e:

        print(f"Error submitting form: {e}")

        import traceback

        traceback.print_exc()

        error_label = ctk.CTkLabel(window, text=f"Error: {e}",
text_color="red")

        error_label.pack(pady=5)

        window.after(3000, error_label.destroy)

        self._publishing_in_progress = False

    def publish_timeout(self, window, status_label):

        status_label.configure(text="Server not responding. Item may have
been published. Check your profile", text_color="orange")

        window.after(3000, window.destroy)

    def check_publish_response(self, window, status_label,
submission_id=None):

        if self.in_message != '':

            print(f"Received publish response: {self.in_message}")

            if hasattr(self, '_publish_timeout'):

                window.after_cancel(self._publish_timeout)

            parts = self.in_message.split('#')

            if len(parts) < 3:

                print(f"Invalid publish response format: {self.in_message}")

                self._publish_check_count += 1
```



```
        if self._publish_check_count < 10:

            window.after(500, lambda:
self.check_publish_response(window, status_label, submission_id))

        else:

            self.publish_timeout(window, status_label)

        return

    if parts[0] == 'MAIN' and parts[1] == 'PUBLISH_ITEM' and
len(parts) >= 3:

        self.in_message = ''

        if parts[2] == 'OK':

            print('Item published successfully')

            status_label.configure(text="Item published
successfully!")

            self._publishing_in_progress = False

            window.after(2000, window.destroy)

        else:

            print(f'Failed to publish item: {parts[2]}')

            status_label.configure(text=f"Failed to publish item:
{parts[2]}", text_color="red")

            window.after(3000, status_label.destroy)

            self._publishing_in_progress = False

        else:

            self._publish_check_count += 1

            if self._publish_check_count < 10:

                window.after(500, lambda:
self.check_publish_response(window, status_label, submission_id))

            else:

                self.publish_timeout(window, status_label)

    else:
```



```
        self._publish_check_count += 1

        if self._publish_check_count < 20:

            window.after(500, lambda:
self.check_publish_response(window, status_label, submission_id))

        else:

            self.publish_timeout(window, status_label)

def clear_all_widgets(self):

    for widget in self.root.winfo_children():

        widget.destroy()

def put_all_items(self, json_list):

    self.row = 0

    self.col = 0

    max_cols = 3

    print(f"Displaying {len(json_list)} items")

    for widget in self.scrollable_frame_items:

        if widget.winfo_exists():

            widget.destroy()

    self.scrollable_frame_items = []

    for i, json_item in enumerate(json_list):

        try:

            if not json_item:

                continue

            if isinstance(json_item, str):

                json_item = json_item.strip()
```



```
        if json_item.startswith('"') and
json_item.endswith('"'):

            json_item = json_item[1:-1]

            item_id = "unknown"

            self.pack_items_to_frame(json_item, item_id, self.row,
self.col)

            self.col += 1

            if self.col >= max_cols:

                self.col = 0

                self.row += 1

    except Exception as e:

        print(f"Error displaying item {i}: {e}")

        self.col += 1

        if self.col >= max_cols:

            self.col = 0

            self.row += 1

def pack_items_to_frame(self, json_string, item_id, row, column):

    try:

        original_json_data = json_string

        json_data = None

        if '@' in json_string:

            parts = json_string.split('@', 1)

            if len(parts) == 2:

                item_id = parts[0].strip()

                json_data = parts[1].strip()

            else:

                json_data = json_string

        else:
```



```
json_data = json_string

import json

data = {}

try:
    try:
        if json_data.startswith('"') and
json_data.endswith('"'):
            json_data = json_data[1:-1]

            json_data = json_data.replace("\\'", "'").replace('\\"',
"'')

            data = json.loads(json_data)
        except json.JSONDecodeError:
            cleaned = json_data.replace('\\"', "'").replace("\\'",
"'')

            if not cleaned.startswith('{'):
                cleaned = '{' + cleaned

            if not cleaned.endswith('}'):
                cleaned = cleaned + '}'

            try:
                data = json.loads(cleaned)
            except json.JSONDecodeError:
                import re

                data = {}

                team_match = re.search(r'"team"\s*:\s*"([^"]+)"',
json_data)

                if team_match:
                    data['team'] = team_match.group(1)
```



```
        team_color_match =
re.search(r'"team_color"\s*:\s*"([^"]+)"', json_data)

        if team_color_match:

            data['team_color'] = team_color_match.group(1)

        size_match = re.search(r'"size"\s*:\s*"([^"]+)"',
json_data)

        if size_match:

            data['size'] = size_match.group(1)

        price_match = re.search(r'"price"\s*:\s*"([^"]+)"',
json_data)

        if price_match:

            data['price'] = price_match.group(1)

        image_match = re.search(r'"image"\s*:\s*"([^"]*)"',
json_data)

        if image_match:

            data['image'] = image_match.group(1)

    except Exception as e:

        print(f"Error parsing item JSON: {e}")

        data = {

            "team": "Unknown Team",

            "size": "Unknown Size",

            "price": "0",

            "team_color": self.white_or_black

        }

        frame = ctk.CTkFrame(self.scrollable_frame,
fg_color=self.opposite_color)

        self.item_width = int(self.screen_width * 0.2)
```



```
frame.configure(width=self.item_width, height=self.item_width *
1.3)

frame.grid(row=row, column=column, padx=(10, 10), pady=(10, 10),
sticky="n")

self.scrollable_frame_items.append(frame)

frame.grid_columnconfigure(0, weight=1)

team = data.get("team", "Unknown Team")
team_color = data.get("team_color", self.white_or_black)
size = data.get("size", "Unknown Size")
price = data.get("price", "0")

from PIL import Image as PILImage

placeholder = PILImage.new('RGB', (self.item_width - 20,
self.item_width - 20), color=(73, 109, 137))

img = ctk.CTkImage(

    light_image=placeholder,

    dark_image=placeholder,

    size=(self.item_width - 20, self.item_width - 20)

)

if "image" in data and data["image"]:

    try:

        import requests

        from io import BytesIO

        from PIL import Image

        import time

        # Add user agent to mimic a browser

        headers = {
```



```
'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64;
x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124
Safari/537.36'

    }

    print(data["image"])

    max_retries = 3

    retry_delay = 2

    for attempt in range(max_retries):

        try:

            response = requests.get(data["image"],
headers=headers)

            response.raise_for_status()

            pil_img = Image.open(BytesIO(response.content))

            img = ctk.CTkImage(

                light_image=pil_img,

                dark_image=pil_img,

                size=(self.item_width - 20, self.item_width
- 20)

            )

            break # Success, exit retry loop

        except requests.exceptions.HTTPError as err:

            if err.response.status_code == 429:

                print(

                    f"Rate limited (attempt {attempt +
1}/{max_retries}), waiting {retry_delay} seconds..."

                )

                time.sleep(retry_delay)

                retry_delay *= 2 # Exponential backoff

            else:
```



```
error                                     raise # Re-raise if it's not a rate limit

except Exception as e:
    print(f"Error loading image data: {e}")

team_label = ctk.CTkLabel(
    frame,
    text=team,
    font=("Arial", int(self.item_width * 0.12), "bold"),
    text_color=team_color
)
team_label.grid(row=0, column=0, padx=10, pady=(5, 0))

image_label = ctk.CTkLabel(frame, image=img, text="")
image_label.grid(row=1, column=0, padx=10, pady=10)

size_label = ctk.CTkLabel(
    frame,
    text=size,
    font=("Arial", int(self.item_width * 0.08), "bold"),
    text_color=self.defaultbg
)
size_label.grid(row=2, column=0, padx=10, pady=(0, 5))

buy_button = ctk.CTkButton(
    frame,
    text=f'{price}₪',
    font=("Arial", int(self.item_width * 0.12), "bold"),
    fg_color=team_color,
```



```
        hover_color=team_color.replace("2", "3"),

        text_color=self.defaulttbg,

        command=lambda: self.display_item(original_json_data,
item_id, img, team, size, price, team_color)

    )

    buy_button.grid(row=3, column=0, padx=10, pady=(0, 10))

except Exception as e:

    print(f"Error displaying item: {e}")

    import traceback

    traceback.print_exc()

    error_frame = ctk.CTkFrame(self.scrollable_frame,
fg_color="gray")

    error_frame.configure(width=int(self.screen_width * 0.2),
height=int(self.screen_width * 0.2 * 1.3))

    error_frame.grid(row=row, column=column, padx=10, pady=10,
sticky="n")

    error_frame.grid_propagate(False)

    self.scrollable_frame_items.append(error_frame)

    error_label = ctk.CTkLabel(

        error_frame,

        text="Error loading item",

        font=("Arial", 14, "bold"),

        text_color="white"

    )

    error_label.place(relx=0.5, rely=0.5, anchor="center")

def display_item(self, json_data, item_id, img, team, size, price,
team_color):

    import json
```



```
data = {}

try:
    try:
        if json_data.startswith('"') and json_data.endswith('"'):
            json_data = json_data[1:-1]

        json_data = json_data.replace("\\'", "'").replace('\\"',
'''

        data = json.loads(json_data)
    except json.JSONDecodeError:
        cleaned = json_data.replace('\\\"', '"').replace('\\"', '"')
        if not cleaned.startswith('{'):
            cleaned = '{' + cleaned
        if not cleaned.endswith('}'):
            cleaned = cleaned + '}'

        try:
            data = json.loads(cleaned)
        except json.JSONDecodeError:
            import re

            data = {}

            item_type_match =
re.search(r'"item_type"\s*:\s*"([^"]+)"', json_data)

            if item_type_match:
                data['item_type'] = item_type_match.group(1)

            item_title_match =
re.search(r'"item_title"\s*:\s*"([^"]+)"', json_data)

            if item_title_match:
```



```
data['item_title'] = item_title_match.group(1)

year_match = re.search(r'"year"\s*:\s*"([^"]+)"',
json_data)

if year_match:
    data['year'] = year_match.group(1)

city_match =
re.search(r'"city_of_pickup"\s*:\s*"([^"]+)"', json_data)

if city_match:
    data['city'] = city_match.group(1)

except Exception as e:
    print(f"Error parsing item JSON: {e}")

item_type = data.get("item_type", "Unknown Team")
item_title = data.get("item_title", "No Title")
year = data.get("year", "Unknown Year")
city_of_pickup = data.get("city_of_pickup", "Israel")

item_root = ctk.CTkToplevel(self.root)
item_root.title("FanZone Marketplace")
item_root.iconbitmap(r'FZ.ico')
item_root.geometry("405x720+-10+-10")
item_root.grab_set()

title_label = ctk.CTkLabel(
    item_root,
    text=item_title,
    font=("Arial", 20, "bold"),
    text_color=self.opposite_color
```



```
)

title_label.pack(pady=(20, 5))

image_label = ctk.CTkLabel(item_root, image=img, width=250,
height=250, text="")

image_label.pack(pady=5)

team_label = ctk.CTkLabel(

    item_root,

    text=f'Team: {team}',

    font=("Arial", 20),

    text_color=self.opposite_color

)

team_label.pack(pady=5)

size_label = ctk.CTkLabel(

    item_root,

    text=f'Size: {size}',

    font=("Arial", 20),

    text_color=self.opposite_color

)

size_label.pack(pady=5)

year_label = ctk.CTkLabel(

    item_root,

    text=f'Year produced: {year}',

    font=("Arial", 20),

    text_color=self.opposite_color

)

year_label.pack(pady=5)
```



```
city_label = ctk.CTkLabel(
    item_root,
    text=f'Location of pickup: {city_of_pickup}',
    font=("Arial", 20),
    text_color=self.opposite_color
)
city_label.pack(pady=5)

price_label = ctk.CTkLabel(
    item_root,
    text=f'Price: {price}₪',
    font=("Arial", 25, "bold"),
    text_color=self.opposite_color
)
price_label.pack(pady=5)

buy_button = ctk.CTkButton(
    item_root,
    text=f'BUY',
    font=("Arial", 30, "bold"),
    height=40, width=250,
    fg_color=team_color,
    hover_color=team_color.replace("2", "3"),
    text_color=self.defaultbg,
    command=lambda: self.buy_item(item_id, price, item_title,
item_root)
)
buy_button.pack(pady=5)
```



```
def buy_item(self, item_id, price, item_title, window):

    print("bought")

    try:

        if hasattr(self, '_buying_in_progress') and
self._buying_in_progress:

            return

        self._buying_in_progress = True

self.Send_Message(f"BUY_ITEM#{self.secret_code}#{item_id}©{price}©{item_title}")

        status_label = ctk.CTkLabel(window, text="Buying item, please
wait...",

                                     text_color="green", font=("Arial",
16, "bold"))

        status_label.pack(pady=5)

        self._buy_check_count = 0

        self.root.after(1000, lambda: self.check_buy_response(window,
status_label))

        self._buyin_timeout = window.after(15000, lambda:
self.buy_timeout(window, status_label))

    except Exception as e:

        print(f"Error submitting form: {e}")

        import traceback

        traceback.print_exc()

        error_label = ctk.CTkLabel(window, text=f"Error: {e}",
text_color="red")

        error_label.pack(pady=5)

        window.after(3000, error_label.destroy)
```



```
self._buying_in_progress = False

def check_buy_response(self, window, status_label, submission_id=None):
    if self.in_message != '':
        print(f"Received buy response: {self.in_message}")

        if hasattr(self, '_buy_timeout'):
            window.after_cancel(self._buy_timeout)

        parts = self.in_message.split('#')

        if len(parts) < 3:
            print(f"Invalid publish response format: {self.in_message}")
            self._buy_check_count += 1
            if self._buy_check_count < 10:
                window.after(500, lambda:
self.check_publish_response(window, status_label, submission_id))
            else:
                self.buy_timeout(window, status_label)
            return

        if parts[0] == 'MAIN' and parts[1] == 'BUY_ITEM' and len(parts)
>= 3:
            self.in_message = '' # Reset message

            if parts[2] == 'OK':
                print('Item bought successfully')
                status_label.configure(text="Item bought successfully!")

                self._buying_in_progress = False
```



```
        window.after(2000, window.destroy)

        self.root.after(2500, lambda: self.clear_all_widgets())

        self.root.after(3000, lambda: self.landing_page())

    else:

        print(f'Failed to buy item: {parts[2]}')

        status_label.configure(text=f"Failed to buy item: {parts[2]}", text_color="red")

        window.after(3000, status_label.destroy)

        self._buying_in_progress = False

    else:

        self._buy_check_count += 1

        if self._buy_check_count < 10:

            window.after(500, lambda: self.check_buy_response(window, status_label, submission_id))

        else:

            self.buy_timeout(window, status_label)

    else:

        self._buy_check_count += 1

        if self._buy_check_count < 20:

            window.after(1000, lambda: self.check_buy_response(window, status_label, submission_id))

        else:

            self.buy_timeout(window, status_label)

def buy_timeout(self, window, status_label):

    status_label.configure(text="Server not responding. Item may have been bought. Check your E-mail", text_color="orange")

    window.after(3000, window.destroy)

    self.clear_all_widgets()

    self.landing_page()

def DetermineTeamColor(self, team):

    if team == "Hapoel Tel Aviv" or team == "Hapoel Jerusalem":
```



```
        return "red2"

    if team == "Maccabi Tel Aviv":

        return "yellow2"

    if team == "Maccabi Haifa":

        return "green2"

    if team == "Beitar Jerusalem" or team == "Maccabi Natanya":

        return "gold2"

    if team == "Hapoel Haifa" or team == "Hapoel Beer Sheva":

        return "firebrick2"

    return "gray2"

def Menu(self, master, **kwargs):

    return ctk.CTkFrame(master, **kwargs)
```



server.py

```
import random

import threading

import logging

from server_communication import Server_com

from database import Database

import smtplib

from email.message import EmailMessage

import time

import sqlite3

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

def com_thread():

    global server

    thread_db = Database()

    logging.info("Communication thread started")

    while True:

        try:

            server.run()

            server.inL.acquire()

            if len(server.in_msgs) > 0:

                msgs = server.in_msgs.copy()

                server.in_msgs.clear()

                server.inL.release()

                handle_msgs(msgs, thread_db)

            else:
```



```
        server.inL.release()

        time.sleep(0.01)

    except Exception as e:

        logging.error(f"Error in communication thread: {e}")

        time.sleep(1)

thread_db.Disconnect()


def handle_msgs(msgs, thread_db):

    global code, codes, server, secret_code, secret_codes

    logging.info(f'Handling {len(msgs)} message(s)')

    processed_messages = set()

    for socket, msg_text in msgs:

        message_id = f"{socket}:{hash(msg_text)}"

        if message_id in processed_messages:

            logging.info(f"Skipping duplicate message: {message_id}")

            continue

        processed_messages.add(message_id)

        logging.info(f'Processing message: {msg_text}')

        parts = msg_text.split('#')

        if len(parts) < 2:

            logging.warning(f"Invalid message format: {msg_text}")

            continue

        if parts[0] == 'USER_INTRO':
```



```
        if parts[1] == 'LOGIN_REQUEST' and len(parts) >= 3:

            reply = thread_db.login_compare(parts[2])

            logging.info(f"Login response: {reply} for request:
{parts[2]}")

            server.out_msgs.append((socket, 'USER_INTRO#LOGIN_RESPONSE#'
+ reply))

        elif parts[1] == 'SIGNUP_REQUEST' and len(parts) >= 3:

            try:

                user_data = parts[2].split('@')

                if len(user_data) >= 4:

                    logging.info(f"Processing signup for:
{user_data[0]}")

                    reply = thread_db.AddToUserPass(user_data[0],
user_data[1], user_data[2], user_data[3])

                    server.add_to_out((socket,
'USER_INTRO#SIGNUP_RESPONSE#' + reply))

                    print("signup sent")

                else:

                    logging.error(f"Invalid signup data format:
{parts[2]}")

                    server.add_to_out((socket,
'USER_INTRO#SIGNUP_RESPONSE#FORMAT_ERROR'))

            except Exception as e:

                logging.error(f"Error processing signup: {e}")

                server.add_to_out((socket,
'USER_INTRO#SIGNUP_RESPONSE#ERROR'))

        elif parts[1] == 'CODE_REQUEST' and len(parts) >= 3:

            verification_code = random.randint(10000, 100000)

            logging.info(f"Sending verification code to: {parts[2]}")

            send_mail(parts[2], verification_code)

        elif parts[1] == 'CODE_VALIDATION' and len(parts) >= 3:
```



```
        try:
            user_email = parts[2].split(':')[0] if ':' in parts[2]
        else ""
            user_code = parts[2].split(':')[1] if ':' in parts[2]
        else parts[2]

        logging.info(f"Validating code for: {user_email}")

        valid = False

        for entry in codes:
            if entry.startswith(user_email + "#"):
                stored_code = entry.split('#')[1]
                if user_code == stored_code:
                    valid = True
                    break

        server.outL.acquire()

        if valid:
            secret_code = random.randint(100, 999) *
random.randint(100, 999)
            logging.info(f'Generated secret code: {secret_code}
for {user_email}')
            server.out_msgs.append((socket,
f'USER_INTRO#CODE_VALIDATION#OK:{secret_code}'))
            server.client_secret_codes.append(f'{socket}:{secret_code}:{user_email}')
            logging.info(f"Active clients:
{server.client_secret_codes}")
        else:
            logging.warning(f"Invalid code for {user_email}")
            server.out_msgs.append((socket,
'USER_INTRO#CODE_VALIDATION#NO'))
            server.outL.release()

    except Exception as e:
```



```
        logging.error(f"Error validating code: {e}")

        server.outL.acquire()

        server.out_msgs.append((socket,
'USER_INTRO#CODE_VALIDATION#ERROR'))

        server.outL.release()

    elif parts[0] == 'MAIN':

        try:

            if len(parts) < 3:

                logging.warning(f"Invalid MAIN message format:
{msg_text}")

                continue

            logging.info(f"Processing MAIN request: {parts[1]} with
code: {parts[2]}")

            if parts[1] == 'PUBLISH_ITEM' and len(parts) >= 4:

                logged = False

                email = ''

                for client_info in server.client_secret_codes:

                    client_parts = client_info.split(':')

                    if len(client_parts) >= 3:

                        if client_parts[1] == parts[3] or str(socket) in
client_parts[0]:

                            logged = True

                            email = client_parts[2]

                            logging.info(f"Client authenticated:
{email}")

                            break

                item_data = parts[2]

                logging.info(f"Publishing item with data length:
{len(item_data)}")
```



```
import json

try:
    data = json.loads(item_data)

    if 'submission_id' in data:
        submission_id = data['submission_id']

        if hasattr(thread_db,
                    'processed_submissions') and
submission_id in thread_db.processed_submissions:

            logging.warning(f"Duplicate submission
detected: {submission_id}")

            server.outL.acquire()

            server.out_msgs.append((socket,
f'MAIN#PUBLISH_ITEM#DUPLICATE'))

            server.outL.release()

            continue

        if not hasattr(thread_db,
'processed_submissions'):

            thread_db.processed_submissions = set()

thread_db.processed_submissions.add(submission_id)

    except json.JSONDecodeError as e:

        logging.warning(f"Could not parse item JSON to check
for duplicates: {e}")

    except Exception as e:

        logging.error(f"Error checking for duplicate
submission: {e}")

    logged = False

    email = ''

    for client_info in server.client_secret_codes:
```



```
client_parts = client_info.split(':')

if len(client_parts) >= 3 and str(socket) in
client_parts[0]:

    logged = True

    email = client_parts[2]

    logging.info(f"Found client: {email}")

    break

if logged:

    reply = thread_db.AddData(item_data, email)

    logging.info(f"Publish item result: {reply}")

    server.outL.acquire()

    server.out_msgs.append((socket,
f'MAIN#PUBLISH_ITEM#{reply}'))

    server.outL.release()

else:

    logging.warning("Client not authenticated for
publish")

    server.outL.acquire()

    server.out_msgs.append((socket,
'MAIN#PUBLISH_ITEM#NOT_AUTHENTICATED'))

    server.outL.release()

else:

    logged = False

    email = ''

for client_info in server.client_secret_codes:

    client_parts = client_info.split(':')

    if len(client_parts) >= 3:

        if client_parts[1] == parts[2] or str(socket) in
client_parts[0]:

            logged = True
```



```
        email = client_parts[2]

        logging.info(f"Client authenticated:

{email}")

        break

    if logged or parts[2] == '21':

        if parts[1] == 'USERNAME_REQUEST':

            reply = thread_db.get_user(email) if email else

'admin'

            logging.info(f"Username request for {email}:

{reply}")

            server.outL.acquire()

            server.out_msgs.append((socket,

f'MAIN#USERNAME#{reply}'))

            server.outL.release()

        elif parts[1] == 'DATA_REQUEST':

            reply = thread_db.GetData(email)

            logging.info(f"Data response length:

{len(str(reply))}")

            server.outL.acquire()

            server.out_msgs.append((socket,

f'MAIN#DATA#{reply}'))

            server.outL.release()

        elif parts[1] == 'DATA_FOR_USER_REQUEST':

            logging.info(f"Data request for: {email}")

            reply = thread_db.GetDataForUser(email)

            logging.info(f"Data response length:

{len(str(reply))}")

            server.outL.acquire()

            server.out_msgs.append((socket,

f'MAIN#DATA_FOR_USER#{reply}'))

            server.outL.release()

        elif parts[1] == 'BUY_ITEM':
```



```
logging.info(f"Data request for: {email}")

item_title = parts[3].split('©')[2]

price = parts[3].split('©')[1]

reply =
thread_db.BuyItem(parts[3].split('©')[0], email, price, item_title)

mail_sent = send_bought_mail(email, reply,
item_title, price)

if mail_sent == 'OK' and reply["response"] ==
'OK':

logging.info(f"Data response length:
{len(str(reply))}")

server.outL.acquire()

server.out_msgs.append((socket,

server.outL.release()

else:

logging.info(f"Data response length:
{len(str(reply))}")

server.outL.acquire()

server.out_msgs.append((socket,

f'MAIN#BUY_ITEM#FAIL'))

server.outL.release()

else:

logging.warning(f"Unauthenticated request:
{msg_text}")

server.outL.acquire()

server.out_msgs.append((socket,
'MAIN#ERROR#NOT_AUTHENTICATED'))

server.outL.release()

except Exception as e:

logging.error(f"Error processing MAIN request: {e}")

server.outL.acquire()

server.out_msgs.append((socket, f'MAIN#ERROR#{str(e)}'))

server.outL.release()
```



```
def send_mail(mail, data):

    global codes, server

    sender_email = 'fanzonemarketplce@gmail.com'

    sender_password = 'mdxgiaebdaxehkug'

    msg = EmailMessage()

    msg['Subject'] = 'Your Verification Code'

    msg['From'] = sender_email

    msg['To'] = mail

    msg.set_content(f'''

        <!DOCTYPE html>

        <html>

        <head>

            <link rel="stylesheet" type="text/css" href="https://fonts.googleapis.com/css?family=Lato|Lato:i,b,bi"
            hs-webfonts="true"

            <meta http-equiv="Content-Type" content="text/html;
            charset=UTF-8">

            <meta name="viewport" content="width=device-width,
            initial-scale=1.0">

            <style type="text/css">

                h1{{font-size:56px}}

                h2{{font-size:45px;font-weight:900}}

                p{{font-weight:100}}

                td{{vertical-align:top}}

                #email{{margin:auto;width:600px;background-color:#fff}}

            </style>

        </head>

        <body bgcolor="#F5F8FA" style="width: 100%; font-family:Lato,
        sans-serif; font-size:18px;">

        <div id="email">
```



```
<table role="presentation" width="100%">

    <tr>

        <td bgcolor="#2E8B57" align="center" style="color:
white;">

            <h1> FanZone Marketplace</h1>

        </td>

    </tr>

</table>

<table role="presentation" border="0" cellpadding="0"
cellspacing="10px" style="padding: 30px 30px 30px 60px;">

    <tr>

        <td>

            <h2>Two Factor Authentication</h2>

            <p>

                Your code is: {data}.

            </p>

        </td>

    </tr>

</table>

</div>

</body>

</html>

'', subtype='html')

try:

    with smtplib.SMTP_SSL('smtp.gmail.com', 465) as smtp:

        smtp.login(sender_email, sender_password)

        smtp.send_message(msg)

    logging.info(f'Verification code {data} sent to {mail}')

for sock in server.client_sockets:

    server.outL.acquire()
```



```
server.out_msgs.append((sock, 'USER_INTRO#CODE_RESPONSE#OK'))

server.outL.release()

codes.append(f"{mail}#{data}")

logging.info(f"Active codes: {codes}")

except Exception as e:

    logging.error(f'Failed to send email to {mail}. Error: {e}')

    # Find the client socket to send the error message

    for sock in server.client_sockets:

        server.outL.acquire()

        server.out_msgs.append((sock, 'USER_INTRO#CODE_RESPONSE#FAIL'))

        server.outL.release()

def send_bought_mail(buyer_mail, data, item_title, price):

    global codes, server

    sender_email = 'fanzonemarketplce@gmail.com'

    sender_password = 'mdxgiaebdaxehkug'

    msg = EmailMessage()

    msg['Subject'] = 'An Item You Publish Was Bought!'

    msg['From'] = sender_email

    msg['To'] = data["seller_mail"]

    msg.set_content(f'''

        <!DOCTYPE html>

        <html>

        <head>

            <link rel="stylesheet" type="text/css" hs-webfonts="true"

href="https://fonts.googleapis.com/css?family=Lato|Lato:i,b,bi">

            <meta http-equiv="Content-Type" content="text/html;

charset=UTF-8">

            <meta name="viewport" content="width=device-width,

initial-scale=1.0">
```



```
<style type="text/css">

    h1{{font-size:56px}}

    h2{{font-size:45px;font-weight:900}}

    p{{font-weight:100}}

    td{{vertical-align:top}}

    #email{{margin:auto;width:600px;background-color:#fff}}

    .contact-info {{margin-left: 20px;}}

    .transaction-info {{margin-left: 20px;}}

</style>

</head>

<body bgcolor="#F5F8FA" style="width: 100%; font-family:Lato,
sans-serif; font-size:18px;">

<div id="email">

    <table role="presentation" width="100%">

        <tr>

            <td bgcolor="#2E8B57" align="center" style="color:
white;">

                <h1> FanZone Marketplace</h1>

            </td>

        </tr>

    </table>

    <table role="presentation" border="0" cellpadding="0"
cellspacing="10px" style="padding: 30px 30px 30px 60px;">

        <tr>

            <td>

                <h2>{item_title} was bought!</h2>

                <p>Your item: {item_title} was bought by
{data["buyer_user"]}!</p>

                <p><strong>Buyer's contact information:</strong></p>

                <div class="contact-info">

                    <p>Email address: {buyer_mail}</p>

                    <p>Phone number: {data["buyer_phone"]}</p>

                </div>

            </td>

        </tr>

    </table>

</div>
```



```
</div>

<p>The buyer has received your contact information
as well and will contact you soon.</p>

<p>{price} ILS has been transferred to your bank
account.</p>

<hr>

<p><strong>Transaction information:</strong></p>

<div class="transaction-info">

    <p>Item: {item_title}</p>

    <p>Price: {price}</p>

    <p>Buyer: {data["buyer_user"]}</p>

    <p>Seller: {data["seller_user"]}</p>

    <p>Time And Date Of Transaction:
{data['time']}</p>

    <p>Transaction ID: {data["transaction_id"]}</p>

</div>

<hr>

</td>

</tr>

</table>

</div>

</body>

</html>

'', subtype='html')

msg2 = EmailMessage()

msg2['Subject'] = 'Item Bought Successfully!'

msg2['From'] = sender_email
```



```
msg2['To'] = buyer_mail

msg2.set_content(f'''

<!DOCTYPE html>

<html>

<head>

    <link rel="stylesheet" type="text/css" hs-webfonts="true"
href="https://fonts.googleapis.com/css?family=Lato|Lato:i,b,bi">

    <meta http-equiv="Content-Type" content="text/html;
charset=UTF-8">

    <meta name="viewport" content="width=device-width,
initial-scale=1.0">

    <style type="text/css">

        h1{{font-size:56px}}

        h2{{font-size:45px;font-weight:900}}

        p{{font-weight:100}}

        td{{vertical-align:top}}

        #email{{margin:auto;width:600px;background-color:#fff}}

        .contact-info {{margin-left: 20px;}}

        .transaction-info {{margin-left: 20px;}}

    </style>

</head>

<body bgcolor="#F5F8FA" style="width: 100%; font-family:Lato,
sans-serif; font-size:18px;">

    <div id="email">

        <table role="presentation" width="100%">

            <tr>

                <td bgcolor="#2E8B57" align="center" style="color:
white;">

                    <h1> FanZone Marketplace</h1>

                </td>

            </tr>

        </table>

        <table role="presentation" border="0" cellpadding="0"
cellspacing="10px" style="padding: 30px 30px 30px 60px;">
```



```
<tr>

    <td>

        <h2>Item Bought Successfully!</h2>

        <p>Your purchase of the item: {item_title} was
successful!</p>

        <p><strong>Seller's contact
information:</strong></p>

        <div class="contact-info">

            <p>Email address: {data["seller_mail"]}</p>

            <p>Phone number: {data["seller_phone"]}</p>

        </div>

        <p>We suggest you contact the seller as soon as
possible for arranging the pickup.</p>

        <hr>

        <p><strong>Transaction information:</strong></p>

        <div class="transaction-info">

            <p>Item: {item_title}</p>

            <p>Price: {price}</p>

            <p>Buyer: {data["buyer_user"]}</p>

            <p>Seller: {data["seller_user"]}</p>

            <p>Time And Date Of Transaction:
{data['time']}</p>

            <p>Transaction ID: {data["transaction_id"]}</p>

        </div>

        <hr>

    </td>

</tr>

</table>

</div>
```



```
</body>

</html>

'', subtype='html')

try:

    with smtplib.SMTP_SSL('smtp.gmail.com', 465) as smtp:

        smtp.login(sender_email, sender_password)

        smtp.send_message(msg)

        smtp.send_message(msg2)

    logging.info('Both mails sent')

    return "OK"

except Exception as e:

    logging.error(f'Failed to send email: error: {e}')

    return "FAIL"

# Initialize globals

code = ''

codes = []

secret_code = ''

secret_codes = []

server = Server_com()

comm_thread = None

if __name__ == "__main__":

    try:

        logging.info("Starting FanZone Marketplace server")

        comm_thread = threading.Thread(target=com_thread)

        comm_thread.daemon = False
```



```
comm_thread.start()

print("Server is running. Press Ctrl+C to stop.")

while True:
    time.sleep(1)
except KeyboardInterrupt:
    logging.info("Shutting down server...")
except Exception as e:
    logging.error(f"Fatal error: {e}")
finally:
    logging.info("Server stopped.")
```



server_communication.py

```
import socket

import threading

import select

import logging

import time

import os

from encryption_server import Server_Encryption

# Set up logging

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

class Server_com():

    def __init__(self):

        self.MAX_MSG_LENGTH = 10485760

        self.SERVER_PORT = 5678

        self.SERVER_IP = '0.0.0.0'

        self.iv = None

        self.inL = threading.Lock()

        self.outL = threading.Lock()

        self.en = Server_Encryption()

        logging.info("Setting up server...")

        try:

            self.server_socket = socket.socket(socket.AF_INET,

socket.SOCK_STREAM)

            # Allow socket to be reused immediately after closing

            self.server_socket.setsockopt(socket.SOL_SOCKET,

socket.SO_REUSEADDR, 1)

            self.server_socket.bind((self.SERVER_IP, self.SERVER_PORT))
```



```
        self.server_socket.listen(5)

        logging.info(f"Server listening on
{self.SERVER_IP}:{self.SERVER_PORT}")

    except Exception as e:

        logging.critical(f"Failed to start server: {e}")

        raise

self.client_sockets = []

self.client_secret_codes = []

self.out_msgs = []

self.in_msgs = []

def add_to_out(self, msg):

    self.outL.acquire()

    try:

        self.out_msgs.append(msg)

        logging.info(f"Queued message: {msg[1][:100]}...")

    finally:

        self.outL.release()

def run(self):

    try:

        # Use select with a timeout to avoid blocking indefinitely

        ready_to_read, ready_to_write, in_error = select.select(

            [self.server_socket] + self.client_sockets,

            self.client_sockets,

            [],

            0.1

        )
```



```
# Handle incoming connections and messages

self.inL.acquire()

try:
    for current_socket in ready_to_read:
        if current_socket is self.server_socket:
            try:
                (client_socket, client_address) =
current_socket.accept()

                client_socket.settimeout(5)

                logging.info(f"New client connected:
{client_address}")

                client_socket.send(str(self.en.public_parameter).encode())

                self.en.generate_secret_key(client_socket.recv(1024).decode())

                self.client_sockets.append(client_socket)
            except Exception as e:
                logging.error(f"Error accepting client
connection: {e}")
        else:
            logging.info(f"Receiving data from client:
{current_socket}")

            data = ''

            try:
                self.iv = current_socket.recv(16)

                if self.iv != "":
                    recv_data =
current_socket.recv(self.MAX_MSG_LENGTH)

                    if recv_data:
                        data = self.en.decrypt(recv_data,
self.iv)
                    else:
                        logging.info(f"Client disconnected
(empty recv): {current_socket}")
```



```
self.handle_client_disconnect(current_socket)

        continue

    except ConnectionResetError as e:

        logging.error(f"Connection reset: {e}")

        self.handle_client_disconnect(current_socket)

        continue

    except socket.timeout as e:

        logging.error(f"Socket timeout: {e}")

        continue

    except Exception as e:

        logging.error(f"Error receiving data: {e}")

        self.handle_client_disconnect(current_socket)

        continue

    if data == "" or self.iv == "":

        logging.info(f"Client disconnected (empty
string): {current_socket}")

        self.handle_client_disconnect(current_socket)

    else:

        logging.info(

            f"Received data: {data[:100]}..." if
len(data) > 100 else f"Received data: {data}")

        self.in_msgs.append((current_socket, data))

    finally:

        self.inL.release()

    self.outL.acquire()

    try:

        for message in self.out_msgs[:]:

            current_socket, data = message
```



```
        if current_socket in ready_to_write:
            try:
                logging.info(f"Sending {len(data)} bytes to
client")
                self.iv = os.urandom(16)
                current_socket.send(self.iv)
                current_socket.send(self.en.encrypt(data,
self.iv))
                self.out_msgs.remove(message)
            except Exception as e:
                logging.error(f"Error sending message: {e}")
                if current_socket in self.client_sockets:
self.handle_client_disconnect(current_socket)
                self.out_msgs.remove(message)
        finally:
            self.outL.release()

    except Exception as e:
        logging.error(f"Error in server run loop: {e}")
        import traceback
        traceback.print_exc()

def handle_client_disconnect(self, socket):
    try:
        if socket in self.client_sockets:
            self.client_sockets.remove(socket)

    codes_to_remove = []
    for code_entry in self.client_secret_codes:
        if ':' in code_entry:
```



```
        parts = code_entry.split(':')

        if len(parts) >= 1 and str(socket) in parts[0]:

            codes_to_remove.append(code_entry)

        else:

            logging.warning(f"Invalid secret code format:
{code_entry}")

    for code in codes_to_remove:

        self.client_secret_codes.remove(code)

        logging.info(f"Removed secret code: {code}")

    logging.info(f"Active clients: {len(self.client_sockets)}")
    logging.info(f"Active secret codes: {self.client_secret_codes}")

    try:

        socket.close()

    except Exception as e:

        logging.warning(f"Error closing socket: {e}")

except Exception as e:

    logging.error(f"Error handling client disconnect: {e}")

    import traceback

    traceback.print_exc()

def shutdown(self):

    logging.info("Shutting down server...")

    for client in self.client_sockets:

        try:

            client.close()

        except:
```



```
        pass

    try:

        self.server_socket.close()

    except:

        pass

logging.info("Server shutdown complete")
```



database.py

```
import sqlite3

import random

import logging

import os

import time

import requests

import base64

import bcrypt


logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')


class Database:

    def __init__(self):

        self.logged_users = []

        self.code = None

        self.processed_submissions = set()

        db_file = 'database.db'

        db_exists = os.path.exists(db_file)

        self.users_items = []

        self.api_client_id = '011c608399adfb9'

        max_attempts = 5

        for attempt in range(max_attempts):

            try:

                self.conn = sqlite3.connect(db_file, timeout=30)

                self.cursor = self.conn.cursor()

                break

            except sqlite3.OperationalError as e:
```



```
        if attempt < max_attempts - 1:

            logging.warning(f"Database connection attempt {attempt +
1} failed: {e}. Retrying...")

            time.sleep(1)

        else:

            logging.error(f"Failed to connect to database after
{max_attempts} attempts: {e}")

            raise

self.cursor.execute('''

    CREATE TABLE IF NOT EXISTS UserPass (

        User TEXT PRIMARY KEY,

        Password TEXT NOT NULL,

        Username TEXT NOT NULL,

        PhoneNumber TEXT NOT NULL

    )

''')

self.cursor.execute('''

    CREATE TABLE IF NOT EXISTS ItemData (

        Id INTEGER PRIMARY KEY AUTOINCREMENT,

        Item TEXT NOT NULL,

        Email TEXT NOT NULL,

        Status TEXT NOT NULL

    )

''')

self.cursor.execute('''

    CREATE TABLE IF NOT EXISTS TransactionHistory (

        Id INTEGER PRIMARY KEY AUTOINCREMENT,

        Item_title TEXT NOT NULL,

        Buyer TEXT NOT NULL,

        Seller TEXT NOT NULL,
```



```
        Date TEXT NOT NULL,

        Price INTEGER NOT NULL

    )

'''

self.conn.commit()

if not db_exists:

    logging.info("Database created successfully")

else:

    logging.info("Connected to existing database")

def upload_image(self, img):

    logging.info("Uploading image")

    try:

        headers = {"Authorization": f"Client-ID {self.api_client_id}"}

        data = {"image": img}

        response = requests.post("https://api.imgur.com/3/image",
headers=headers, data=data)

        response.raise_for_status()

        logging.info(f"Uploaded image
successfully{response.json() ["data"] ["link"]}")

        return response.json() ["data"] ["link"]

    except requests.exceptions.RequestException as e:

        logging.info(f"Request error: {e}")

    except KeyError:

        logging.info("Error: Could not extract image URL from the
response.")

    except Exception as e:

        logging.info(f"An unexpected error occurred: {e}")
```



```
def AddToUserPass(self, user, password, username, number):

    logging.info(f'Adding user: {user}, username: {username}')

    try:

        self.cursor.execute("SELECT User FROM UserPass WHERE User = ?",
(user,))

        result = self.cursor.fetchone()

        if result:

            logging.warning(f"User already exists: {user}")

            return "USER_EXISTS"

        self.cursor.execute('''

            INSERT INTO UserPass (User, Password, Username, PhoneNumber)

            VALUES (?, ?, ?, ?)

            ''', (user, password, username, number))

        self.conn.commit()

        logging.info(f"User added successfully: {user}")

        return "OK"

    except sqlite3.IntegrityError as e:

        logging.error(f"Database integrity error adding user: {e}")

        return "USER_EXISTS"

    except Exception as e:

        logging.error(f"Error adding user: {e}")

        return f"ERROR: {str(e)}"

def AddData(self, json_item, email):

    try:

        if not json_item or json_item.isspace():

            logging.error("Empty JSON item data")

            return "ERROR_EMPTY_DATA"
```



```
import json

try:
    data = json.loads(json_item)

    if "image" in data and isinstance(data["image"], str):
        img_len = len(data["image"])
        logging.info(f"Item contains image data of length
{img_len}")

        if not data.get("image"):
            logging.warning("Image data is empty, setting
placeholder")

            data["image"] = ""
            json_item = json.dumps(data)

    img_url = self.upload_image(data.get("image"))
    data["image"] = img_url
    json_item = json.dumps(data)
    self.cursor.execute('''
        INSERT INTO ItemData (Item, Email, Status)
        VALUES (?, ?, ?)
        ''', (json_item, email, "Available"))
    self.conn.commit()
    logging.info(f"Added new item with JSON length
{len(json_item)}")
    return "OK"

except json.JSONDecodeError as e:
    logging.error(f"Invalid JSON data: {e}")
    return f"ERROR_INVALID_JSON: {e}"
```



```
except sqlite3.IntegrityError as e:

    logging.error(f"Database integrity error adding item: {e}")

    return "Error"

except Exception as e:

    logging.error(f"Error adding item: {e}")

    import traceback

    traceback.print_exc()

    return f"ERROR: {str(e)}"


def GetData(self, email, page=1, items_per_page=12):

    try:

        offset = (page - 1) * items_per_page

        self.cursor.execute("SELECT COUNT(*) FROM ItemData")

        count = self.cursor.fetchone()[0]

        if count == 0:

            logging.info("No items found in database")

            return "NO_DATA"

        self.cursor.execute('''

            SELECT Id, Item FROM ItemData

            WHERE Status != ? AND Email != ?

            ORDER BY Id DESC

            LIMIT ? OFFSET ?

            ''', ("Sold", email, items_per_page, offset))

        results = self.cursor.fetchall()

        if not results:
```



```
        logging.info(f"No items found for page {page}")

        return "NO_DATA"

    import json

    items_list = []

    for row in results:

        item_id, item_json = row

        try:

            json.loads(item_json)

            items_list.append(f"{item_id}©{item_json}")

        except json.JSONDecodeError as e:

            logging.warning(f"Invalid JSON in item {item_id}: {e}")

            continue

    logging.info(f"Retrieved {len(items_list)} items for page {page}")

    return json.dumps(items_list)

except Exception as e:

    logging.error(f"Error retrieving data: {e}")

    import traceback

    traceback.print_exc()

    return "NO_DATA"

def GetDataForUser(self, email):

    try:

        self.cursor.execute("SELECT Id, Item, Status FROM ItemData WHERE Email = ?", (email,))
```



```
        results = self.cursor.fetchall()

        if not results:

            logging.info(f"No items found for {email}")

            return "NO_DATA"

        import json

        items_list = []

        for row in results:

            item_id, item_json, item_status = row

            try:

                json.loads(item_json)

            items_list.append(f"{item_id}©{item_json}©{item_status}")

            except json.JSONDecodeError as e:

                logging.warning(f"Invalid JSON in item {item_id}: {e}")

                continue

        logging.info(f"Retrieved {len(items_list)} items for {email}")

        return json.dumps(items_list)

    except Exception as e:

        logging.error(f"Error retrieving data: {e}")

        import traceback

        traceback.print_exc()

        return "NO_DATA"

def BuyItem(self, item_id, buyer, price, item_title):

    try:
```



```
self.cursor.execute('''

    UPDATE ItemData

    SET Status = 'Sold'

    WHERE Id = ?

''', (item_id,))

self.conn.commit()

self.cursor.execute('''

    SELECT Email FROM ItemData

    WHERE Id = ?

''', (item_id,))

seller_result = self.cursor.fetchone()

if not seller_result:

    logging.error(f"No item found with ID: {item_id}")

    return {"response": "ERROR", "message": "Item not found"}

seller_email = seller_result[0]

self.cursor.execute("SELECT Username, PhoneNumber FROM UserPass
WHERE User = ?", (buyer,))

buyer_contact_result = self.cursor.fetchall()

self.cursor.execute("SELECT Username, PhoneNumber FROM UserPass
WHERE User = ?", (seller_email,))

seller_contact_result = self.cursor.fetchall()

if not buyer_contact_result:

    logging.error(f"No buyer contact found for: {buyer}")

    return {"response": "ERROR", "message": "Buyer not found"}
```



```
        if not seller_contact_result:

            logging.error(f"No seller contact found for:
{seller_email}")

            return {"response": "ERROR", "message": "Seller not found"}

        buyer_contact = buyer_contact_result[0]

        seller_contact = seller_contact_result[0]

    from datetime import datetime

    time_of_purchase = datetime.now().strftime("%d-%m-%Y %H:%M")

    transaction_id = random.randint(100000, 999999)

    data = {

        "seller_mail": seller_email,

        "seller_user": seller_contact[0],

        "seller_phone": seller_contact[1],

        "buyer_user": buyer_contact[0],

        "buyer_phone": buyer_contact[1],

        "time": time_of_purchase,

        "transaction_id": transaction_id,

        "response": "OK",

    }

    self.cursor.execute('''

        INSERT INTO TransactionHistory (Id, Item_title, Buyer,
Seller, Date, Price)

        values (?, ?, ?, ?, ?, ?)

        ''', (transaction_id, item_title, buyer, seller_email,
time_of_purchase, price))

    self.conn.commit()
```



```
        logging.info(f"Transaction {transaction_id} completed successfully")

        return data

    except Exception as e:

        logging.error(f"Error in BuyItem: {e}")

        import traceback

        traceback.print_exc()

        return {"response": "ERROR", "message": str(e)}


def login_compare(self, user_password):

    try:

        parts = user_password.split(' ', 1)

        if len(parts) != 2:

            logging.warning(f"Invalid login format: {user_password}")

            return "USER_OR_PASS_INVALID"

        username, password = parts

        self.cursor.execute("SELECT User, Password FROM UserPass WHERE
User = ?", (username,))

        result = self.cursor.fetchone()

        hashed_pass = result[1][2:-1]

        if result and bcrypt.checkpw(password.encode(),
hashed_pass.encode()):

            logging.info(f"Successful login for user: {username}")

            return f"LOGIN_SUCCESS"

        else:
```



```
        logging.warning(f"Failed login attempt for user:
{username}")

        return "USER_OR_PASS_INVALID"

    except Exception as e:

        logging.error(f"Error during login: {e}")

        return "USER_OR_PASS_INVALID"

def get_user(self, email):

    try:

        self.cursor.execute("SELECT Username FROM UserPass WHERE User =
?", (email,))

        result = self.cursor.fetchone()

        if result:

            logging.info(f"Retrieved username for email: {email}")

            return result[0]

        else:

            logging.warning(f"No user found for email: {email}")

            return "Unknown User"

    except Exception as e:

        logging.error(f"Error retrieving username: {e}")

        return "Error"

def Disconnect(self):

    try:

        self.conn.commit()

        self.conn.close()

        logging.info("Database connection closed")

    except Exception as e:

        logging.error(f"Error disconnecting from database: {e}")
```



encryption_server

```
import random

import logging

from hashlib import sha256

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms,
modes

import os

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s
- %(message)s')

class Server_Encryption:

    def __init__(self):

        self.p = 23

        self.g = 5

        self.private_parameter = random.randint(1, self.p-1)

        self.public_parameter = pow(self.g, self.private_parameter, self.p)

        self.secret = None

        self.key = None

    def generate_secret_key(self, client_parameter):

        self.secret = pow(int(client_parameter), self.private_parameter,
self.p)

        self.key = sha256(str(self.secret).encode()).digest()[0:16]

    def encrypt(self, message, iv):

        if isinstance(message, str):

            message = message.encode('utf-8')

        from cryptography.hazmat.primitives import padding
```



```
    padder = padding.PKCS7(128).padder()

    padded_data = padder.update(message)

    padded_data += padder.finalize()

    cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv))
    encryptor = cipher.encryptor()

    return encryptor.update(padded_data) + encryptor.finalize()

def decrypt(self, message, iv):

    cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv))
    decryptor = cipher.decryptor()

    padded_data = decryptor.update(message) + decryptor.finalize()

    from cryptography.hazmat.primitives import padding
    unpadder = padding.PKCS7(128).unpadder()

    data = unpadder.update(padded_data)

    data += unpadder.finalize()

    return data.decode('utf-8')
```